

نسخهٔ 1.8.1 · راهنمای کامل کاربر

Tehran Stock Exchange Data & Analytics Library

کتابخانهٔ داده و تحلیل بازار سرمایهٔ ایران

Stocks · Live Market · Options · ETFs · Debt · Gold & FX

سهام · بازار زنده · اختیار معامله · صندوق · اراد و گام · طلا و ارز

Python 3.8–3.14

25 October 2026

algotik.com

Mohsen Alipour · @algotik

راهنمای مطالعه

چطور از این دفترچه استفاده کنید؟

این راهنما دو لایه دارد: ابتدا مسیرهای سریع و توابعی که بیشتر کاربران هر روز نیاز دارند؛ سپس مرجع کامل ورودی‌ها، گزینه‌ها، خروجی‌ها و نکات رفتاری همهٔ APIهای عمومی پکیج.

۱
شروع سریع

نصب، import و اولین دریافت تاریخچهٔ قیمت در چند دقیقه.

۲
توابع پرکاربرد

قیمت، بازار زنده، سفارش، طلا و ارز، صندوق، اخزا، اراد، گام و اختیار.

۳
قراردادهای داده

نوع خروجی، تاریخ، freshness، ردیف امروز، CSV و مرز منابع داده.

۴
مرجع تفصیلی

signatureها، تمام ورودی‌ها، مقدارهای مجاز، پیش‌فرض‌ها و خروجی هر خانوادهٔ تابع.

نکتهٔ معاملاتی: پیش از استفاده در تصمیم‌گیری یا اجرای الگوریتم، زمان snapshot، ویژگی‌های `DataFrame.attrs`، وضعیت `partial` و freshness داده را بررسی کنید. این پکیج توصیهٔ سرمایه‌گذاری ارائه نمی‌کند.

A Python toolkit for historical, live and analytical data from Iran's capital market.

Fetch TSETMC prices, client type, trades, order books, market calendars, index impact, instrument reference data, funds, bonds, options, Iran Energy Exchange auctions, IME commodity markets and official subscriber TOP with Jalali date support, then use explainable comparison, liquidity, market-regime and market-map analytics for research and algorithmic trading.

معرفی فارسی IR

`algotik-tse` برای دریافت و تحلیل داده‌های بازار سرمایهٔ ایران ساخته شده است. با نام فارسی نماد می‌توانید تاریخچهٔ قیمت، حقیقی/حقوقی، معاملات ریز، سفارش‌ها و اطلاعات لحظه‌ای بازار را بگیرید؛ شاخص‌های صنعت و اعضای دقیق آن‌ها را تحلیل کنید؛ حراج و ابزارهای بورس انرژی و تابلوها و بازار فیزیکی بورس کالا را بخوانید؛ اراد، گام، اخزا، صندوق و اختیارها را بسازید؛ و دریافت کنید TGJU تاریخچهٔ ۸۷ سری تأییدشدهٔ ارز، طلا، نقره، سکه و فلزات گران‌بها را از.

بیشتر خروجی‌های جدولی به‌صورت **Pandas DataFrame** ارائه می‌شوند و تاریخ شمسی، تاریخ میلادی و داده‌های چندنمادی پشتیبانی می‌شوند. خروجی‌های ساختاریافته‌ای مانند snapshot بازار، زنجیرهٔ اختیار و منحنی بازده در بخش نوع خروجی توضیح داده شده‌اند.

ویژگی‌ها

- دریافت تاریخچهٔ قیمت و حقیقی/حقوقی با نماد فارسی، تاریخ شمسی/میلادی، تعدیل قیمت، بازده و خروجی چندنمادی
- افزودن کنترل‌شدهٔ ردیف امروز با `include_today=True` و metadata مربوط به freshness و partial بودن داده
- نمای زندهٔ کل بازار یا یک نماد، قدرت خریدار، جریان پول، spread و imbalance پنج سطح سفارش
- معاملات ریز، order book و صف خرید/فروش به‌صورت زنده و تاریخی
- تقویم معاملاتی بورس/فرابورس، ارزش بازار تاریخی، آمار فعالیت روزانه و اثر هر نماد بر شاخص
- دریافت یکجای خلاصهٔ روزانهٔ همهٔ ابزارها و فهرست مرجع کامل TSETMC با تشخیص تغییرات بر پایهٔ `InsCode`
- قیمت تئوریک گشایش رسمی (TOP) برای مشترکان وب‌سرویس TSETMC و محاسبهٔ شفاف عدم تعادل پیش‌گشایش
- SQLite جریان صنایع و ذخیرهٔ اختیاری تاریخچه روی breadth، بازار، پیام‌ها، تغییر وضعیت watcher
- ۴۵ شاخص صنعت، اعضای رسمی، snapshot تحلیلی، تاریخچهٔ اعضا، کندل درون‌روزی، مقایسه و هم‌بستگی صنایع
- `InsCode` رخدادهای تعدیل قیمت و حل دقیق هویت ابزار با P/E، و EPS
- سهامداران عمدهٔ هر نماد با تفکیک تاریخ معامله/تاریخ مؤثر، تغییر دارایی اصلاح‌شده و تاریخچهٔ روزانه، هفتگی یا ماهانه با بودجهٔ درخواست
- پنج‌روزهٔ فعالیت سهامداران عمدهٔ کل بازار، تغییرات روزانه، رتبه‌بندی انباشت/توزیع، شبکهٔ سهامدار–نماد، تمرکز snapshot هر نماد TSETMC رویدادهای timeline مالکیت و
- صندوق‌های ثبت‌شده و بورسی با طبقه‌بندی چندمحوره، پوشش ابزارهای ۳۰۵ و ۳۸۰ و فیلتر مستقیم طلا، نقره، کالایی، اهرمی، بخشی و شاخصی
- فهرست و فیلتر دقیق اوراق `اراد` و `گام` با سررسید شمسی/میلادی و جدول رسمی YTM فرابورس، بدون حدس جریان نقدی

- تحلیل اخزا شامل DV01، convexity، duration، YTM و منحنی بازده زنده و تاریخی
- تحلیل اختیار معامله شامل PCR، put-call parity، Greeks، IV، Black–Scholes و نقدشوندگی
- حراج‌های بورس انرژی با شرایط پرداخت و تحویل، حجم، دامنهٔ قیمت، اندازهٔ lot، قیمت کشف‌شده و معاملات همان حراج
- بازار برق استاندارد، سبز و آزاد؛ سلف، گواهی سپردهٔ انرژی، گواهی ظرفیت، گواهی صرفه‌جویی و قرارداد آتی انرژی
- تابلوی رسمی بورس کالا برای گواهی سپرده، سلف استاندارد، صندوق کالایی و آتی همراه پنج سطح سفارش و حقیقی/حقوقی
- تاریخچه و خلاصهٔ بازار فیزیکی بورس کالا، نسبت معامله به عرضه و آمار تجمیعی بازارهای فیزیکی، آتی، اختیار و مالی
- منحنی سررسید آتی، calendar spread و تحلیل cash-and-carry با فرض‌های صریح نرخ، انبارداری و convenience yield
- مقایسهٔ هم‌زمان نمادها از نظر بازده، ریسک، افت سرمایه، نقدشوندگی، بتا و جریان پول حقیقی
- سنجه‌های نقدشوندگی تاریخی و زنده شامل Amihud، گردش، spread، عمق پنج‌سطحی و روزهای بدون معامله
- تشخیص توضیح‌پذیر وضعیت `neutral` ، `risk_on` یا `risk_off` بازار با نمایش امتیاز و وزن هر مؤلفه
- داده و نمودار نقشهٔ بازار با گروه‌بندی نماد، صنعت، بازار یا نوع ابزار و معیارهای قابل انتخاب اندازه/رنگ
- کاتالوگ ۸۷ سری TGJU: ۳۶ ارز آزاد، ۲۴ نرخ رسمی، طلای ۱۸/۲۴ عیار، مثقال و آب‌شده، نقره، سکه و حباب، و فلزات جهانی
- اینترادی، سهامداران، شاخص‌ها، ETFها، صندوق‌ها و اوراق بدهی
- و محدودسازی منبع داده retry، rate limiting، به‌صورت پیش‌فرض TLS و اعتبارسنجی HTTPS

🌐 **وب‌سایت: algotik.com** | 📱 **تلگرام: t.me/algotik**

> این کتابخانه توصیهٔ سرمایه‌گذاری نیست. برای استفادهٔ معاملاتی، زمان partial، freshness، snapshot بودن داده و `DataFrame.attrs` را بررسی کنید.

API در یک نگاه

اگر اولین بار است از پکیج استفاده می‌کنید، معمولاً همین توابع نیاز شما را پوشش می‌دهند:

تابع	چه کاری انجام می‌دهد؟	خروجی
`get_history()` ⭐	تاریخچهٔ OHLCV یک یا چند نماد	`DataFrame`
`get_client_type()` ⭐	تاریخچهٔ خریدوفروش حقیقی/حقوقی	`DataFrame`
`get_live_symbol()` ⭐	نمای زنده و تحلیلی یک نماد	یک‌ردیفی `DataFrame`
`get_live_market()` ⭐	نمای زنده و تحلیلی کل بازار	`DataFrame`
`get_market_snapshot()` ⭐	خام قیمت و پنج سطح snapshot سفارش کل بازار	`dict`
`get_intraday()` ⭐	تیک یا کندل اینترادی	قدیمی API در `None` یا `DataFrame`

تابع	چه کاری انجام می‌دهد؟	خروجی
`get_symbols()` ⭐	فهرست نمادها بر اساس بازار و نوع ابزار	یا فهرست `DataFrame`
`get_tgju_history()` ⭐	تاریخچهٔ طلا، ارز، نقره، سکه، حباب و فلزات جهانی	`DataFrame`
`list_tgju_assets()` ⭐	کاتالوگ نام‌ها، slugها، دسته و واحد ۸۷ سری TGJU	`DataFrame`
`list_debt_instruments()` ⭐	فهرست و فیلتر `gam` ، `erad` ، اخزا و سایر اوراق	`DataFrame`
`get_debt_yields()` ⭐	مرجع رسمی فرابورس برای اراد YTM و گام	`DataFrame`
`compare_symbols()` ⭐	مقایسهٔ بازده، ریسک، نقدشوندگی و جریان پول چند نماد	`DataFrame`
`get_liquidity_metrics()` ⭐	سنجه‌های نقدشوندگی تاریخی و زنده	`DataFrame`
`get_market_regime()` ⭐	رژیم توضیح‌پذیر بازار و پنج امتیاز سازنده	یک‌ردیفی `DataFrame`
`get_market_map()` ⭐	دادهٔ مستقل از renderer برای نقشهٔ بازار	`DataFrame`
`plot_market_map()` ⭐	نمودار تعاملی treemap با Plotly	`plotly.graph_objects.Figure`
`get_energy_auctions()` ⭐	حراج‌های بورس انرژی با شرایط کامل عرضه	`DataFrame`
`list_power_instruments()` ⭐	قراردادهای برق استاندارد، سبز و آزاد	`DataFrame`
`list_energy_securities()` ⭐	سلف، گواهی‌ها و آتی انرژی	`DataFrame`
`get_commodity_market()` ⭐	تابلوی گواهی، سلف، صندوق و آتی بورس کالا	`DataFrame`
`get_commodity_physical_history()` ⭐	حجم و ارزش روزانهٔ بازار فیزیکی بورس کالا	`DataFrame`
`get_commodity_physical_summary()` ⭐	خلاصهٔ تالارها و نسبت معامله به عرضه	`DataFrame`
`get_futures_curve()` ⭐	منحنی سررسید و basis قراردادهای آتی	`DataFrame`
`get_live_trades()` ⭐	معاملات ریز امروز یک نماد	`DataFrame`
`get_order_book()` ⭐	پنج سطح سفارش زنده	`DataFrame`
`get_trading_calendar()` ⭐	تقویم معاملاتی رسمی بورس و فرابورس	`DataFrame`
`get_market_activity()` ⭐	تعداد، حجم و ارزش معاملات روزانهٔ کل بازار	`DataFrame`

تابع	چه کاری انجام می‌دهد؟	خروجی
`get_market_value_history()` ⭐	تاریخچهٔ ارزش بازار بورس و فرابورس	`DataFrame`
`get_index_impact()` ⭐	نمادهای اثرگذار مثبت و منفی روی شاخص	`DataFrame`
`get_market_trades()` ⭐	خلاصهٔ روزانهٔ همهٔ ابزارها در یک درخواست	`DataFrame`
`get_shareholders()` ⭐	سهامداران عمدهٔ فعلی یا پس از پایان یک روز معاملاتی	`DataFrame`
`get_shareholder_history()` ⭐	تاریخچهٔ ترکیب سهامداران عمدهٔ یک نماد با درخواست‌های جفتی	`DataFrame`
`get_major_shareholder_changes()` ⭐	تغییرات سهامداران عمده در پنجرهٔ پنج‌روزهٔ TSETMC	`DataFrame`
`get_active_shareholders()` ⭐	جمع‌بندی انباشت و توزیع سهامداران فعال	`DataFrame`
`rank_shareholder_accumulation()` ⭐	رتبه‌بندی فعالیت هر جفت سهامدار–نماد بر پایهٔ درصد یا تعداد سهم	`DataFrame`
`get_shareholder_network()` ⭐	شبکهٔ دوبخشی سهامداران و نمادها در آخرین snapshot	`DataFrame` edge list
`get_ownership_concentration()` ⭐	سهامداران عمدهٔ یک HHI و Top-N نماد	یک‌ردیفی `DataFrame`
`get_symbol_events()` ⭐	پیام ناظر، افزایش سرمایه و timeline تعدیل قیمت یک نماد	`DataFrame`
`get_industry_snapshot()` ⭐	بازده، breadth، جریان پول و صف یک یا همهٔ صنایع	`DataFrame`
`compare_industries()` ⭐	مقایسهٔ سری زمانی چند شاخص صنعت	`DataFrame`
`get_industry_relative_strength()` ⭐	محاسبهٔ برتری نسبی به مقابل شاخص مبنا	`DataFrame`
`get_industry_correlation()` ⭐	ماتریس همبستگی بازدهی روزانهٔ صنایع	`DataFrame`
`get_industry_membership_overlap()` ⭐	اندازه‌گیری همپوشانی رسمی اعضای صنایع	`DataFrame`
`get_industry_membership_events()` ⭐	ردیف‌های `added` / `dropped` عضویت رسمی صنعت	`DataFrame`
`get_industry_membership_churn()` ⭐	شاخص تغییرپذیری اعضا بین جلسات اخیر	`DataFrame`
`get_industry_concentration()` ⭐	شاخص تمرکز و متمرکزشدن سرمایهٔ صنعت	`DataFrame`

تابع	چه کاری انجام می‌دهد؟	خروجی
`get_industry_momentum_profile()` ⭐	پروفایل مومنتوم چنددوره‌ای برای صنایع	`DataFrame`
⭐ `get_industry_correlation_neighborhood()`	همبستگی همسایگی یک صنعت با نزدیک‌ترین‌ها	`DataFrame`
`get_industry_health_score()` ⭐	نمرهٔ سلامت ترکیبی برای رتبه‌بندی صنایع	`DataFrame`
`rank_industries()` ⭐	رتبه‌بندی صنایع با معیار انتخابی	`DataFrame`
`list_etfs()` ⭐	NAV ها همراه قیمت وETF	`DataFrame`
`list_funds()` ⭐	صندوق‌ها همراه NAV، بازده و ترکیب دارایی	`DataFrame`
`list_listed_funds()` ⭐	صندوق‌های بورسی و فیلتر نوع دارایی، استراتژی و کالای پایه	`DataFrame`

نقشهٔ کامل توابع کاربری

جدول‌های زیر APIهای canonical را نشان می‌دهند. نام‌های قدیمی مانند `stock()` و `stock_RI()` همچنان کار می‌کنند، اما برای کد جدید نام‌های `get_*` پیشنهاد می‌شوند.

قیمت، حقیقی/حقوقی و اطلاعات نماد

تابع	کاربرد
`get_history()`	تاریخچهٔ قیمت تعدیل‌شده/خام، بازده و دادهٔ چندنمادی
`get_client_type()`	تاریخچهٔ حقیقی/حقوقی و قدرت خریدار
`get_intraday()`	تیک و کندل‌های ۱ دقیقه تا ۱۲ ساعت
`get_live_trades()` , `get_trades()`	معاملات ریز تاریخی و امروز
`get_stats()` , `get_info()` , `get_detail()`	جزئیات، اطلاعات و آمار TSETMC نماد
`get_shareholders()`	سهامداران عمدهٔ فعلی یا تاریخی
`get_shareholder_history()`	های تاریخی یک نماد روی تاریخ مؤثر، با فرکانس و بودجهٔsnapshot درخواست
`get_major_shareholder_snapshots()`	های منتشرشدهٔ موارد فعال در تابلوی تغییرات پنج‌روزهsnapshot
`get_major_shareholder_changes()`	اختلاف دارایی بین snapshotهای متوالی سهامداران عمده
`get_active_shareholders()`	جمع‌بندی ورود/خروج موارد فعال در پنجرهٔ اخیر
`rank_shareholder_accumulation()`	رتبه‌بندی بدون جمع‌زدن تعداد سهم نمادهای ناهمگون
`get_shareholder_network()`	اخیر snapshot شبکهٔ سهامدار–ابزار در یک edge list
`get_ownership_concentration()`	تمرکز فهرست افشاشدهٔ سهامداران عمدهٔ یک نماد

تابع	کاربرد
`get_capital_increase()`	تاریخچهٔ افزایش سرمایه
`get_price_adjustments()`، `get_latest_price_adjustment()`	رخدادهای تعدیل قیمت
`get_introduction()`	فقط سازگاری قدیمی؛ همیشه `UnsupportedDataSourceError`

دادهٔ زنده، سفارش و تحلیل بازار

تابع	کاربرد
`get_market_snapshot()`	خام و اتمیک کل بازار snapshot
`get_market_client_type()`	حقیقی/حقوقی bulk کل بازار
`get_live_symbol()`، `get_live_market()`	نمای زندهٔ ادغام‌شدهٔ بازار یا یک نماد
`get_queue()`، `get_order_book()`	پنج سطح سفارش و صف زنده
`get_order_book_history()`، `get_queue_history()`	تاریخچهٔ بازسازی‌شدهٔ سفارش و صف
`get_market_messages()`	پیام‌های ناظر بازار
`get_instrument_state_changes()`	تغییر وضعیت ابزارها
`get_symbol_events()`	برای پیام ناظر، تغییر سرمایه و تعدیل قیمت یک نماد TSETMC بومی timeline
`get_market_overview()`	نمای کلی رسمی بازار
`get_market_breadth()`	و شمار نمادهای مثبت/منفی breadth، A/D
`get_sector_flow()`	و جریان پول به تفکیک صنعت breadth
`MarketWatcher`، `watch_market()`	پایش افزایشی بازار و تولید `MarketEvent`
`get_trading_calendar()`	جلسات معاملاتی منتشرشدهٔ بورس و فرابورس
`get_market_activity()`	تجمیع روزانهٔ تعداد ابزار، معامله، حجم و ارزش
`get_market_value_history()`	تاریخچهٔ رسمی ارزش بازار با تغییر روزانه
`get_index_impact()`	اثر نمادها بر شاخص کل هر بازار attribution
`get_market_trades()`	خلاصهٔ bulk روزانهٔ ابزارها؛ معاملات ریز نیست
`get_theoretical_opening_price()`	TSETMC رسمی برای مشترکان وب‌سرویس TOP
`get_preopen_imbalance()`	عدم تعادل حجم خرید و فروش روی خروجی TOP
`compare_symbols()`	مقایسهٔ چند نماد با benchmark از نظر بازده، ریسک، افت سرمایه، نقدشوندگی و جریان حقیقی

تابع	کاربرد
`get_liquidity_metrics()`	نقدشوندگی تاریخی و snapshot زنده با Amihud، گردش، imbalance، spread و عمق سفارش
`get_market_regime()`	طبقه‌بندی توضیح‌پذیر `risk_on/neutral/risk_off` از پنج مؤلفهٔ قابل مشاهده
`get_market_map()`	ساخت جدول نقشهٔ بازار برای نماد، صنعت، flow یا نوع ابزار
`plot_market_map()`	رندر treemap تعاملی از جدول نقشه با وابستگی اختیاری Plotly

شاخص‌ها و تحلیل صنایع

تابع	کاربرد
`list_industry_indices()`	فهرست ۴۵ شاخص صنعت و وضعیت فعلی آن‌ها
`get_industry_members()`	اعضای رسمی یک شاخص با اتصال دقیق `InsCode` به دادهٔ زنده
`get_industry_snapshot()`	یک ردیف تحلیلی برای هر صنعت: شاخص، breadth، معامله، حقیقی و صف
`get_industry_history()`	تاریخچهٔ روزانهٔ خود شاخص صنعت بدون حجم ساختگی
`get_industry_members_history()`	تاریخچهٔ کوتاه همهٔ اعضای فعلی صنعت در قالب long-form
`get_industry_intraday()`	مشاهدات خام یا کندل‌های درون‌روزی شاخص صنعت
`compare_industries()`	مقایسه سری زمانی شاخص‌ها و متریک‌های قیمت/بازده
`get_industry_relative_strength()`	محاسبهٔ بازده تجمعی نسبی هر شاخص در برابر benchmark
`get_industry_correlation()`	ماتریس همبستگی بازده روزانه بین چند صنعت
`get_industry_membership_overlap()`	همپوشانی رسمی اعضای چند شاخص صنعت را pairwise محاسبه می‌کند
`get_industry_membership_events()`	رویدادهای `added` / `dropped` تغییرات رسمی اعضا
`get_industry_membership_churn()`	اعضای رسمی صنعت طی چند جلسه اخیر churn
`get_industry_concentration()`	تمرکز روی چند عضو بزرگ و وزن سهمی، HHI
`get_industry_momentum_profile()`	پروفایل مومنتوم چند پنجره‌ای برای هر صنعت
`get_industry_correlation_neighborhood()`	نزدیک‌ترین همبستگی‌ها برای یک صنعت هدف
`get_industry_health_score()`	نمرهٔ سلامت ترکیبی breadth, momentum و concentration
`rank_industries()`	رتبه‌بندی صنایع بر اساس بازده، breadth، ارزش یا جریان پول

تاریخچهٔ محلی و فاندامنتال

تابع	کاربرد
`load_market_snapshots()` , `save_market_snapshot()`	ذخیره و خواندن snapshotهای SQLite

تابع	کاربرد
`get_live_market_history()`	تاریخچهٔ نمای زندهٔ ذخیره‌شده
`get_market_overview_history()`	تاریخچهٔ overview رسمی
`get_market_snapshot_summary_history()`	خلاصهٔ snapshotهای ذخیره‌شده
`get_sector_flow_history()` ، `get_market_breadth_history()`	تاریخچهٔ تحلیل بازار و صنایع
`get_market_event_history()` ، `record_market_event()`	ثبت و replay رخدادهای watcher
`archive_market_records()`	آرشیو batch رکوردهای مستقل
`get_market_messages_history()`	پیام‌های آرشیوشدهٔ بازار
`get_instrument_state_changes_history()`	تغییر وضعیت‌های آرشیوشده
`check_market_history()`	بررسی سلامت و سازگاری فایل SQLite
`get_market_fundamentals()`	بازار P/E snapshot و EPS
`get_market_fundamentals_history()`	تاریخچهٔ EPS/P/E بدون look-ahead

فهرست ابزارها و بازارها

تابع	کاربرد
`get_symbols()`	فهرست سهام، حق‌تقدم، صندوق، اوراق و اختیار
`get_instrument_master()`	جدول مرجع کامل ابزارها با ISIN و `InsCode`
`get_instrument_changes()`	مقایسهٔ دو snapshot مرجع و گزارش افزوده/حذف/تغییر
`list_options()`	فهرست قراردادهای اختیار فعال
`get_options_chain()`	زنجیرهٔ call/put یک دارایی پایه
`list_etfs()`	مسیر قدیمی صندوق‌های بورسی عمومی نوع ۳۰۵ همراه NAV و discount/premium
`list_funds()`	صندوق‌های ثبتی با NAV، بازده، ترکیب دارایی و دسته/استراتژی قابل فیلتر
`list_listed_funds()`	ابزارهای بورسی دقیق، شامل صندوق‌های عمومی و کالایی با فیلتر طلا/نقره
`list_bonds()`	اوراق بدهی همراه سررسید
`list_debt_instruments()`	نمای جدید اوراق با `DebtType` دقیق و فیلتر `gam` / `erad`
`get_index_companies()` ، `list_indices()`	های بالا پیشنهاد می‌شوندAPI قدیمی شاخص‌ها و اعضای شاخص؛ برای صنعت API
`list_tgju_assets()`	کاتالوگ تأییدشدهٔ دارایی‌های TGJU، دسته و واحد هر سری
`get_currency()` ، `get_tgju_history()`	تاریخچهٔ ارز، طلا، نقره، سکه/حباب و فلزات جهانی؛ نام قدیمی حفظ شده است

بورس انرژی و بورس کالا

تابع	کاربرد
`get_energy_auctions()`	فهرست عرضه‌های آماده، فعال، مازاد یا پایان‌یافته در تابلوهای فیزیکی و برق
`get_energy_auction()`	جزئیات یک حراج، معاملات خود حراج و در صورت درخواست سابقهٔ همان ابزار
`get_energy_market_overview()`	آمار روزانهٔ جریان‌های رسمی بورس انرژی و بازار برق
`list_power_instruments()`	برق استاندارد، برق سبز و برق آزاد
`list_energy_securities()`	سلف استاندارد، گواهی ظرفیت/صرفه‌جویی/سپرده و آتی انرژی
`get_energy_future_contract()`	مشخصات قرارداد، سررسید، وجه تضمین، کارمزد و قواعد تحویل آتی انرژی
`get_commodity_market()`	برای گواهی، سلف، صندوق کالایی و آتی IME رسمی snapshot
`get_commodity_physical_history()`	سری روزانهٔ حجم و ارزش بازار فیزیکی در بازهٔ صریح
`get_commodity_physical_summary()`	تجمیع تالارهای فیزیکی و نسبت معامله به عرضه
`get_commodity_market_activity()`	فعالیت تجمیعی بازار فیزیکی، آتی، اختیار، مالی و گواهی
`get_futures_curve()`	ساخت term structure از ردیف‌های صریح قرارداد
`get_calendar_spreads()`	contango/backwardation سررسیدهای مجاور و تشخیص spread
`analyze_cash_and_carry()`	مقایسهٔ قیمت آتی و ارزش منصفانه با فرض‌های صریح هزینهٔ حمل

اخزا و درآمد ثابت

تابع	کاربرد
`parse_treasury_maturity()`	استخراج سررسید از نماد اخزا
`day_count_fraction()`	محاسبهٔ فاصلهٔ زمانی با day-count convention
`treasury_yield()`	بازده اخزا از قیمت و سررسید
`bond_price()` ، `yield_to_maturity()`	قیمت اوراق و حل YTM
`bond_analytics()`	duration، convexity و DV01
`build_yield_curve()`	ساخت شیء `YieldCurve` از nodeها
`get_ifb_yield_table()`	جدول مرجع YTM فرابورس
`get_debt_yields()`	جدول رسمی همهٔ اوراق با فیلتر دقیق `erad` ، `treasury` و `gam`
`get_treasury_yields()`	اخزا و تحلیل بازده snapshot
`get_treasury_yield_history()`	تاریخچهٔ YTM اخزا
`get_yield_curve_history()` ، `get_yield_curve()`	منحنی بازده زنده و تاریخی

اختیار معامله

تابع	کاربرد
`black_scholes_price()`	قیمت بلک–شولز اروپایی
`black_scholes_greeks()`	Delta، Gamma، Vega، Theta و Rho
`option_price_bounds()`	کران‌های بدون آربیتراژ
`implied_volatility()`	حل نوسان ضمنی
`get_option_market()`	اتمیک بازار اختیار snapshot
`analyze_option_chain()`	IV، Greeks، parity و نقدشوندگی
`option_put_call_ratios()`	حجم، ارزش و موقعیت باز PCR
`get_option_history()`	تاریخچهٔ قرارداد اختیار
`save_option_snapshot()` , `load_option_snapshots()`	ذخیره و خواندن snapshotهای JSON

هویت ابزار و ابزارهای کمکی

تابع	کاربرد
`resolve_instrument()`	تبدیل نماد یا `InsCode` به `InstrumentRef` دقیق
`validate_ins_code()`	اعتبارسنجی شناسهٔ ابزار
`normalize_instrument_text()`	یکسان‌سازی ی/ک و فاصله‌های فارسی

برای signature و همهٔ ورودی‌های هر تابع به مرجع تفصیلی همهٔ توابع مراجعه کنید.

فهرست مطالب

- نصب و به‌روزرسانی
- شروع سریع
- راهنمای توابع پرکاربرد
- قراردادهای مهم داده
- حل دقیق هویت نماد
- قیمت و حقیقی/حقوقی
- معاملات ریز
- سفارش و صف
- و تحلیل کل بازار Watcher

- تقویم، نمای کلان، مستر ابزار و پیش‌گشایش
- تحلیل مدیریتی بازار
- شاخص‌ها و تحلیل صنایع
- تاریخچهٔ محلی SQLite
- فاندامنتال، صندوق و تعدیل قیمت
- اخزا و درآمد ثابت
- اختیار معامله
- بورس انرژی و بورس کالا
- سایر APIهای بازار
- تنظیمات و خطاها
- مرجع تفصیلی همهٔ توابع
- مثال‌های کاربردی
- تست و مشارکت
- تاریخچهٔ تغییرات

نصب و به‌روزرسانی

```
pip install algotik-tse
```

برای ارتقا به آخرین نسخه:

```
python -m pip install --upgrade algotik-tse
```

نیازمندی نسخهٔ فعلی: `3.8` Python تا `3.14` .

برای نصب نسخهٔ توسعه:

```
git clone https://github.com/mohsenalipour/algotik_tse.git
cd algotik_tse
python -m pip install -e ".[dev]"
```

شروع سریع

```
import algotik_tse as att
```

نیاز شما	تابع پیشنهادی
تاریخچهٔ قیمت تعدیل‌شده	`att.get_history("شتران")`
حقیقی/حقوقی یک نماد	`att.get_client_type("شتران")`
اطلاعات زندهٔ یک نماد	`att.get_live_symbol("شتران")`
لحظه‌ای کل بازار snapshot	`att.get_market_snapshot()`
کندل‌های اینترادی	`att.get_intraday("شتران", interval="5min")`
معاملات ریز امروز	`att.get_live_trades("شتران")`
پنج سطح سفارش	`att.get_order_book("شتران")`
تقویم روزهای معاملاتی	`att.get_trading_calendar(market="tse")`
ارزش و فعالیت تاریخی بازار	`att.get_market_activity()` و `att.get_market_value_history()`
اثرگذارترین نمادهای شاخص	`att.get_index_impact(top=10)`
خلاصهٔ همهٔ ابزارهای یک روز	`att.get_market_trades("1405-06-18")`
تصویر تحلیلی صنایع	`att.get_industry_snapshot()`
رتبه‌بندی صنایع	`att.rank_industries(metric="breadth")`
فهرست نمادها و ابزارها	`att.get_symbols()`
طلای ۱۸/۲۴ عیار، ارز و سکه	`att.get_tgju_history("gold-18k")`
دیدن همهٔ سری‌های TGJU	`att.list_tgju_assets()`
صندوق‌های ETF با NAV	`att.list_etfs()`
اخزا و YTM	`att.get_treasury_yields()`
اراد و گام	`att.list_debt_instruments(["erad", "gam"])`
مرجع اراد و گام YTM	`att.get_debt_yields(["erad", "gam"])`
زنجیره و تحلیل اختیار	`att.analyze_option_chain()` و `att.get_option_market()`
عرضه‌های بورس انرژی	`att.get_energy_auctions(top=10)`
برق سبز، استاندارد و آزاد	`att.list_power_instruments()`
سلف و گواهی‌های انرژی	`att.list_energy_securities()`
تابلوی بورس کالا	`att.get_commodity_market(kind="certificate")`

نیاز شما	تابع پیشنهادی
سابقهٔ بازار فیزیکی کالا	`att.get_commodity_physical_history("1405-01-01")`
منحنی آتی و calendar spread	`att.get_calendar_spreads(...)` و `att.get_futures_curve(...)`

اولین دریافت: تاریخچهٔ قیمت

```
prices = att.get_history(
    "شتران",
    start="1403-01-01",
    end="1403-03-31",
    progress=False,
)
print(prices.tail())
```

خروجی نماینده:

```
              Open    High     Low   Close      Volume
J-Date
1403-03-26    2630    2680    2605    2668    82471422
1403-03-27    2670    2715    2641    2692    70938510
1403-03-28    2695    2734    2670    2718    93612045
```

قیمت‌ها در حالت پیش‌فرض تعدیل می‌شوند. برای دادهٔ خام از `auto_adjust=False` و برای ستون‌های کامل‌تر از `output_type="full"` استفاده کنید.

چند کاربرد رایج در یک نگاه

```
# حقیقی/حقوق ۳۰ روز اخیر
client_type = att.get_client_type("فملی", limit=30, progress=False)

# تاریخچه به‌همراه observation این رفتار امروز؛ opt-in است.
prices_today = att.get_history(
    "فملی", limit=20, include_today=True, progress=False
)

# نمای زندهٔ نماد و قدرت خریدار
live = att.get_live_symbol("فملی", fallback="none")
print(live[["Symbol", "Last", "Close", "IndividualPower"]])

# پنج سطح سفارش، صف و معاملات امروز
book = att.get_order_book("فملی")
queue = att.get_queue("فملی", side="both", strict=True)
today_trades = att.get_live_trades("فملی")

# ابزارهای بازار
etfs = att.list_etfs()
funds = att.list_funds(fund_type="fixed_income")

# اخزا و اختیار معامله
treasuries = att.get_treasury_yields(min_volume=1)
options = att.get_option_market(underlying="خودرو")
analytics = att.analyze_option_chain(options, risk_free_rate=0.30)

# بورس انرژی: عرضه‌های آماده و قراردادهای برق سبز
auctions = att.get_energy_auctions(status="upcoming", board="physical", top=10)
green_power = att.list_power_instruments(market="green")

# بورس کالا: گواهی‌های سپرده با پنج سطح سفارش و حقیقی/حقوق
certificates = att.get_commodity_market(kind="certificate")

# سابقهٔ بازار فیزیکی و نسبت معامله به عرضهٔ هر تالار
physical = att.get_commodity_physical_history("1405-01-01", "1405-06-01")
halls = att.get_commodity_physical_summary("1405-01-01", "1405-06-01")
```

خروجی نمایندهٔ `get_live_symbol()` در زمان بازار:

```
  Symbol   Last  Close  IndividualPower
0   1.31            73950  74200  فملی
```

مقادیر مثال‌های متصل به شبکه «خروجی نماینده» هستند و با زمان بازار تغییر می‌کنند. مثال‌های ریاضی deterministic هستند و در تست‌های آفلاین کنترل می‌شوند. ستون‌های `Last` و `Close` در feed زنده به‌ترتیب «آخرین معامله» و «قیمت پایانی» هستند؛ تفاوت نام‌گذاری live و history در بخش بعد آمده است.

سند مرجع واحد پروژه است. اگر تازه شروع کرده‌اید، بخش راهنمای توابع پرکاربرد را بخوانید؛ `README.md` همهٔ تفصیلی مرجع توابع برای دقیق lookup، signature ورودی، خروجی و قرارداد هر تابع است و لازم نیست از ابتدا تا انتها خوانده شود.

راهنمای توابع پرکاربرد

این بخش برای استفادهٔ روزمره است. ورودی‌های هر تابع کنار همان تابع توضیح داده شده‌اند؛ پس از آن، مرجع تخصصی تمام APIها قرار دارد.

`get_history()` — تاریخچهٔ قیمت

```
get_history(
    symbol="", start=None, end=None, limit=0,
    raw=False, auto_adjust=True, output_type="standard",
    date_format="jalali", progress=True, save_to_file=False,
    dropna=True, adjust_volume=False, return_type=None,
    ascending=True, save_path=None, include_today=False,
    *, ins_code=None, asset_type="auto", **kwargs,
)
```

ورودی	گزینه‌ها و معنی
`symbol`	نماد فارسی یا فهرست نمادها؛ مانند `"فملی"` یا `["فملی", "شتران"]` .
`start` , `end`	ابتدا و انتهای شامل بازه؛ تاریخ شمسی `1403-01-01` یا میلادی `2024-03-20` .
`limit`	تعداد آخرین ردیف‌های معاملاتی؛ `0` یعنی همهٔ تاریخچهٔ موجود.
`raw`	با `True` نام ستون‌ها به فرمت نزدیک TSETMC برمی‌گردد.
`auto_adjust`	پیش‌فرض `True` ؛ OHLC را با سری تعدیل‌شده بازسازی می‌کند.
`output_type`	و اطلاعات بیشتر `Value` , `No.` , `Final` برای `"full"` یا OHLCV برای `"standard"` .
`date_format`	`"both"` یا `"gregorian"` ، `"jalali"` .
`progress`	نمایش پیام پیشرفت؛ روی دادهٔ خروجی اثری ندارد.
`save_to_file`	ذخیرهٔ CSV را فعال می‌کند.
`save_path`	پوشهٔ مقصد CSV؛ نام فایل از نماد ساخته می‌شود.
`dropna`	در خروجی چندنمادی ستون‌های کاملاً تهی را حذف می‌کند.
`adjust_volume`	حجم را نیز متناسب با ضریب افزایش سرمایه تعدیل می‌کند.
`return_type`	`["simple", "Close", 5]` . یا فرم سفارشی مانند `"both"` ، `"log"` ، `"simple"`
`ascending`	`True` قدیمی به جدید؛ `False` جدید به قدیمی.
`include_today`	با observation `True` معتبر امروز را به‌صورت opt-in اضافه/جایگزین می‌کند.
`ins_code`	شناسهٔ دقیق ابزار؛ برای نمادهای تکراری و production پیشنهاد می‌شود.
`asset_type`	`"bond"` , `"index"` , `"equity"` نوع ابزار مانند hint یا `"auto"` .

ورودی	گزینه‌ها و معنی
`**kwargs`	فقط aliasهای قدیمی مستند مانند `tse_format` ، `values` ، `stock` ؛ کلید ناشناخته API عمومی نیست.

خروجی: `DataFrame` تک‌نمادی یا `DataFrame` با ستون‌های MultiIndex برای چند نماد.

`get_client_type()` — حقیقی/حقوقی

```
get_client_type(
    symbol="", start=None, end=None, limit=0, raw=False,
    output_type="standard", date_format="jalali", progress=True,
    save_to_file=False, dropna=True, ascending=True, save_path=None,
    include_today=False, *, ins_code=None, asset_type="auto", **kwargs,
)
```

ورودی	گزینه‌ها و معنی
`symbol` ، `ins_code` ، `asset_type`	انتخاب نماد؛ قواعد آن‌ها مانند `get_history()` است.
`start` ، `end` ، `limit`	بازه یا تعداد آخرین روزهای معاملاتی.
`raw`	خروجی نزدیک به schema خام provider.
`output_type`	value، سرانه و قدرت را نیز نگه می‌دارد. `"full"` ؛ حالت کامل یا `"standard"`
`date_format`	`"both"` یا `"gregorian"` ، `"jalali"` .
`include_today`	ردیف حقیقی/حقوقی امروز را با provenance و برچسب برآوردی بودن اضافه می‌کند.
`progress` ، `dropna` ، `ascending`	نمایش پیشرفت، حذف ستون تهی چندنمادی و ترتیب زمانی.
`save_to_file` ، `save_path`	ذخیرهٔ CSV و پوشهٔ مقصد.
`**kwargs`	`limit=30` . به‌جای `values=30` های قدیمی مانندalias

خروجی: `DataFrame` روزانهٔ تعداد/حجم/ارزش خریدوفروش حقیقی و حقوقی و شاخص‌های مشتق‌شده.

`get_live_symbol()` و `get_live_market()` — دادهٔ زنده

```
get_live_symbol(symbol=None, *, ins_code=None, fallback="none")
get_live_market(symbol=None, *, strict=False)
```

تابع/ورودی	گزینه‌ها و معنی
`get_live_symbol.symbol`	یک نماد فارسی یا `InsCode` .
`get_live_symbol.ins_code`	شناسهٔ دقیق keyword-only؛ بر selector مبهم ترجیح دارد.
`fallback`	نقطه‌ای استفاده می‌کند. `"point"` در نبود نماد از endpoint ؛MarketWatch فقط `"none"`

تابع/ورودی	گزینه‌ها و معنی
`get_live_market.symbol`	برای کل بازار، یک نماد یا فهرست نمادها `None`.
`strict`	با `False` نماد گم‌شده در `attrs['missing_selectors']` ثبت می‌شود؛ با `True` خطا می‌دهد.

خروجی: `get_live_symbol()` یک `DataFrame` یک‌ردیفی و `get_live_market()` یک `DataFrame` فیلترشده یا کل بازار می‌دهد. ستون‌های مهم شامل `SpreadBps`، `EstimatedNetIndividualFlow`، `IndividualPower`، `Close`، `Last` و `L5Imbalance` هستند.

`get_market_snapshot()` و feed — `get_market_client_type()` خام bulk

```
get_market_snapshot()
get_market_client_type()
```

این دو تابع ورودی کاربری ندارند. `get_market_snapshot()` یک `dict` شامل `order_book`، `stocks`، زمان بازار و metadata تازگی می‌دهد. `get_market_client_type()` یک bulk `DataFrame` حقیقی/حقوقی کل بازار برمی‌گرداند. برای مصرف معمول، `get_live_market()` نسخهٔ ادغام‌شده و راحت‌تر است.

`get_intraday()` — تیک و کندل

```
get_intraday(
    symbol="شتران", interval="1min",
    start=None, end=None, progress=True, **kwargs,
)
```

ورودی	گزینه‌ها و معنی
`symbol`	نماد فارسی.
`interval`	`"1m"`، `"12h"` ؛ هایی مانندalias `"tick"`، `"1min"`، `"5min"`، `"15min"`، `"30min"`، `"1h"`، `"4h"`، `"720"`، `"240m"`، `"60min"` نیز پذیرفته می‌شوند.
`start`، `end`	بدون مقدار، دادهٔ امروز؛ با تاریخ، snapshotهای تاریخی بازه.
`progress`	نمایش پیشرفت.
`**kwargs`	فقط سازگاری نام‌های قدیمی.

خروجی: در حالت tick دادهٔ معامله و در حالت candle ستون‌های `Open, High, Low, Close, Volume, TradeCount`. این API قدیمی برای ورودی نامعتبر ممکن است پیام چاپ کند و `None` بدهد.

`get_symbols()` — فهرست ابزارها

```
get_symbols(
    bourse=True, farabourse=True, payeh=True,
    haghe_taqadom=False, sandogh=False, bonds=False,
    options=False, mortgage=False, commodity=False, energy=False,
    payeh_color=None, output="dataframe", progress=True, **kwargs,
)
```

ورودی	گزینه‌ها و معنی
`bourse` , `farabourse` , `payeh`	بازارهای سهام که به‌صورت پیش‌فرض فعال‌اند.
`haghe_taqadom`	افزودن حق‌تقدم‌ها.
`sandogh`	افزودن صندوق‌ها.
`bonds`	افزودن اوراق بدهی.
`options`	افزودن اختیار معامله‌ها.
`mortgage`	افزودن اوراق تسهیلات مسکن.
`commodity`	افزودن گواهی‌های کالایی.
`energy`	افزودن ابزارهای انرژی.
`payeh_color`	`"قرمز"` , `"نارنجی"` , `"زرد"` یا فهرستی از آن‌ها.
`output`	`"dataframe"` یا خروجی فهرستی قدیمی.
`progress`	نمایش پیشرفت.
`**kwargs`	نام‌های انگلیسی قدیمی فیلترها.

خروجی: فهرست ابزارها با هویت، نماد، نام، بازار و نوع دارایی.

`get_tgju_history()` و `get_currency()` — طلا، ارز، سکه و فلزات

```
get_currency(
    name="", start=None, end=None, limit=0,
    output_type="standard", date_format="jalali", progress=True,
    save_to_file=False, dropna=True, return_type=None,
    ascending=True, save_path=None, **kwargs,
)
get_tgju_history(...)  # نام پیشنهادی برای همهٔ دسته‌های TGJU همان قرارداد؛
list_tgju_assets(category=None, include_aliases=False)
```

ورودی	گزینه‌ها و معنی
`name`	یک نام canonical، نام فارسی، slug رسمی یا فهرست؛ مانند `"طلای ۲۴ عیار"`, `"gold-18k"`, `"seke"`, `"dollar"`, `"geram18"`.
`start`, `end`, `limit`	بازه یا آخرین تعداد ردیف.
`output_type`	`"standard"` قدیمی API یا حالت کامل پشتیبانی‌شدهٔ.
`date_format`	`"both"`, `"gregorian"`, `"jalali"`.
`return_type`	محاسبهٔ بازده ساده/لگاریتمی.
`progress`, `dropna`, `ascending`	پیشرفت، خروجی چندارزی و ترتیب زمانی.
`save_to_file`, `save_path`	ذخیرهٔ CSV و پوشهٔ مقصد.
`**kwargs`	های سازگاری قدیمیalias.
`category`	در `list_tgju_assets()` یکی یا فهرستی از `silver`, `gold`, `official_rate`, `currency`, `global_metal`, `coin_bubble`, `coin`.
`include_aliases`	افزودن tuple نام‌های جایگزین به کاتالوگ.

خروجی: تاریخچه `DataFrame[Open, High, Low, Close]` است؛ چند دارایی ستون‌های MultiIndex می‌سازد. کاتالوگ ستون‌های `Name, PersianName, Slug, Category, Unit` دارد. واحد سری در `df.attrs['unit']` هم ثبت می‌شود.

`get_trades()` و `get_live_trades()` — معاملات ریز

```
get_trades(
    symbol=None, *, ins_code=None, start=None, end=None,
    include_canceled=False, max_requests=None, raw=False, progress=True,
)
get_live_trades(
    symbol=None, *, ins_code=None,
    include_canceled=False, max_requests=None, raw=False, progress=True,
)
```

ورودی	گزینه‌ها و معنی
`symbol`, `ins_code`	انتخاب نماد با نام یا شناسهٔ دقیق.
`start`, `end`	بازهٔ تاریخی شامل دو سر؛ فقط در `get_trades()`.
`include_canceled`	نگه‌داشتن معاملات ابطالی.
`max_requests`	سقف سخت تعداد درخواست‌ها؛ برای بازه‌های بزرگ حتماً تعیین کنید.
`raw`	افزودن ستون‌های audit نزدیک provider.
`progress`	نمایش پیشرفت.

خروجی: `DataFrame` معاملات با `TradeNo`، `Timestamp`، `Price`، `Volume`، `Value`، `Canceled`، `Source`.

`get_order_book()` و `get_queue()` — سفارش و صف زنده

```
get_order_book(symbol=None, *, selector_strict=False)
get_queue(symbol=None, side="both", strict=True, *, selector_strict=False)
```

ورودی	گزینه‌ها و معنی
`symbol`	برای کل بازار، یک نماد یا فهرست نمادها `None`.
`selector_strict`	گم‌شده یا مبهم را به خطا تبدیل می‌کند selector.
`side`	در `"both"`، `"sell"`، `"buy"` : `get_queue()`.
`strict`	فقط صف قطعی مبتنی بر قیمت مجاز و level یک را نگه می‌دارد.

خروجی: order book به‌شکل long با پنج level؛ queue با سمت، قیمت، حجم، تعداد سفارش و ارزش صف.

`list_bonds()`، `list_funds()`، `list_etfs()` — اوراق و اختیارها

```
list_etfs(progress=True)
list_funds(fund_type=None, progress=True, *, listed_only=False,
           strategy=None, commodity_underlying=None,
           classification_status=None)
list_listed_funds(progress=True, *, fund_category=None, strategy=None,
                  commodity_underlying=None, classification_status=None,
                  include_unknown=True)
list_bonds(progress=True)
list_debt_instruments(debt_type=None, active_only=False, progress=True)
list_options(underlying=None, progress=True)
get_options_chain(underlying, fetch_oi=False, progress=True)
```

تابع/ورودی	گزینه‌ها و معنی
`progress`	در همهٔ این توابع فقط نمایش پیشرفت را کنترل می‌کند.
`fund_type`	`None` برای همه یا یکی/فهرستی از `equity`، `mixed`، `fixed_income`، `commodity`، `real_estate`، `sector`، `leveraged`، `index`، `capital_guaranteed`، `market_maker`، `venture_capital`، `project`، `private`، `fund_of_funds`، `supplementary_retirement`. نام قدیمی `venture` نیز پذیرفته می‌شود.
`listed_only`	در `list_funds()` فقط ابزارهای قابل معامله را برمی‌گرداند؛ هم‌زمان با `fund_type` مجاز نیست.
`fund_category`	در `list_listed_funds()` دستهٔ اثبات‌شده، مانند `"commodity"`.
`strategy`	تگ دقیق مانند `"index_tracking"`، `"leveraged"`، `"sector"` یا `"market_making"`.
`commodity_underlying`	کالای پایهٔ اثبات‌شده: `"silver"`، `"gold"` یا `"saffron"`؛ نام عمومی حدس زده نمی‌شود.

تابع/ورودی	گزینه‌ها و معنی
`classification_status`	فیلتر بر اساس `verified_exact_mapping`، `authoritative_field` یا `explicit_official_name` و `unknown`.
`include_unknown`	در فهرست بورسی، با `False` ردیف‌های فاقد طبقه‌بندی کافی را حذف می‌کند؛ پیش‌فرض `True` همه را نگه می‌دارد.
`debt_type`	در `list_debt_instruments()` یک نوع یا فهرست از `treasury`، `erad`، `gam`، `murabaha`، `ijara`، `salaf` و `other`.
`active_only`	فقط اوراقی را نگه می‌دارد که سررسیدشان امروز یا بعد از امروز است.
`underlying`	در `list_options()` فیلتر اختیاری و در `get_options_chain()` دارایی پایهٔ اجباری.
`fetch_oi`	در زنجیرهٔ اختیار، دریافت Open Interest و metadata تکمیلی را فعال می‌کند و ممکن است درخواست‌های بیشتری بسازد.

خروجی: همه `DataFrame` هستند، به‌جز `get_options_chain()` که `dict` شامل `calls`، `puts`، `underlying_price`، `expiry_dates` و `market_time` می‌دهد.

در `list_funds()` ستون `pct_cash` از سرویس FIPIRAN و فقط با اتصال دقیق `reg_no` گرفته می‌شود؛ `pct_deposit` همان سپردهٔ بانکی است و این دو ستون معادل هم نیستند. اگر مقدار معتبر وجه نقد برای یک شناسه منتشر نشده باشد یا FIPIRAN در دسترس نباشد، `pct_cash` خالی می‌ماند و از سپرده یا باقیماندهٔ سایر درصدها حدس زده نمی‌شود. وضعیت پوشش در `DataFrame.attrs` با کلیدهای `pct_cash_values_available`، `pct_cash_source_status`، `pct_cash_missing` و `pct_cash_inferred=False` ثبت می‌شود.

قراردادهای مهم داده

نوع خروجی

همهٔ خروجی‌ها DataFrame نیستند:

خانواده	نوع خروجی
قیمت، حقیقی/حقوقی، fundamentals، order book، trades و فهرست ابزارها	`pandas.DataFrame` گاهی `None` در APIهای legacy یا هنگام خطا
`get_market_snapshot()` / `market_watch()`	`dict` شامل `stocks`، `order_book` و زمان metadata مشاهده
`get_options_chain()`	`dict` شامل `calls` و `puts`
`resolve_instrument()`	شیء immutable از نوع `InstrumentRef`
`MarketWatcher` / `watch_market()`	iterator از `MarketEvent`
`build_yield_curve()` / `get_yield_curve()`	شیء `YieldCurve`

خانواده	نوع خروجی
توابع Black–Scholes	عدد، tuple یا `dict` مطابق تابع

برای DataFrameهای جدید، خروجی خالی همان columns و dtypes خروجی غیرخالی را نگه می‌دارد. metadataهای مهم مانند منبع، freshness، پوشش، partial بودن و بودجهٔ درخواست در `df.attrs` قرار می‌گیرند:

```
df = att.get_live_market("فملی")
print(df.attrs)
```

```
{
  'trade_date': datetime.date(...),
  'exchange_time': '12:28:41',
  'fetched_at': Timestamp(..., tz='Asia/Tehran'),
  'is_realtime_fresh': True,
  'is_partial': False,
  'missing_selectors': []
}
```

قیمت پایانی و آخرین معامله

- در feed زنده: `Last` آخرین قیمت معامله و `Close` قیمت پایانی TSETMC است.
- در تاریخچهٔ سهام: `Close` آخرین قیمت معاملهٔ روز است؛ `Final` قیمت پایانی است و فقط در `output_type="full"` دیده می‌شود.
- با `auto_adjust=True` (پیش‌فرض)، OHLC و `Final` در صورت حضور، تعدیل‌شده‌اند.
- با `auto_adjust=False`، `Close` و `Final` خام‌اند و `Adj Close` نیز ارائه می‌شود.
- هنگام `include_today=True`، `Last` زنده به `Close` تاریخچه و `Close` زنده به `Final` تاریخچه نگاشت می‌شود. بنابراین صرفاً بر اساس نام ستون بین live و history join نزنید.

تاریخ، ردیف امروز و freshness

- `start` و `end` تاریخ ISO شمسی شامل دو سر بازه‌اند و میلادی (`2024-07-22`) یا شمسی (`1403-05-01`) می‌پذیرند.
- رفتار قبلی حفظ شده است: `include_today=False` هیچ درخواست زندهٔ اضافه‌ای انجام نمی‌دهد.
- قدیمی snapshot. می‌کند append/replace معتبر همان روز معاملاتی را observation فقط `include_today=True` خواهد داشت. `live_is_realtime_fresh=False` همان روز ممکن است برای تکمیل تاریخچه پذیرفته شود ولی
- توقف نماد، نبود معامله، نبود هویت قطعی یا شکست live باعث جعل ردیف امروز نمی‌شود؛ تاریخچهٔ موفق برمی‌گردد و هشدار/attrs دلیل را نشان می‌دهند.
- مانند server-side ادعا نمی‌کند. تاریخچهٔ backfill محلی فقط از زمان ضبط شما پوشش دارد و history قرارداد جداگانه دارد price/trades/order-book.

تقارن API زنده/تاریخی:

داده	زنده	تاریخی
قیمت و نمای نماد	`get_live_symbol` , `get_live_market`	`get_history(include_today=...)`
حقیقی/حقوقی	`get_live_market` , `get_market_client_type`	`get_client_type(include_today=...)`
معاملات ریز	`get_live_trades`	`get_trades`
پنج سطح سفارش	`get_order_book`	`get_order_book_history`
صف	`get_queue`	`get_queue_history`
فاندامنتال snapshot	`get_market_fundamentals`	`get_market_fundamentals_history`
اخزا و YTM	`get_treasury_yields`	`get_treasury_yield_history`
منحنی بازده	`get_yield_curve`	`get_yield_curve_history`
اختیار	`get_option_market`	`get_option_history` و snapshotهای `save_option_snapshot` / `load_option_snapshots`
overview/breadth/sector/message/state	متناظر live هایhelper	`*_history` پس از `archive_to` یا helperهای snapshot

تقارن به معنی یکسان‌بودن منبع نیست: بعضی historyها server-side هستند و بعضی فقط observationهای ذخیره‌شدهٔ کاربر را می‌خوانند. `NoBackfill` ، `Source` و coverage را بررسی کنید.

ذخیره در CSV

در APIهای تاریخی `save_path` ،legacy **مسیر پوشه** است، نه نام فایل. نام فایل از نماد/دارایی ساخته می‌شود:

```
att.get_history(
    "فملی", save_to_file=True, save_path="exports", progress=False
)
# exports/فملی.csv
```

source boundary

درخواست‌های runtime فقط به providerهای پشتیبانی‌شدهٔ TSETMC، صفحهٔ مرجع YTM فرابورس ایران (`ifb.ir`) و API تاریخچهٔ بازار TGJU برای دارایی‌های کاتالوگ محدودند. redirect به origin دیگر پیش از درخواست دوم رد می‌شود. هیچ API کدال در این پکیج وجود ندارد.

قطعی و قابل استناد منتشر نمی‌کند. اختلاف قیمت تعدیل‌شده و تعدیل‌نشده **معادل** DPS ،برای رخداد تعدیل قیمت TSETMC .استنتاج نمی‌کند DPS **سود نقدی نیست** و کتابخانه از آن

حل دقیق هویت نماد

نمادهای تکراری، ابزار منقضی، حق‌تقدم، اختیار و اوراق می‌توانند نام مشابه داشته باشند. APIهای جدید از `InstrumentRef` و `InsCode` استفاده می‌کنند:

```
ref = att.resolve_instrument("فملی", asset_type="equity")
print(ref)
```

خروجی نماینده:

```
InstrumentRef(
    ins_code='35425587644337450', symbol='فملی',
    name='ملی صنایع مس ایران', asset_type='equity',
    is_active=True, provenance='tsetmc_search_exact', selector='فملی'
)
```

برای اجرای قطعی در `ins_code`، production بدهید:

```
prices = att.get_history(
    ins_code="35425587644337450", asset_type="equity", progress=False
)
live = att.get_live_symbol(ins_code="35425587644337450")
```

`validate_ins_code()` را می‌پذیرد و نتیجه را به‌صورت رشته برمی‌گرداند؛ ASCII عدد صحیح مثبت دقیق یا رشتهٔ ۱ تا ۲۰ رقم بیش از یک نتیجهٔ معتبر داشته باشد، selector رقم فارسی/عربی، صفر و مقدار مبهم پذیرفته نمی‌شوند. اگر `float` ، `bool` ، کتابخانه حدس نمی‌زند:

```
try:
    att.resolve_instrument("نماد تکراری")
except att.AmbiguousSymbolError as exc:
    print(exc)
```

```
AmbiguousSymbolError: selector matched more than one instrument; pass ins_code
```

گزینه‌های اصلی:

```
att.resolve_instrument(
    selector=None,
    ins_code=None,
    asset_type="auto",
    snapshot=None,
    require_active=True,
)
```

- `snapshot` امکان resolve بدون fetch دوباره را می‌دهد.
- `require_active=False` برای تاریخچهٔ ابزار منقضی مناسب است.
- هویتی fuzzy join و فاصله‌های متداول را یکسان می‌کند، اما `ک/ك` ، `ی/ي` اختلاف `normalize_instrument_text()` انجام نمی‌دهد.

قیمت و حقیقی/حقوقی؛ تاریخچه و زنده

`get_history()`

```
att.get_history(
    symbol="فملی", start=None, end=None, limit=0,
    raw=False, auto_adjust=True, output_type="standard",
    date_format="jalali", progress=True, save_to_file=False,
    dropna=True, adjust_volume=False, return_type=None,
    ascending=True, save_path=None, include_today=False,
    ins_code=None, asset_type="auto",
)
```

```
history = att.get_history(
    "فملی", limit=10, include_today=True,
    output_type="full", progress=False,
)
print(history.tail(2))
print(history.attrs["include_today_appended"])
```

خروجی نماینده:

```
             Open   High    Low  Close  Final   Volume  No.       Value
Date
1405-06-02   73100  74600  72800  74200  73950  1820043  3912  1.35e+11
1405-06-03   74000  74900  73600  74700  74420   814220  1830  6.06e+10
['فملی']
```

در `standard` ستون‌ها `Open, High, Low, Close, Volume` هستند. `full` ستون‌های `Final, No., Value` و اطلاعات تقویم/Ticker را نیز اضافه می‌کند. `raw=True` فرمت TSE را برمی‌گرداند. `return_type` یکی از `both` ، `log` ، `simple` یا فرم سفارشی مانند `['simple', 'Close', 5]` است. برای چند نماد، DataFrame با MultiIndex ستونی برمی‌گردد.

`get_client_type()`

```
client = att.get_client_type(
    "فملی", limit=30, include_today=True,
    output_type="full", progress=False,
)
print(client.tail(1))
```

خروجی نمایندهٔ ردیف امروز:

```
            No_buy_retail  Vol_buy_retail  No_sell_retail  Vol_sell_retail  Power_retail  Is_partial
Date
1403-05-07           1284          920000             991           701000          1.24        True
```

ردیف امروز volume/count را از `ClientTypeAll` می‌گیرد. valueهای امروز در صورت نیاز با VWAP بازار برآورد می‌شوند و ستون‌های `Is_estimated` / `Value_source` در خروجی opt-in مشخص می‌کنند که مقدار رسمی یا برآوردی است. رفتار پیش‌فرض و schema قدیمی بدون `include_today` تغییر نکرده است.

live کل بازار و یک نماد

```
snapshot = att.get_market_snapshot()       # dict سازگار با market_watch()
live_all = att.get_live_market()           # DataFrame ادغام‌شده
live_one = att.get_live_market("فملی")
point = att.get_live_symbol("فملی", fallback="none")
```

`get_live_market()` قیمت، client type را join می‌کند. ستون‌های کلیدی و بهترین سفارش‌ها:

```
InsCode, Symbol, Name, Last, Close, PreviousClose, Volume, Value,
IndividualBuyVolume, IndividualSellVolume, LegalBuyVolume, LegalSellVolume,
IndividualPower, NetIndividualVolume, EstimatedNetIndividualFlow,
BidPrice1, AskPrice1, Spread, SpreadBps, L1Imbalance, L5Imbalance,
trade_date, exchange_time, fetched_at, is_realtime_fresh, is_partial
```

`get_live_market()` ستون row-wise نیست، attr عمومی است. attrs واقعی برای audit schema/selection و freshness در:

```
print(live_all.attrs.keys())
print(live_all.attrs["missing_selectors"])
```

```
dict_keys(['field_validity', 'source_schema_presence', 'migration', 'missing_selectors'])
[]
```

`StockNotFoundError` می‌دهد. `get_live_symbol(..., fallback="none")` در نبود نماد در MarketWatch، دقیقاً یکی از `SnapshotSource` را امتحان می‌کند؛ ستون TSETMC نقطه‌ای endpoint فقط در آن حالت `fallback="point"` است. `PresentInMarketWatch`، `FallbackReason`، `market_watch` یا `closing_price_info_fallback` قابلیت استفادهٔ قیمت را شفاف می‌کنند `PriceActionable` و `has_trade_today`، `IdentityVerified`.

`get_order_book()` خروجی long دارد پنج‌سطحی:

```
InsCode Symbol Level BidOrderCount BidVolume BidPrice AskPrice AskVolume AskOrderCount
...     فملی   1     124000        74200     74100    180000   42        1
```

ستون‌های metadata آن شامل `trade_date`، `exchange_time`، `fetched_at`، `snapshot_age_seconds`، `is_stale`، `is_realtime_fresh` و `is_partial` است.

معاملات ریز

`get_live_trades()` همان قرارداد را استفاده می‌کند و TSETMC تاریخچهٔ معاملات endpoint برای هر روز از `get_trades()` برای روز جاری ارائه می‌دهد:

```
trades = att.get_trades(
    "فملی",
    start="1403-05-01",
    end="1403-05-03",
    include_canceled=False,
    max_requests=10,
    raw=False,
    progress=False,
)

live_trades = att.get_live_trades(
    ins_code="35425587644337450",
    include_canceled=False,
    max_requests=1,
    progress=False,
)
```

استاندارد schema:

```
InsCode, Symbol, GregorianDate, JalaliDate, TradeNo, Time, Timestamp,
Price, Volume, Value, Canceled, Source
```

خروجی نماینده:

```
             TradeNo      Time  Price  Volume      Value  Canceled                          Source
Timestamp
...                1  09:01:01  50000     100    5000000     False  tsetmc_trade_history_lossless
...                2  09:01:01  50010      50    2500500     False  tsetmc_trade_history_lossless
```

را نیز نگه `RawCanceled` و `RawDate` ، `RawInsCode` ، `iSensVarP` ، `qTitNgJ` مانند provider فیلدهای خام `raw=True` می‌دارد. attrsهای مهم:

```
request_count, trade_request_count, max_requests, request_budget_scope,
partial, failures, source_coverage, reconciliation,
resolved_ins_code, resolved_symbol
```

- ممکن است هزینهٔ جداگانه داشته باشد و resolver است؛ trade endpoint سقف سخت درخواست‌های `max_requests` مشخص می‌کند شمارش کل شناخته‌شده است یا نه attrs.
- شکست بخشی از روزها با `partial=True` و جزئیات `failures` برمی‌گردد؛ دادهٔ موجود دور ریخته نمی‌شود.
- رد می‌شود. `DataParsingError` پاسخ، تاریخ یا فیلدهای عددی با `InsCode` در mismatch
- رکورد ابطالی فقط با `include_canceled=True` نگه داشته می‌شود.

سفارش و صف

سفارش زنده

```
book = att.get_order_book("فملی")
queue = att.get_queue("فملی", side="both", strict=True)
```

قیمت صف را با `strict=True` قطعی. `False` است، نه `NA` صف را سه‌حالته گزارش می‌کند؛ نبود شواهد کافی `get_queue()` بازار تطبیق می‌دهد. state دامنهٔ مجاز و

```
InsCode Symbol Side QueuePrice QueueVolume QueueOrders QueueValue
...     فملی   buy       73500      820000         163  6.027e+10

is_queue=True, is_strict=True, is_partial=False,
threshold_source='market_watch_price_limits', book_source='market_watch_live_snapshot'
```

تاریخچهٔ پنج سطح سفارش

```
books = att.get_order_book_history(
    "فملی",
    limit=20,
    output_type="standard",   # longهر در ردیف یک ؛ level
    include_today=True,
    complete_only=True,
    max_requests=25,
    progress=False,
)

wide = att.get_order_book_history(
    "فملی", limit=10, output_type="wide", max_requests=25, progress=False
)
```

دادهٔ تاریخی `BestLimits` یک delta stream روزانه است. کتابخانه state هر روز را مستقل و به ترتیب `hEven/refID` بازسازی می‌کند؛ مقدار صفر پاک‌شدن level است. schema long:

```
InsCode, Symbol, Name, Date, GregorianDate, JalaliDate, Time, Timestamp,
DEven, hEven, refID, _sequence, Level,
BidOrderCount, BidVolume, BidPrice, AskPrice, AskVolume, AskOrderCount,
is_partial, is_complete, market_partial_status, is_reconstructed,
is_stale, record_type, source
```

خروجی نماینده:

```
Timestamp                 Level BidPrice BidVolume AskPrice AskVolume is_complete source
2024-07-22 09:05:01+03:30     1    74100    180000    74200    124000        True
tsetmc_best_limits_history_reconstructed
2024-07-22 09:05:01+03:30     2    74050     95000    74250     88000        True
tsetmc_best_limits_history_reconstructed
```

برابر source زندهٔ معتبر را با snapshot `include_today=True` ناقص را حذف می‌کند snapshot `complete_only=True` سقف سخت است و `max_requests` اضافه یا جایگزین می‌کند `market_watch_live_snapshot`. پوشش ناقص را توضیح می‌دهند `attrs['request_count']` و `attrs['failed_requests']`.

نام alias `get_orderbook_history` عینی `get_order_book_history` است.

تاریخچهٔ صف

```
queues = att.get_queue_history(
    "فملی",
    limit=20,
    include_today=True,
    complete_only=True,
    side="both",
    strict=True,
    max_requests=25,
    progress=False,
)
```

schema:

```
InsCode, Symbol, Name, Date, GregorianDate, JalaliDate, Time, Timestamp,
DEven, hEven, Side, QueuePrice, QueueVolume, QueueOrders, QueueValue,
PriceLimit, is_strict, is_queue, is_partial, is_complete,
market_partial_status, book_state, is_crossed, is_preopen_or_stopped,
threshold_hEven, threshold_source, book_source, source
```

اگر static threshold تاریخی در دسترس نباشد، `threshold_source='unavailable'` و `is_queue=NA` می‌ماند. این رفتار برای backtest مهم است: نبود داده به «صف نبود» تبدیل نمی‌شود.

Watcher و تحلیل کل بازار

iterator بازار

```
for event in att.watch_market(
    symbol="فملی",
    interval=2,
    max_updates=5,
    notifications=("messages", "state"),
):
    print(event.kind, event.sequence, event.fetched_at, event.changed_inscodes)
```

شامل event .وجود ندارد `kind="update"` .است `resync` ، `heartbeat` ، `delta` ، `initial` یکی از `MarketEvent.kind` .است persistence و وضعیت retry، cursor، های تغییرکرده‌level، های تغییرکرده‌InsCode فهرست، snapshot

برای کنترل کامل:

```
watcher = att.MarketWatcher(
    symbol=None,
    interval=2,
    max_updates=None,
    include_initial=True,
    emit_heartbeats=True,
    notifications=("messages", "state"),
    notification_top=50,
    error_policy="retry",
    max_consecutive_retries=5,
    max_backoff=30,
    callback_error_policy="raise",
)
```

resync محدود backoff را نگه می‌دارد و در قطع ارتباط با cursor ،استفاده می‌کند `MarketWatchInit/Plus` از Watcher
.هستند `state` و `messages` ها فقطnotification .می‌شود

پیام، وضعیت و نمای بازار

```
messages = att.get_market_messages(flow=0, top=20, since_id=None)
states = att.get_instrument_state_changes(top=20, since_id=None)
overview = att.get_market_overview(flow=0)
```

:پیام schema

```
message_id, date, time, timestamp, title, description, flow
```

:وضعیت schema

```
event_id, date, time, timestamp, InsCode, Symbol, Name,
state_code, state, real_time, under_supervision, state_title
```

رسمی payload برمی‌گرداند؛ مجموعهٔ ستون‌ها به `flow` را با ستون overview رسمی raw فیلدهای `get_market_overview()`
.وابسته است و به تعداد ثابتی از ستون‌ها متعهد نیست provider

breadth و جریان صنایع

```
breadth = att.get_market_breadth(traded_only=True)
sectors = att.get_sector_flow(traded_only=True)
```

:خروجی نمایندهٔ breadth

```
 instrument_count advances declines unchanged ad_difference ad_ratio total_volume upper_limit_count
lower_limit_count
              612      318      241        53            77     1.32   8.91e+09                42
18
```

ستون‌های breadth علاوه بر موارد بالا شامل `no_trade`، `missing_previous`، `missing_current_price`، درصد صعود/نزول، `total_value`، `trade_date`، `exchange_time`، `fetched_at`، `is_realtime_fresh` است.

`SectorCode` را برای هر breadth همین `get_sector_flow()` همراه با پوشش حقیقی/حقوقی ارائه می‌کند:

```
SectorCode instrument_count advances declines client_coverage net_individual_volume
estimated_net_individual_value value_method is_stale
      27               43       25       13            0.91              820000
5.8e+10 market_vwap    False
```

`estimated_net_individual_value` برآورد است؛ `value_available`، `value_method`، `client_coverage` و freshness را در استراتژی لحاظ کنید.

فیلترهای مشترک عبارت‌اند از `include_base_market`، `traded_only`، `flow`، `sector`، `symbol` و `instrument_types`. برای محاسبهٔ چند خروجی روی یک مشاهده، snapshot را یک‌بار دریافت و به مسیر خصوصی `_snapshot` ندهید؛ API عمومی `save_market_snapshot()` یک snapshot اتمیک می‌سازد و historyها را هم‌زمان نگه می‌دارد.

تقویم، نمای کلان، مستر ابزار و پیش‌گشایش

برای تقویم، TSETMC های این بخش دو گروه داده را یکجا در نسخهٔ 1.4.0 فراهم می‌کنند: داده‌های عمومی و بدون حسابAPI قابل دریافت است TSETMC رسمی که فقط با اشتراک وب‌سرویس TOP ارزش بازار، اثر شاخص، خلاصهٔ روزانه و مستر ابزار؛ و.

تقویم معاملاتی بورس و فرابورس

```
calendar = att.get_trading_calendar(
    start="1405-06-01",
    end="1405-06-18",
    market="all",
    include_closed=True,
)
print(calendar.tail())
```

تاریخ شمسی یا `end` و `start` است. `فرابورس` / `farabourse` / `ifb` یا `بورس` / `bourse` / `tse`، `all` یکی از `market` در حالت `include_closed` آخرین جلسهٔ هر بازار را برمی‌گرداند. پارامتر `limit`، میلادی و شامل دو مرز هستند. اگر بازه ندهید روزهای تقویمی بین دو مرز را نیز می‌سازد و `True` فقط جلسات منتشرشدهٔ شاخص مرجع را می‌آورد؛ مقدار `False` پیش‌فرض `IsTradingDay=False` می‌گذارد.

```
Market, Flow, GregorianDate, JalaliDate, IsTradingDay,
ReferenceIndex, ReferenceInsCode, Source
```

تعریف روز معاملاتی در این تابع «وجود مشاهده در تاریخچهٔ شاخص کل رسمی همان بازار» است. بنابراین تعطیلی عادی یا نبود جلسه مشخص می‌شود، اما برنامهٔ ساعات جلسه و علت تعطیلی از این منبع استخراج نمی‌شود. این معنا در `attrs['calendar_semantics']` ثبت شده است.

ارزش و فعالیت روزانهٔ بازار

```
market_value = att.get_market_value_history(
    start="1405-01-01",
    market="all",
    limit=90,
)

activity = att.get_market_activity(
    start="1405-06-15",
    end="1405-06-18",
    market="tse",
    max_requests=10,
    progress=False,
)
```

`get_market_value_history()` برای هر بازار `MarketValue` و `Change` را به ریال و دو ستون مشتق‌شدهٔ `ChangePct` مشخص باشد تا ۵۰۰۰ مشاهده از منبع درخواست می‌شود و سپس بازه فیلتر می‌شود؛ در حالت بدون بازه `start` می‌دهد. اگر `limit` سقف دریافت است.

```
Market, Flow, GregorianDate, JalaliDate, MarketValue,
Change, ChangePct, Source
```

`get_market_activity()` بودجهٔ سخت ۱ تا ۳۶۶ است و `max_requests` انجام می‌دهد bulk برای هر جلسه یک درخواست. خروجی شامل fan-out پیش از کنترل می‌شود. `InstrumentCount`، `TradedInstrumentCount`، `TradeCount`، `Volume`، `Value` و `AverageTradeValue` است. این آمار از خلاصهٔ رسمی روزانهٔ ابزارها ساخته می‌شود؛ `AverageTradeValue = Value / TradeCount` و در نبود معامله `NaN` است.

اثر نمادها بر شاخص

```
impact = att.get_index_impact(
    date="1405-06-18",
    market="tse",
    top=10,
    direction="both",
)
print(impact[["Rank", "Symbol", "Impact", "AbsSharePct"]])
```

`date=None` آخرین تاریخ دارای ارزش بازار را انتخاب می‌کند. `top` عدد صحیح مثبت تا ۱۰۰۰ است. `direction` فقط `both`، `positive` یا `negative` را می‌پذیرد. `Impact` عدد رسمی اثر و `AbsSharePct` سهم قدرمطلق هر ردیف از مجموع قدرمطلق **ردیف‌های بازگردانده‌شدهٔ همان بازار** است؛ این درصد سهم از کل universe را برگرداند مگر اینکه منبع تمام ردیف‌ها را برگرداند.

خلاصهٔ bulk همهٔ ابزارها در یک روز

```
daily = att.get_market_trades(
    date="1405-06-18",
    market="all",
    progress=False,
)
leaders = daily.nlargest(10, "Value")
```

هر ردیف یک خلاصهٔ روزانهٔ یک هویت ابزار است و ستون‌های `Close`، `High`، `Low`، `Open`، `Value`، `Volume` و `TradeCount` را دارد. نام قدیمی سرویس رسمی `TradeOneDayAll` است، اما این خروجی **معاملات ریز و شماره‌دار نیست**. برای معامله‌به‌معامله از `get_trades()` یا `get_live_trades()` استفاده کنید.

ردیف‌های `market='all'` فیلتر می‌شوند. در ISIN با نگاشت قطعی خانوادهٔ پیشوند `market='ifb'` و `market='tse'` و `MarketClassification` جریان‌های دیگر، بورس کالا، انرژی یا طبقه‌بندی‌نشده نیز حفظ می‌شوند و کیفیت اتصال در `attrs['unknown_market_rows']` نیست، اما پیشوندهایی که بازارشان به‌تنهایی قابل اثبات fuzzy دیده می‌شود. این روش `unknown` یا `other` نیست عمداً. می‌مانند. `progress` فقط پیام پایان را کنترل می‌کند.

مستر ابزارها و کشف تغییرات

```
master = att.get_instrument_master(
    active=None,
    market=None,
    asset_type=["stock", "fund", "bond"],
    progress=False,
)
master.to_parquet("instrument-master-1405-06-18.parquet", index=False)

changes = att.get_instrument_changes(
    previous="instrument-master-1405-06-18.parquet",
    current=None,
    progress=False,
)
```

در صورت استفاده یکی از `get_instrument_master()` می‌خواند. `InsCode` را با کلید پایدار TSETMC جدول عمومی `market`، `stock`، `right`، `fund`، `bond`، `option`، `future`، `asset_type` یک رشته یا iterable از `mortgage`، `commodity`، `energy`، `unknown` بورس یا فرابورس است. پارامتر `active` با مقدار `None` وضعیت فعالیت را نامعلوم نگه می‌دارد و مستر را با `True` یا `False` تطبیق می‌دهد. MarketWatch لحظه‌ای snapshot درخواست زندهٔ اضافه ندارد؛ مقدار

```
InsCode, ISIN, Symbol, Name, EnglishName, CompanyCode, CompanyISIN,
Market, MarketGroup, Industry, AssetType, InstrumentStatus, Active, Source
```

یا مسیر `DataFrame` مقایسه می‌کند. هر ورودی می‌تواند `current` قبلی را با snapshot یک `get_instrument_changes()` مستر فعلی را دریافت می‌کند. `current=None` .داشته باشد `InsCode` باشد و باید ستون CSV، Parquet، JSON/JSONL است. `NewValue`, `OldValue`, `Field` و ستون‌های `ChangeType` های با long-form خروجی `changed`, `removed`, `added` .های خودتان را نگه دارید snapshot **نمی‌کند**؛ برای مقایسهٔ روزهای گذشته باید **backfill** این تابع **تاریخچه را از سرور**

TOP رسمی و عدم تعادل پیش‌گشایش

```
top = att.get_theoretical_opening_price(
    username="TSETMC_WEB_SERVICE_USERNAME",
    password="TSETMC_WEB_SERVICE_PASSWORD",
    market="tse",
    timeout=20,
    progress=False,
)

imbalance = att.get_preopen_imbalance(top_data=top)
print(imbalance.nlargest(10, "ImbalanceRatio"))
```

کاربرد ندارد. PyPI هستند؛ حساب عادی سایت یا حساب **TSETMC** مشخصات **اشتراک وب‌سرویس** `password` و `username` ،به سرویس رسمی می‌فرستد و ذخیره نمی‌کند. برای امنیت، رمز را در کد HTTPS پکیج آن‌ها را فقط در همان فراخوانی از راه .مهلت درخواست بر حسب ثانیه است `timeout` و `all` بازار یا `market` .قرار ندهید notebook، log یا repository

```
Market, Flow, InsCode, Symbol, Name, GregorianDate, JalaliDate, Time,
TheoreticalOpeningPrice, TheoreticalVolume, RemainderSide, RemainderVolume,
TheoreticalBuyVolume, TheoreticalBuyPrice, TheoreticalSellPrice,
TheoreticalSellVolume, ChangePct, Source
```

اگر قبلاً TOP را گرفته‌اید، آن را با `top_data` به `get_preopen_imbalance()` بدهید تا درخواست دیگری انجام نشود. اگر `top_data=None` باشد، همان تابع با `timeout`, `market`, `password`, `username` و `progress` دادهٔ رسمی را می‌گیرد. خروجی سه ستون اضافه دارد:

```
ImbalanceVolume = TheoreticalBuyVolume - TheoreticalSellVolume
ImbalanceRatio  = ImbalanceVolume / (TheoreticalBuyVolume + TheoreticalSellVolume)
ImbalanceSide   = buy | sell | balanced | unavailable
```

این نسبت صرفاً عدم تعادل حجم‌های TOP رسمی است و سیگنال خرید یا فروش محسوب نمی‌شود. در مجموع حجم صفر یا دادهٔ ناقص، نسبت `NaN` و سمت `unavailable` است.

تحلیل مدیریتی بازار

این لایه دادهٔ تازه‌ای را حدس نمی‌زند؛ محاسبات آن روی خروجی‌های تاریخچه، حقیقی/حقوقی، نمای زنده و فعالیت روزانهٔ همین پکیج انجام می‌شود. هر تابع تحلیلی علاوه بر حالت دریافت خودکار، ورودی `DataFrame` صریح می‌پذیرد تا یک snapshot مشخص را بازپخش کنید، تست تکرارپذیر بسازید و از look-ahead جلوگیری شود.

مقایسهٔ نمادها

```
comparison = att.compare_symbols(
    ["فولاد", "فملی", "شپنا"],
    start="1404-01-01",
    benchmark="شاخص کل",
    align="inner",
    min_observations=20,
)

print(comparison[[
    "Symbol", "TotalReturnPct", "AnnualizedVolatilityPct",
    "MaxDrawdownPct", "Beta", "NetIndividualFlow"
]])
```

`benchmark` .تاریخ شمسی/میلادی را می‌پذیرند `end` و `start` .نمادهای غیرخالی است iterable یک نماد یا `symbols` فقط تاریخ‌های مشترک همهٔ سری‌ها را نگه `align='inner'` .اختیاری است و با همان قرارداد هویت به تاریخچه اضافه می‌شود .اتحاد تاریخ‌ها را نگه می‌دارد و هر متریک از مشاهدات موجود خودش محاسبه می‌شود `align='outer'` می‌دارد؛ حداقل بازده روزانهٔ لازم برای `min_observations` .تعداد جلسات سال، با پیش‌فرض ۲۴۰ و بازهٔ ۱ تا ۳۶۶ است `annualization` .تبدیل می‌شود `QualityFlags` و `NaN` آمار ریسک است؛ کمبود داده به

با `include_client_type=False` دریافت حقیقی/حقوقی حذف می‌شود. `history_data` و `client_data` نگاشت `{symbol: DataFrame}` برای محاسبهٔ آفلاین هستند و ورودی کاربر mutate نمی‌شود. `strict=True` برای نماد فاقد تاریخچه `DataParsingError` می‌دهد؛ در حالت پیش‌فرض ردیف همان نماد با flag برمی‌گردد. `progress` فقط نمایش پیشرفت دریافت چندنمادی را کنترل می‌کند.

خروجی شامل بازده کل/سالانه، نوسان سالانه، بیشینه افت، بهترین/بدترین روز، درصد روز مثبت، `Beta` ، همبستگی، آلفا، information ratio، tracking error، ارزش و حجم متوسط، ارزش متوسط هر معامله، خالص پول حقیقی و قدرت حقیقی است. ستون‌های مبلغی بر حسب **ریال** هستند. `Observations` تعداد قیمت‌ها، `ReturnObservations` تعداد بازده‌ها و `CoveragePct` پوشش سری روی تقویم هم‌تراز را ثبت می‌کنند.

سنجه‌های نقدشوندگی

```
liquidity = att.get_liquidity_metrics(
    ["فولاد", "فملی"],
    window=20,
    include_live=True,
)

print(liquidity[[
    "Symbol", "AverageDailyValue", "Amihud1B", "CurrentTurnover",
    "SpreadBps", "L5DepthValue", "ZeroTradeRatio"
]])
```

`symbols=None` است؛ اگر نماد بدهید، تاریخچهٔ هر نماد هم خوانده snapshot یک حالت سریع فقط-زنده برای تمام نمادهای `annualization` فقط .را به کار می‌گیرد `start/end` کل بازهٔ `None` تعداد آخرین جلسات تاریخی است و `window` .می‌شود `live_data` را حذف می‌کند و در این حالت دادن snapshot سنجه‌های `include_live=False` .نوسان را سالانه می‌کند `strict` ,است. `get_live_market()` خروجی صریح `live_data` نگاشت تاریخچه‌ها و `history_data` .خطاست `progress` , `start` و `end` .همان معنای بخش قبل را دارند

`Amihud1B` برابر میانگین قدرمطلق بازده تقسیم بر ارزش معامله به میلیارد ریال است و واحد آن در `AmihudUnit` .می‌آید تقسیم می‌کند؛ این فرض در **snapshot** حجم متوسط تاریخی را بر **تعداد سهام فعلی** `HistoricalTurnover` ثبت می‌شود. `CurrentTurnover` , `SpreadBps` , `L1Imbalance` , `SharesOutstandingSource` و `QualityFlags` زنده می‌آیند. عمق پنج‌سطحی مجموع snapshot از `L5Imbalance` , `L5DepthValue` , `MarketCap` و `VolumeToBaseVolume` .قیمت ضربدر حجم سفارش خرید و فروش است و واحد مبلغی آن ریال است `HistoryCoveragePct` و `LiveCoveragePct` .کامل‌بودن داده را نشان می‌دهند

رژیم توضیح‌پذیر بازار

```
regime = att.get_market_regime()
print(regime[[
    "Regime", "Score", "Confidence", "TrendScore", "BreadthScore",
    "FlowScore", "LiquidityScore", "QueueScore"
]])
```

پنجره‌های محاسبه را تعیین `annualization=240` و `lookback=60` , `liquidity_window=20` , `benchmark='شاخص کل'` است؛ وزن‌های پیش‌فرض `queue` , `liquidity` , `flow` , `breadth` , `trend` نگاشتی اختیاری با کلیدهای `weights` .می‌کنند سقف دریافت تاریخچهٔ `max_requests` .به‌ترتیب ۰٫۳۰، ۰٫۲۵، ۰٫۲۰، ۰٫۱۵ و ۰٫۱۰ هستند و پس از اعتبارسنجی نرمال می‌شوند سه ورودی صریح برای بازپخش کامل `activity_history` و `live_data` , `benchmark_history` .فعالیت روزانه است `progress` و `end` , `start` .محاسبه‌اند .دریافت خودکار را کنترل می‌کنند

پنج امتیاز در بازهٔ تقریبی منفی یک تا مثبت یک‌اند:

- `TrendScore` : نسبت به نوسان همان افق؛ benchmark بازده لگاریتمی
- `BreadthScore` : نسبت خالص نمادهای مثبت به منفی در ابزارهای سهامی نوع ۳۰۰، ۳۰۳ و ۳۰۹؛

- سازگار؛ snapshot خالص برآوردی پول حقیقی نسبت به ارزش معاملات همان ابزارهای سهامی و فقط برای : `FlowScore`
- بازار به میانهٔ ارزش روزانهٔ snapshot نسبت ارزش کل : `LiquidityScore` ؛ `get_market_activity(market='all')`
- .عدم تعادل ارزش صف خرید و فروش تازه در ابزارهای سهامی : `QueueScore`

دوباره نرمال score میانگین وزنی **مؤلفه‌های موجود** است. مؤلفهٔ گمشده صفر فرض نمی‌شود؛ وزن‌های باقی‌مانده برای `Score` می‌شوند و جمع وزن اولیهٔ موجود در `Confidence` می‌آید. آستانه‌های ثابت `Score > 0.25` برای `risk_on` ، `Score < -0.25` می‌شود. `Regime='unavailable'` ،است. اگر هیچ مؤلفه‌ای موجود نباشد `neutral` و فاصلهٔ میان آن‌ها برای `risk_off` برای ثبت می‌شوند؛ بنابراین این برچسب `DataFrame.attrs` در `is_partial` وزن‌ها، آستانه‌ها، فهرست مؤلفه‌های مفقود و .سیگنال معامله یا مدل پیش‌بینی نیست

نقشهٔ بازار؛ داده و نمودار

```
market_map = att.get_market_map(
    group_by="sector",
    size="market_cap",
    color="net_individual_flow",
    top=30,
)

fig = att.plot_market_map(
    market_map,
    output_path="market-map.html",
    show=False,
    title="نقشه جریان پول صنایع",
)
```

`get_market_map()` .مستقل از کتابخانهٔ رسم است `group_by` یکی از `symbol` , `sector` , `flow` , ؛ `instrument_type` , `size` یکی از `value` , `volume` , `market_cap` , و ؛ `queue_value` `color` یکی از `return` , `net_individual_flow` , `individual_power` , `order_imbalance` .است `top=None` .همهٔ گروه‌ها و عدد مثبت حداکثر ردیف‌ها را نگه می‌دارد `flow` همهٔ ابزارها و `'all'` فقط سهام ۳۰۹/۳۰۳/۳۰۰، مقدار `instrument_types=None` .یک یا چند کد جریان بازار را فیلتر می‌کند است؛ نبود آن یک `get_live_market()` صریح از snapshot یک `data` .مجموعهٔ دلخواه را انتخاب می‌کند iterable/یک مقدار .تازه می‌گیرد snapshot

در گروه‌بندی تجمیعی، بازده/قدرت/عدم تعادل با معیار `size` وزن‌دهی و جریان پول جمع می‌شود. `UniverseWeightPct` سهم از کل universe پیش از `top` و `DisplayedWeightPct` سهم از ردیف‌های نمایش‌داده‌شده است؛ پس بعد از برش، فقط دومی جمع ۱۰۰ می‌شود. `QualityFlags` , `IsRealtimeFresh` , `ColorCoveragePct` و attrs کیفیت نقشه را قابل بررسی می‌کنند.

رسم اختیاری است و dependency اصلی پکیج را سنگین نمی‌کند:

```
pip install "algotik-tse[visualization]"
```

.برمی‌گرداند Plotly از `Figure` یک `plot_market_map(map_data=None, output_path=None, show=False, title=None)` `show=True` تعاملی می‌نویسد؛ HTML فایل `output_path` را اجرا می‌کند؛ `get_market_map()` ابتدا `map_data=None` با دستور `ImportError` نصب نباشد Plotly عنوان نمودار است. اگر `title` را باز می‌کند و Plotly تنظیم‌شدهٔ renderer/مرورگر

قرارداد نقشه باشد و دادهٔ تهی یا ستون‌های ضروری ناقص DataFrame باید `map_data` .نصب بالا صادر می‌شود
.می‌دهد `DataParsingError`

شاخص‌ها و تحلیل صنایع

:صنعت در نسخهٔ 1.2.2 دو مفهوم را از هم جدا می‌کند API

- **عضویت رسمی شاخص:** نمادهایی که endpoint رسمی `GetIndexCompany` برای همان شاخص برمی‌گرداند. توابع این فصل به‌طور پیش‌فرض از این universe استفاده می‌کنند.
- **گروه دیده‌بان:** دسته‌بندی سریع `SectorCode` در MarketWatch که مبنای `get_sector_flow()` است و الزاماً با اعضای رسمی شاخص برابر نیست.

پارامترهای اختصاصی این خانواده عبارت‌اند از , `industry` , `industries` , `include_member_count` , `include_live` , `include_client_type` , `include_orderbook` , `include_empty` , `days` , `metric` , `top` , `refresh` و `max_workers` .
ورودی‌های عمومی `ascending` , `limit` , `end` , `start` , `progress` و `interval` همان معنای توضیح‌داده‌شده در هر تابع را دارند.

هویت عضوها فقط با `InsCode` متصل می‌شود. نام مشابه یا جست‌وجوی fuzzy برای join استفاده نمی‌شود. همچنین بعضی شاخص‌ها تجمیعی‌اند و عضویت صنایع می‌تواند هم‌پوشانی داشته باشد؛ بنابراین جمع‌زدن ردیف‌های تمام صنایع، کل بازار بدون تکرار تولید نمی‌کند.

مسیر سریع برای کاربر معمولی

```
import algotik_tse as att

# شاخص صنعت؛ سریع و تنها با یک درخواست ۴۵
indices = att.list_industry_indices(progress=False)

# اعضای رسمی یک صنعت همراه دادهٔ جاری بازار
members = att.get_industry_members("فلزات اساسی", progress=False)

# snapshot تحلیلی همهٔ صنایع همراه حقیقی/حقوق
snapshot = att.get_industry_snapshot(progress=False)

# همپوشانی رسمی اعضا برای شناسایی هم‌پوشانی بین صنایع
overlap = att.get_industry_membership_overlap(
    industries=["فلزات" ,"خودرو" ,"بانک"],
    progress=False,
)

# تغییرات عضویت رسمی هر روز در بازهٔ کوتاه
membership_events = att.get_industry_membership_events(
    industries=["خودرو" ,"بانک"],
    days=7,
    progress=False,
)

# پنج صنعت برتر از نظر breadth
leaders = att.rank_industries(
    metric="breadth",
    top=5,
    progress=False,
)
```

فهرست شاخص‌های صنعت

```
indices = att.list_industry_indices(
    progress=False,
    include_member_count=True,
    refresh=False,
    max_workers=6,
)
```

تهی می‌مانند. با `include_member_count=False` پیش‌فرض و سریع است؛ در این حالت `HasMembers` و `MemberCount` عضویت را دور می‌زند cache. `refresh=True` عضویت رسمی همهٔ صنایع دریافت و در حافظه cache می‌شود. `True`، درخواست‌های عضویت است concurrency و فقط سقف `1..16` عدد صحیح `max_workers`.

schema:

```
IndustryName, IndustryNameEn, IndustryGroupCode, IndexInsCode,
IndexValue, IndexPreviousValue, DayHigh, DayLow,
IndexChange, IndexChangePct, MemberCount, HasMembers, ExchangeTime
```

نسخه‌های `list_indices()` درصد تغییر است. این نگاشت در `IndexChangePct` تغییر واحد شاخص و `IndexChange` برعکس بود و در 1.2.0 اصلاح شده است `<=1.1.3`.

اعضای دقیق یک شاخص

```
members = att.get_industry_members(
    industry="بانک",
    include_live=True,
    include_client_type=True,
    include_orderbook=True,
    progress=False,
    refresh=False,
)
```

ورودی `industry` می‌تواند نام فارسی، alias شناخته‌شده مانند `بانک`، صورت کامل مانند `شاخص صنعت بانکها` یا `IndexInsCode` باشد.

- ادغام می‌کند MarketWatch جاری snapshot اعضای رسمی را با `include_live=True`.
- شده قابلreconcile قدرت خریدار و جریان حقیقی/حقوقی را اضافه می‌کند؛ فقط ردیف‌های `include_client_type=True` استفاده‌اند.
- `include_live=True` و صف تخمینی پنج سطح را اضافه می‌کند و به spread، imbalance `include_orderbook=True` نیاز دارد.
- می‌گیرد TSETMC عضویت را دوباره از `refresh=True`.

schema خروجی ثابت:

```
IndustryName, IndustryIndexCode, InsCode, Symbol, Name,
SectorCode, Flow, MarketCode, InstrumentType,
PreviousClose, Open, High, Low, Close, Last, Change, ChangePct,
TradeCount, Volume, Value, SharesOutstanding, EstimatedMarketCap,
IndividualPower, NetIndividualVolume, EstimatedNetIndividualFlow,
ClientDataAvailable, BidPrice1, AskPrice1, SpreadBps,
L1Imbalance, L5Imbalance,
EstimatedBuyQueueVolume, EstimatedBuyQueueValue,
EstimatedSellQueueVolume, EstimatedSellQueueValue,
TradeDate, ExchangeTime, FetchedAt, IsRealtimeFresh, IsStale
```

ستون‌های مربوط به client type یا order book وقتی درخواست نشده‌اند nullable می‌مانند. `EstimatedMarketCap` حاصل `SharesOutstanding × Close` و `EstimatedNetIndividualFlow` برآورد مبتنی بر VWAP است؛ هیچ‌کدام وزن رسمی شاخص نیستند.

در `DataFrame.attrs` مواردی مانند `universe='exact_index_members'`، `membership_is_current`، `historical_membership_available`، `cache_hit`، `client_type_requested`، `orderbook_requested` و `membership_count` ثبت می‌شوند.

snapshot تحلیلی صنعت

برای یک صنعت:

```
metals = att.get_industry_snapshot(
    industries="فلزات اساسی",
    include_client_type=True,
    include_orderbook=True,
    include_empty=False,
    progress=False,
    refresh=False,
    max_workers=6,
)
```

برای چند صنعت:

```
selected = att.get_industry_snapshot(
    industries=["بانک", "خودرو", "شیمیایی"],
    include_client_type=True,
    progress=False,
)
```

با `industries=None` تمام صنایع بررسی می‌شوند. `include_empty=False` شاخص‌هایی را که provider در آن لحظه عضو ندارند حذف می‌کند؛ نام آن‌ها در `attrs['empty_industries']` باقی می‌ماند. این تابع فقط یک MarketWatch bulk، حداکثر یک client-type bulk و عضویت‌های cacheشده را مصرف می‌کند.

گروه‌های اصلی خروجی:

گروه	ستون‌ها
وضعیت شاخص	`IndexValue`، `IndexPreviousValue`، `IndexChange`، `IndexChangePct`، `IndexDayHigh`، `IndexDayLow`
breadth	`MemberCount`، `TradedCount`، `Advances`، `Declines`، `Unchanged`، `NoTrade`، `AdvanceDeclineRatio`، `AdvancePct`، `DeclinePct`
عملکرد اعضا	`EqualWeightReturn`، `MedianReturn`، `ReturnDispersion`
معاملات	`TotalTradeCount`، `TotalVolume`، `TotalValue`، `MarketValueSharePct`، `UpperLimitCount`، `LowerLimitCount`
حقیقی/حقوقی	`ClientCoveredCount`، `ClientCoverage`، `NetIndividualVolume`، `EstimatedNetIndividualValue`، `IndividualPower`، `FlowValueMethod`

گروه	ستون‌ها
سفارش و صف	`OrderBookCoveredCount`, `OrderBookCoverage`, `BuyQueueCount`, `BuyQueueValue`, `SellQueueCount`, `SellQueueValue`
زمان و تازگی	`TradeDate`, `ExchangeTime`, `FetchedAt`, `IsRealtimeFresh`, `IsStale`

`IndividualPower`. میانگین سادهٔ بازده اعضای دارای قیمت معتبر است و بازده رسمی شاخص نیست `EqualWeightReturn` های سازگار باclient row از سرانهٔ تجمیعی خرید حقیقی به سرانهٔ تجمیعی فروش حقیقی ساخته می‌شود. جریان پول فقط از تازه و متعلق به روز order book را کنار آن کنترل کنید. صف‌ها فقط برای `ClientCoverage` حجم بازار جمع می‌شود؛ همیشه .تهران محاسبه می‌شوند

در `membership_is_current=True`, `memberships_may_overlap=True`، attrs
`historical_membership_available=False`, تعداد cache hit/request و روش تخمین جریان پول ثبت می‌شود.

همپوشانی اعضای رسمی صنعت‌ها

```
industry_overlap = att.get_industry_membership_overlap(
    industries=["فلزات" ,"خودرو" ,"بانک"],
    progress=False,
)
```

این تابع هم‌پوشانی رسمی اعضای هر جفت صنعت را در `GetIndexCompany` محاسبه می‌کند. خروجی long-form است و برای هر جفت unordered یک ردیف دارد:

```
IndustryAName, IndustryAIndexCode, IndustryBName, IndustryBIndexCode,
CommonMembers, UnionMembers, Jaccard, OverlapA_Pct, OverlapB_Pct
```

`CommonMembers` است و union تعداد منحصر به فرد اعضای `UnionMembers`، تعداد نمادهای مشترک در دو صنعت را می‌دهد `Jaccard` نسبت `CommonMembers/UnionMembers` است. `OverlapA_Pct` و `OverlapB_Pct` هم‌پوشانی نسبت به اندازهٔ هر .صنعت را نشان می‌دهند و برای شناسایی همپوشانی‌های نامتقارن مفید هستند

رویدادهای عضویت point-in-time برای صنایع

```
membership_events = att.get_industry_membership_events(
    industries=["بانک" ,"خودرو"],
    days=30,
    progress=False,
)
```

این تابع اختلاف اعضای رسمی هر صنعت را بین تاریخ‌های متوالی در `GetIndexCompany` payload نشان می‌دهد. خروجی long-form است و برای هر تغییر، ردیف با `EventType` برابر `added` یا `dropped` می‌آورد:

```
IndustryName, IndustryIndexCode, TradeDate, JalaliDate, PreviousTradeDate,
PreviousJalaliDate, EventType, InsCode, Symbol, Name
```

یعنی در تاریخ قبلی `EventType="dropped"` .یعنی نماد در تاریخ فعلی ظاهر شده و در تاریخ قبلی نبود `EventType="added"` معتبر است. `days=1..30` وجود داشته اما در تاریخ فعلی حذف شده است. این نگاه در حال حاضر فقط برای بازهٔ کوتاه.

تاریخچهٔ شاخص و اعضا

تاریخچهٔ خود شاخص:

```
history = att.get_industry_history(
    industry="شیمیایی",
    start="1404-01-01",
    end=None,
    limit=120,
    ascending=True,
    progress=False,
)
```

یعنی `limit=0` هستند. فیلتر تاریخ ابتدا اعمال می‌شود و `YYYY-MM-DD` / `YYYYMMDD` تاریخ شمسی یا میلادی `start` و `end` جلسه را نگه می‌دارد. خروجی N بدون محدودیت؛ مقدار مثبت آخرین
`IndustryName,IndustryIndexCode,TradeDate,JalaliDate,High,Low,Close,Change,ChangePct` است. منبع رسمی برای `attrs['volume_available']=False` روزانه نمی‌دهد؛ ستون حجم مصنوعی ساخته نمی‌شود و volume شاخص صنعت.است

تاریخچهٔ کوتاه همهٔ اعضای فعلی:

```
member_history = att.get_industry_members_history(
    industry="خودرو",
    days=30,
    ascending=True,
    progress=False,
    refresh=False,
)
```

long-form: رسمی است. خروجی payload و تعداد آخرین تاریخ معاملاتی موجود در `1..30` عدد صحیح `days`

```
IndustryName, IndustryIndexCode, TradeDate, JalaliDate,
InsCode, Symbol, Name, Close, Last, Change, ChangePct,
TradeCount, Volume, Value
```

این history برای **اعضای فعلی** شاخص است، نه عضویت point-in-time. اگر ترکیب شاخص تغییر کرده باشد، survivorship bias محتمل است؛ به همین دلیل attrs صریح `point_in_time_membership=False` و
`survivorship_bias_possible=True` دارد.

دادهٔ درون‌روزی شاخص صنعت

```
intraday = att.get_industry_intraday(
    industry="فلزات",
    interval="5min",
    progress=False,
)
```

مقادیر مجاز `interval` عبارت‌اند از `1h` ، `60min` ، `30min` ، `15min` ، `5min` ، `1min` ، `raw` . حالت `raw` هر مشاهدهٔ provider را به‌صورت یک کندل تک‌نقطه‌ای نگه می‌دارد؛ سایر حالت‌ها سطح شاخص را به OHLC تبدیل می‌کنند.

```
IndustryName, IndustryIndexCode, Timestamp, JalaliDate, Interval,
Open, High, Low, Close, Change, ChangePct
```

مصنوعی تولید نمی‌کند و فقط turnover یا volume حجم ندارد؛ تابع endpoint تهران است. این timezone دارای `Timestamp` .را برمی‌گرداند provider آخرین روز موجود در

رتبه‌بندی صنایع

```
ranking = att.rank_industries(
    metric="money_flow",
    top=10,
    ascending=False,
    include_client_type=True,
    include_orderbook=False,
    progress=False,
    refresh=False,
    max_workers=6,
)
```

:های پشتیبانی‌شده‌alias و `metric`

معیار canonical	های رایجalias
`IndexChangePct`	`change_pct` , `return`
`EqualWeightReturn`	`equal_weight_return`
`MedianReturn`	`median_return`
`AdvancePct`	`advance_pct` , `breadth`
`TotalValue`	`total_value` , `turnover`
`EstimatedNetIndividualValue`	`estimated_net_individual_value` , `money_flow`
`IndividualPower`	`individual_power` , `buyer_power`

`ascending=False` .ردیف اول را نگه می‌دارد N همهٔ ردیف‌های دارای معیار معتبر را برمی‌گرداند؛ مقدار مثبت فقط `top=None` دارد و معیار نهایی در `Rank` را همراه snapshot رتبهٔ بزرگ‌تر به کوچک‌تر است. خروجی تمام ستون‌های `attrs['ranking_metric']` .ثبت می‌شود

مقایسه و همبستگی شاخص‌های صنعت

```
compare = att.compare_industries(
    industries=["فلزات" ,"بانک" ,"خودرو"],
    metric="close",
    limit=20,
    ascending=True,
    progress=False,
)
```

در خروجی، برای هر تاریخ یک ردیف است و هر ستون یک شاخص مقایسه‌شده است. ستون‌های خروجی به این شکل هستند:

```
TradeDate, JalaliDate, صنعت خودرو [ID], صنعت بانک [ID], صنعت فلزات [ID]
```

```
relative = att.get_industry_relative_strength(
    industries=["فلزات" ,"شیمیایی" ,"بانک"],
    benchmark="خودرو",
    metric="close",
    limit=30,
    ascending=False,
    progress=False,
)
```

این خروجی long-form است و برای هر industry (به جز benchmark) ستون‌های `IndustryReturn` , `BenchmarkReturn` , `RelativeStrength` را تولید می‌کند.

```
TradeDate, JalaliDate, BenchmarkIndexCode, BenchmarkName,
IndustryIndexCode, IndustryName, IndustryReturn, BenchmarkReturn, RelativeStrength
```

```
corr = att.get_industry_correlation(
    industries=["ساختمان" ,"صنعت فولاد" ,"خودرو" ,"بانک"],
    limit=120,
    ascending=False,
    progress=False,
)
```

خروجی تابع ماتریس همبستگی n×n است با index/columns برچسب‌دار به فرمت `IndustryName [IndexCode]` .

هر یک از توابع بالا همانند زیر امضای رسمی‌شان را دارند:

```
compare_industries(industries, start=None, end=None, limit=0, metric='close', ascending=True,
progress=True, max_workers=6)
get_industry_relative_strength(industries, benchmark, start=None, end=None, limit=0, metric='close',
ascending=True, progress=True, max_workers=6)
get_industry_correlation(industries, start=None, end=None, limit=0, ascending=True, progress=True,
max_workers=6)
get_industry_membership_overlap(industries, progress=True, refresh=False, max_workers=6)
get_industry_membership_events(industries, days=30, progress=True, refresh=False, max_workers=6)
```

cache، پوشش و محدودیت داده

عضویت کامل شاخص و تاریخچهٔ کوتاه اعضا در یک payload مشترک می‌آیند. TTL پیش‌فرض cache حافظه یک ساعت است:

```
att.settings.industry_membership_cache_ttl = 3600.0
```

مقدار cache `0` را غیرفعال می‌کند. برای دریافت اجباری عضویت تازه از `refresh=True` استفاده کنید. cache فقط در حافظهٔ همان process است و فایلی روی دیسک نمی‌نویسد.

محدودیت‌های رسمی این نسخه:

- وزن رسمی هر عضو، ضریب سهام شناور و divisor شاخص در منبع فعلی ارائه نمی‌شود؛ contribution دقیق نماد به واحد شاخص محاسبه نمی‌شود.
- فعلی ساخته می‌شود و ممکن است در صورت تغییر ترکیب universe بر پایه `get_industry_members_history()` `survivorship` داشته باشد؛ برای تغییرات دقیق اعضا از `get_industry_membership_events()` شاخص، بایاس .استفاده کنید
- صنایع می‌توانند هم‌پوشانی داشته باشند و بعضی کدهای صنعت ممکن است در یک روز بدون عضو باشند.
- برآوردند و نام آن‌ها عمداً `EstimatedMarketCap` , `EstimatedNetIndividualFlow` و `EstimatedNetIndividualValue` .این موضوع را نشان می‌دهد
- برای تصمیم معاملاتی، `IsRealtimeFresh` , `IsStale` , `ClientCoverage` , `OrderBookCoverage` , `TradeDate` و attrs را بررسی کنید.

تاریخچهٔ محلی SQLite

ذخیره‌سازی کاملاً opt-in است؛ بدون path صریح هیچ فایلی نوشته نمی‌شود.

snapshotهای بازار

```
db = "market-history.sqlite"

snapshot_id = att.save_market_snapshot(db)
history = att.load_market_snapshots(db, symbol="فملی", limit=500)

# alias معنایی برای همان نمای ذخیره‌شده live
live_history = att.get_live_market_history(db, symbol="فملی", limit=500)
breadth_history = att.get_market_breadth_history(db, limit=100)
sector_history = att.get_sector_flow_history(db, limit=100)
overview_history = att.get_market_snapshot_summary_history(db, limit=100)
```

خروجی نماینده:

```
AsOf                        InsCode Symbol Last Close Volume NoBackfill SnapshotAtomic
2026-08-25 10:11:12+03:30   ...     812200 73950 74200  فملی       True           True
```

و `CoverageStart`، `CoverageEnd`، `SnapshotFrameAttrs`، `NoBackfill`، `SnapshotAtomic` هایی مانندattrs `SourceAtomic` محدوده و کیفیت archive را توضیح می‌دهند. SQLite دارای application-id، schema version، و کنترل دیتابیس بیگانه/خراب است transaction.

`record_to` برای replay eventها

```
watcher = att.MarketWatcher(
    interval=2,
    max_updates=3,
    record_to=db,
    checkpoint_interval=100,
    record_max_records=10_000,
    record_retention_seconds=7 * 24 * 3600,
    storage_error_policy="raise",
)
for _ in watcher:
    pass

events = att.get_market_event_history(db, kind="delta", limit=1000)
```

اولین event هر session یک checkpoint کامل است؛ deltaها پس از آن replay می‌شوند و pruning فقط از مرز checkpoint معتبر انجام می‌شود. `storage_error_policy="raise"` انتخاب امن پیش‌فرض است. `record_market_event()` برای ثبت مستقیم `MarketEvent` موجود است.

`archive_to` برای رکوردهای مستقل

دو مسیر مستقل‌اند: `archive_to` و `record_to`:

```
att.get_market_overview(flow=0, archive_to=db)
att.get_market_messages(flow=0, top=20, archive_to=db)
att.get_instrument_state_changes(top=20, archive_to=db)

overview = att.get_market_overview_history(db, flow=0, limit=100)
messages = att.get_market_messages_history(db, flow=0, limit=100)
states = att.get_instrument_state_changes_history(db, symbol="فملی", limit=100)
```

`archive_market_records(path, kind, frame, source=...)` API سطح پایین برای kindهای پشتیبانی‌شده است. `limit/offset` و `start/end` هستند؛ archive بخشی از هویت `source` و selector اعمال SQL در pagination قبل از می‌شوند.

توابع مهم نگهداری:

```
info = att.check_market_history(db)
att.record_market_event(db, event, session_id="strategy-a")
```

`MARKET_HISTORY_APPLICATION_ID` و `MARKET_HISTORY_SCHEMA_VERSION` برای migration/inspection عمومی‌اند.

فاندامنتال بازار، صندوق و تعدیل قیمت

EPS و P/E زنده

```
fundamentals = att.get_market_fundamentals(
    symbols=["فملی", "شتران"],
    pe_min=None,
    pe_max=None,
    positive_pe=False,   # ردیف unavailable را هم برای audit نگه دار
    strict=False,
    allow_stale=False,
    archive_to="market-history.sqlite",
    max_requests=1,
    progress=False,
)
```

این تابع یک snapshot bulk می‌گیرد و EPS/P/E را از همان مشاهده می‌سازد؛ برای هر نماد سراغ منبع دیگری نمی‌رود. schema ثابت:

```
InsCode, Symbol, Name, SectorCode, InstrumentType, Close, Last,
EPS, PE, PECalculated, EPSSource, PriceSource, PEStatus,
TradeDate, ExchangeTime, AsOf, SnapshotAgeSeconds,
IsRealtimeFresh, IsStale, Source, NoLookahead
```

خروجی نماینده:

```
Symbol Close   EPS    PE PECalculated EPSSource              PriceSource PEStatus     NoLookahead
10.0  7395  73950   فملی          True market_watch          Close       ok           True
42100  شتران  <NA>  <NA>          True missing_in_market_watch Close      eps_missing  True
```

`PEStatus` یکی از `ok`، `price_missing`، `price_nonfinite`، `eps_nonpositive`، `eps_missing`، `eps_nonfinite` یا `IsStale` جداست. `IsStale` بودن در stale است؛ صفر/خالی به مقدار جعلی تبدیل نمی‌شود EPS. `positive_pe=True` فقط P/E مثبت را نگه می‌دارد و `strict=True` selector مفقود را به خطا تبدیل می‌کند. snapshot زنده attrs می‌شود. `attrs['stale_rejected']=True` نیست: خروجی تهی و exception با `allow_stale=False` stale دقیقاً شامل `request_count,max_requests,missing_selectors,strict,stale_rejected,source,price_source,eps_source,no_lookahead,no_backfill,archive_path` است.

تاریخچهٔ fundamentals

```
history = att.get_market_fundamentals_history(
    "market-history.sqlite",
    start="1403-05-01",
    end="1403-05-07",
    symbols="فملی",
    limit=1000,
)
print(history.attrs["no_lookahead"], history.attrs["current_eps_used"])
```

```
AsOf                       Symbol Close  EPS   PE  EPSSource    Source                 NoLookahead
2026-08-24 10:00:00+03:30   10.0 7350 73500   فملی market_watch local_market_snapshot   True
2026-08-25 10:00:00+03:30   10.0 7420 74200   فملی market_watch local_market_snapshot   True
```

این history فقط از snapshotهای ذخیره‌شدهٔ شما ساخته می‌شود: `attrs['no_backfill']=True` و `attrs['current_eps_used']=False` دیگر. attrs `missing_selectors,strict,source,price_source,eps_source,no_lookahead,coverage_start,coverage_end` هستند. EPS فعلی برای گذشته forward-fill نمی‌شود.

صندوق‌های قابل معامله

```
listed = att.list_listed_funds(progress=False)

gold = att.list_listed_funds(
    fund_category="commodity",
    commodity_underlying="gold",
    progress=False,
)
silver = att.list_listed_funds(commodity_underlying="silver", progress=False)

sector_registry = att.list_funds(fund_type="sector", progress=False)
leveraged_registry = att.list_funds(fund_type="leveraged", progress=False)
index_registry = att.list_funds(fund_type="index", progress=False)
```

و (صندوق بورسی عمومی) `InstrumentType=305` هر دو گروه bulk call با یک `list_listed_funds()` NAV، صندوق‌ها می‌آید و اطلاعاتی مثل registry از `list_funds()` .را می‌گیرد (صندوق کالایی بورسی) `InstrumentType=380` .بازده، ترکیب دارایی و مدیر را برمی‌گرداند. این دو منبع شناسهٔ مشترک قطعی ندارند و با شباهت نام به هم متصل نمی‌شوند

«کالایی» مترادف «طلا» نیست و ETF نیز سازوکار معامله است، نه طبقه دارایی. طلا، نقره و زعفران فقط از نگاشت دقیق `InsCode+ISIN` یا عبارت صریح در نام رسمی تشخیص داده می‌شوند. عباراتی مثل «زر»، «گوهر» یا «سیمین» به‌تنهایی مدرک نیستند. اگر مدرک کافی وجود نداشته باشد، مقدار `unknown` برمی‌گردد و `needs_review=True` می‌شود؛ پوشش زیرنوع‌ها ممکن است کامل نباشد.

:فهرست بورسی schema

```
InsCode, ISIN, Symbol, Name, InstrumentType, Flow, MarketCode,
Last, Close, Yesterday, Volume, Value, TradeCount, Low, High, NAV,
NAV_Discount, Change, ChangePct, fund_category, asset_exposure,
strategy_tags, trading_mechanism, unit_class, commodity_profile,
commodity_underlyings, primary_commodity, classification_status,
classification_source, classification_evidence, taxonomy_version, needs_review
```

```
Symbol InsCode ISIN          Last Close NAV NAV_Discount Volume
افران  ...     IRO3AFRZ0001  21650 21620 ... ...          1250040
```

در خروجی registry نیز , `registry_type_id` , `registry_category` , `asset_exposure` , `strategy_tags` , `classification_status` , `classification_source` , `classification_evidence` , `taxonomy_version` , `record_date` و `needs_review` در کنار ستون‌های قبلی حفظ می‌شوند. ثابت `FUND_TAXONOMY_VERSION` نسخهٔ مرجع طبقه‌بندی را نشان می‌دهد. attrsهای `no_fuzzy_join=True` و `registry_joined=False` قرارداد هویتی را روشن می‌کنند.

رخدادهای تعدیل قیمت

```
adjustments = att.get_price_adjustments(
    "فملی", start="1402-01-01", end="1403-12-29", progress=False
)
latest = att.get_latest_price_adjustment("فملی", progress=False)
```

typed: ثابت و schema

```
InsCode, Symbol, GregorianDate, JalaliDate,
AdjustedClosingPrice, UnadjustedClosingPrice, AdjustmentAmount,
CorporateTypeCode, CorporateActionType, IsConfirmedDPS,
IdentityVerified, Source, FetchedAt
```

خروجی نماینده:

```
GregorianDate JalaliDate AdjustedClosingPrice UnadjustedClosingPrice AdjustmentAmount CorporateTypeCode
CorporateActionType IsConfirmedDPS IdentityVerified
2024-01-01    1402-10-11                 8000                  10000            2000                 7
<NA>          False             True
```

قرارداد مهم:

- `AdjustmentAmount = UnadjustedClosingPrice - AdjustedClosingPrice` فقط اختلاف ریاضی قیمت‌هاست.
- provider خام `CorporateTypeCode` است؛ nullable و `CorporateActionType` همیشه `False` و `IsConfirmedDPS` بدون تفسیر نگه داشته می‌شود.
- `attrs['dps_reason']` ثبت می‌شود و دلیل در `attrs['dps_available'] == False`.
- رکورد با InsCode متفاوت رد می‌شود؛ نبود InsCode در خود رکورد با `IdentityVerified=False` و InsCode حل‌شده گزارش می‌شود.
- `get_latest_price_adjustment()` صفر یا یک‌ردیفی با همان DataFrame schema/dtypes/attrs می‌دهد.

```
assert adjustments["IsConfirmedDPS"].eq(False).all()
assert adjustments.attrs["dps_available"] is False
```

از این API برای ساخت «سری DPS» استفاده نکنید. TSETMC در این endpoint طبقه‌بندی قطعی سود نقدی ارائه نمی‌دهد.

اخزا و درآمد ثابت

قاعدهٔ تاریخ در نماد اخزا

شش رقم انتهای نماد به‌صورت تاریخ شمسی `YYMMDD` تفسیر می‌شود؛ دو رقم سال با `13` یا `14` تکمیل می‌شود:

```
att.parse_treasury_maturity("020322اخزا")
```

```
{
  'maturity_jalali': '1402/03/22',
  'maturity_gregorian': datetime.date(2023, 6, 12),
  'maturity_source': 'user_confirmed_symbol_jalali_yymmdd'
}
```

```
att.parse_treasury_maturity("991117اخزا")
```

```
{
  'maturity_jalali': '1399/11/17',
  'maturity_gregorian': datetime.date(2021, 2, 5),
  'maturity_source': 'user_confirmed_symbol_jalali_yymmdd'
}
```

برای نماد non-match یا تاریخ شمسی نامعتبر، تابع exception نمی‌دهد و `None` برمی‌گرداند. در صورت تعارض یا نماد غیرقابل‌تفسیر، `maturity_date` یا `maturity_map` صریح بدهید؛ provenance ستون `MaturitySource` را بررسی کنید.

محاسبات deterministic

```
result = att.treasury_yield(
    price=800_000,
    maturity_date="2028-01-01",
    settlement_date="2026-01-01",
    face_value=1_000_000,
)
```

خروجی واقعی این مثال:

```
EffectiveAnnualYield  0.1180339887
SimpleAnnualYield     0.125
ContinuousYield       0.1115717757
BankDiscountYield     0.0986301370
MacaulayDuration      2.0
ModifiedDuration      1.7888543820
Convexity             4.8
DV01                   143.10835056
DiscountFactor        0.8
Status                ok
```

برای اوراق کوپنی:

```
cashflows = [
    ("2027-01-01", 80_000),
    ("2028-01-01", 80_000),
    ("2029-01-01", 1_080_000),
]
price = att.bond_price(0.08, cashflows, settlement_date="2026-01-01")
ytm = att.yield_to_maturity(price, cashflows, settlement_date="2026-01-01")
risk = att.bond_analytics(price, cashflows, settlement_date="2026-01-01")
```

`bond_price`, `yield_to_maturity` و `bond_analytics` پارامترهای compounding/frequency، clean/dirty price و accrued interest. پشتیبانی می‌کند `30/360`، `ACT/360`، `ACT/365F` و از قراردادهایی مانند `day_count_fraction()` رد می‌شوند sign-changing های مبهم یاcashflow دارند؛

اخزای زنده

```
ytm = att.get_treasury_yields(
    symbol=None,
    settlement_date=None,
    face_value=1_000_000,
    include_stale=False,
    min_volume=1,
    price_source="auto",   # auto/bid/ask/last/close
    strict=False,
    source="tsetmc",      # یا hybrid برای join مرجع IFB
)
```

ستون‌های اصلی:

```
InsCode, ISIN, Symbol, MaturityJalali, Maturity, MaturitySource,
MaturityConflict, SettlementDate, DaysToMaturity, Tenor,
Price, PriceSource, NoTrade, InstrumentPriceAsOf, IsInstrumentStale,
BidPrice, AskPrice, DiscountFactor,
EffectiveAnnualYield, ContinuousYield, SimpleAnnualYield, BankDiscountYield,
MacaulayDuration, ModifiedDuration, Convexity, DV01,
Volume, Value, TradeCount, FaceValue, FaceValueSource,
DayCount, IsStale, SnapshotAgeSeconds, FetchedAt, Status
```

مقایسه `https://ifb.ir/ytm.aspx` را با جدول مرجع مجاز فرابورس ایران در TSETMC دادهٔ معاملاتی `source="hybrid"` گزارش می‌شود provenance با bps جای قیمت ابزار را نمی‌گیرد و اختلاف در IFB می‌کند؛

```
ifb = att.get_ifb_yield_table(category="treasury")
print(ifb.attrs["reference_url"])
```

schema IFB:

```
Symbol, Price, LastTradeJalali, LastTradeDate, PublishJalali, PublishDate,
MaturityJalali, Maturity, Volume, ReferenceYTM,
ReferenceSimpleYield, ReferenceSource
```

اراد و گام

نمی‌کرد. اکنون امضای قدیمی universe می‌شناخت، اما گام را وارد `murabaha` از نسخه‌های قبلی اراد را در گروه `list_bonds()` تاریخ سررسید شش یا هشت‌رقمی و قیمت/حجم روز دیده می‌شود. برای `BondType='gam'` آن بدون تغییر مانده و گام نیز با جدید استفاده کنید API فیلتر روشن نوع بدهی از:

```
debt = att.list_debt_instruments(
    debt_type=["erad", "gam"],
    active_only=True,
    progress=False,
)

gam = att.list_debt_instruments("gam", progress=False)
print(gam[["Symbol", "MaturityJalali", "DaysToMaturity", "Last"]])
```

برابر `گام` و پیشوند `erad` برابر `اراد` پیشوند، `treasury` برابر `اخزا` با قواعد صریح تعیین می‌شود: پیشوند `DebtType` است و TSETMC فهرست و قیمت امروز snapshot قدیمی نیز حفظ شده تا کد قبلی نشکند. این `BondType` است. `gam` تاریخچهٔ بازده محسوب نمی‌شود.

برای YTM منتشرشدهٔ فرابورس:

```
reference = att.get_debt_yields(["erad", "gam"])
print(reference[["Symbol", "DebtType", "Price", "MaturityJalali", "ReferenceYTM"]])
```

این تابع نمای `all` جدول رسمی `ifb.ir/ytm.aspx` را می‌خواند. بازده درصدی منبع به عدد اعشاری تبدیل می‌شود؛ مثلاً ۳۵٪ برابر `0.35` است. پکیج برای اراد و گام ارزش اسمی، کوپن یا جریان نقدی حدس نمی‌زند (`cashflows_inferred=False`). برای ارزش‌گذاری مستقل اوراق کوپنی باید cashflowهای قطعی قرارداد را به `yield_to_maturity()`، `bond_price()` یا `bond_analytics()` بدهید. اگر صفحهٔ فرابورس در دسترس نباشد `ConnectionError` می‌گیرید؛ این خطا با بازده محاسبه‌شدهٔ ساختگی جایگزین نمی‌شود.

تاریخچهٔ YTM

```
history = att.get_treasury_yield_history(
    "090101اخزا",
    limit=20,
    include_today=True,
    price_source="close",
    max_requests=25,
    progress=False,
)
```

عینی همین تابع است. هر ردیف از قیمت همان روز ساخته می‌شود و alias `get_treasury_yields_history` , `Final` , `Close`، OHLC مناسب نیست. ستون‌های تاریخچه شامل YTM است؛ قیمت تعدیل‌شده برای `auto_adjust=False` از همان price- `include_today` .سررسید/قیمت است provenance و `Status` , `FetchedAt` , `IsPartial`، بالا analytics .استفاده می‌کند live متناظر source

منحنی بازده

```
curve = att.get_yield_curve(
    min_nodes=3,
    interpolation="log_discount",
    duplicate_policy="volume_weighted",
    extrapolate=False,
)

df_18m = curve.discount_factor(1.5)
zero_18m = curve.zero_rate(1.5)
fwd = curve.forward_rate(1.0, 2.0)
```

`YieldCurve` شامل `settlement_date` , `maturities` , `times` , `discount_factors` , `continuous_zero_rates` , .جلوی استفادهٔ خارج از دامنه را می‌گیرد `extrapolate=False` .است `diagnostics` و `node_metadata`

ساخت مستقیم:

```
nodes = [
    {"Maturity": "2027-01-01", "DiscountFactor": 0.90},
    {"Maturity": "2028-01-01", "DiscountFactor": 0.80},
    {"Maturity": "2029-01-01", "DiscountFactor": 0.70},
]
curve = att.build_yield_curve(nodes, settlement_date="2026-01-01")
```

و `duplicate_policy` , `duplicate_count` , `node_count` , `input_node_count` دقیقاً کلیدهای `curve.diagnostics` discount factor نزولی‌بودن guard و dedupe، ورودی audit برای metadata را دارد؛ این `monotonic_discount_enforced` .است

تاریخچهٔ منحنی:

```
curves = att.get_yield_curve_history(
    limit=10,
    min_nodes=3,
    include_today=True,
    max_requests=50,
    progress=False,
)
```

هر node دارای `CurveID`، `CurveStatus`، `CurveNodeCount`، `CurveInterpolation` و flagهای `CurveNoLookahead`، `CurveUniverseNoLookahead`، `CurvePricesNoLookahead` است. اگر universe تاریخی از catalog امروز بازیابی شود، survivor-bias در diagnostics صریح است و `CurveUniverseNoLookahead=False` می‌شود.

اختیار معامله

تحلیل‌های قیمت‌گذاری این بخش برای اختیار **اروپایی** هستند. مشخصات واقعی اعمال قرارداد، تعدیلات شرکتی و dividend yield از TSETMC حدس زده نمی‌شود.

ریاضی Black–Scholes

```
call = att.black_scholes_price(
    spot=100, strike=100, time_to_expiry=1,
    rate=0.05, volatility=0.20,
    option_type="call", dividend_yield=0.0,
)
greeks = att.black_scholes_greeks(100, 100, 1, 0.05, 0.20, "call")
bounds = att.option_price_bounds(100, 100, 1, 0.05, "call")
iv = att.implied_volatility(10.4505835722, 100, 100, 1, 0.05, "call")
```

خروجی واقعی و deterministic:

```
call = 10.4505835722

greeks = {
  'Delta': 0.6368306512,
  'Gamma': 0.0187620173,
  'Vega': 37.5240346917,
  'Vega1Pct': 0.3752403469,
  'ThetaPerYear': -6.4140275464,
  'ThetaPerDay': -0.0175726782,
  'Rho': 53.2324815454,
  'Rho100bp': 0.5323248155,
  'Status': 'ok'
}

bounds = (4.8770575499, 100.0)
iv = {'ImpliedVolatility': 0.1999999955, 'Status': 'ok', 'Iterations': 27}
```

واحدها مهم‌اند: `Vega` تغییر قیمت برای یک واحد volatility و `Vega1Pct` برای یک واحد درصد است؛ `ThetaPerYear/Day` و `Rho/Rho100bp` جدا گزارش می‌شوند. خروجی Greeks همیشه کلید `Status` دارد. `implied_volatility()` همواره `dict` می‌دهد و کلیدهای پایهٔ آن `ImpliedVolatility, Status, Iterations` هستند؛ status یکی از `ok`، `missing`، `expiry`، `non_converged`، `no_bracket`، `out_of_bounds` است. out-of-bounds/no-bracket ممکن است `LowerBound/UpperBound` و non-converged مقدار تشخیصی `CandidateVolatility` داشته باشد؛ عدم همگرایی exception نیست. فقط constraintهای ورودی مانند bracket/tolerance/style نامعتبر `ValueError` می‌دهند.

snapshot اتمیک بازار اختیار

```
options = att.get_option_market(
    exchange=0,
    underlying="خودرو",
    max_requests=1,
    progress=False,
)
```

هر جفت call/put فقط وقتی metadata کافی و هویت سازگار داشته باشد وارد snapshot می‌شود. ستون‌های اصلی:

```
InsCode, PairID, PairSequence, ISIN, Symbol, Name, OptionType,
UnderlyingInsCode, UnderlyingSymbol, ContractSize, Strike,
BeginDate, EndDate, DaysToExpiry,
Last, Close, Yesterday, Volume, Value, TradeCount, NotionalValue,
OpenInterest, YesterdayOpenInterest, BidPrice, AskPrice, BidVolume, AskVolume,
UnderlyingLast, UnderlyingClose, Price, PriceSource,
AsOf, AsOfSource, SnapshotFreshnessKnown, PriceFreshnessKnown,
Stale, NoTrade, AnalyticsEligible, AnalyticsEligibilityReason,
MetadataConflict, Source
```

`atomic_snapshot`، `duplicate_pairs_dropped`، `incomplete_pairs_quarantined` هایattrs، `malformed_pair_status` و `exchange_event_freshness_known` کیفیت snapshot را توضیح می‌دهند.

`get_options_chain(underlying, fetch_oi=False)` قدیمی API فهرست قراردادها را می‌دهد و `list_options(underlying=None)` یک `dict` با کلیدهای دقیق `calls`، `puts`، `underlying_name`، `underlying_price`، `expiry_dates` و `market_time` پیشنهاد می‌شود؛ کلید `price` برمی‌گرداند؛ برای `get_option_market()` حرفه‌ای analytics وجود ندارد.

IV، Greeks، parity و نقدشوندگی

```
analysis = att.analyze_option_chain(
    options=options,
    spot=None,                    # از snapshot اگر معتبر باشد
    risk_free_rate=0.30,          # یا yield_curve=curve
    dividend_yield=0.0,
    valuation_date=None,
    exercise_style="european",
    parity_tolerance=None,
    allow_unverified_freshness=True,
)
```

ستون‌های تحلیلی افزوده‌شده:

```
TimeToExpiry, Spot, SpotSource, RiskFreeRate, RiskFreeRateSource,
DividendYield, DividendYieldSource,
ImpliedVolatility, ImpliedVolatilityBid, ImpliedVolatilityMid, ImpliedVolatilityAsk,
IVStatus, IVStatusBid, IVStatusMid, IVStatusAsk,
Delta, Gamma, Vega, Vega1Pct, ThetaPerYear, ThetaPerDay, Rho, Rho100bp,
PremiumContract, DeltaContract, GammaContract, VegaContract,
Vega1PctContract, ThetaPerYearContract, ThetaPerDayContract,
RhoContract, Rho100bpContract,
SpreadAbs, SpreadPct, QuotedDepth, LiquidityScore,
ParityResidual, ParityToleranceBand, ParityStatus, ImpliedForward,
GreeksStatus, AnalyticsReliability, AnalyticsWarning, AnalyticsComputed
```

خروجی نماینده:

```
Symbol OptionType Strike Price ImpliedVolatility Delta Vega1PctContract SpreadPct LiquidityScore
ParityStatus AnalyticsReliability
ضخود... call       3000  420.0             0.41   0.62          18320.0      0.03          0.81
ok verified_inputs
طخود... put        3000  265.0             0.39  -0.38          17790.0      0.04          0.76
ok verified_inputs
```

است و توصیهٔ آربیتراژ نیست؛ محدودیت وجه تضمین، سبک اعمال، کارمزد و امکان معامله را diagnostic فقط `ParityStatus` .قابل اتکا ساخته نمی‌شود analytics معتبر نداشته باشید curve ندهید و `risk_free_rate` مدل نمی‌کند. اگر فقط زمانی ثبت می‌شود که کاربر مقدار را تعیین کرده باشد؛ مقدار صفر پیش‌فرض `DividendYieldSource='user_supplied'` .نیست DPS به معنی کشف

PCR

```
pcr = att.option_put_call_ratios(analysis, group_by="underlying")
```

```
UnderlyingInsCode CallVolume PutVolume PCRVolume PCRVolumeStatus CallValue PutValue PCRValue
PCRValueStatus CallOpenInterest PutOpenInterest PCROpenInterest PCROpenInterestStatus
65883838195688438      20000     13000      0.65 ok              9.2e9     5.1e9    0.55     ok
40000           20000           0.50 ok
```

است. خروجی برای هر معیار علاوه بر `underlying_expiry` یا `expiry` , `underlying` , `market` دقیقاً یکی از `group_by` را می‌دهد؛ نسبت با `PCROpenInterestStatus` و `PCRValueStatus` , `PCRVolumeStatus` متناظر status ،نسبت است. `zero_denominator` برابر status و nullable صفر denominator

تاریخچهٔ اختیار و snapshot محلی

```
# تاریخچهٔ server-side قیمت قرارداد؛ ردیف امروز opt-in
history = att.get_option_history(
    "ضخود...", limit=10,
    include_today=True, max_requests=3, progress=False,
)

# snapshot بازار اختیار فقط با درخواست صریح کاربر روی فایل نوشته می‌شود.
path = "option-snapshots.json"
att.save_option_snapshot(path, options=options)
saved = att.load_option_snapshots(path)
```

schema history:

```
Timestamp, InsCode, Symbol, Open, High, Low, Close, Last,
Volume, Value, TradeCount, OpenInterest,
BidPrice, AskPrice, BidVolume, AskVolume,
UnderlyingLast, UnderlyingClose, ContractSize, Strike, EndDate,
Price, PriceSource, Source, AsOf, Stale, NoTrade, AnalyticsEligible
```

`prices_no_lookahead`، `underlying_prices_no_lookahead`، `rates_no_lookahead`، هایی مانندattrs دارای snapshot بررسی کنید. فایل backtest را برای `source_coverage` و `curve_applied`، `valuation_date_source` قفل، `OPTION_SNAPSHOT_SCHEMA_VERSION` اتمیک و dedupe، writer، write است.

بورس انرژی و بورس کالا

نسخهٔ ۱.۶.۰ سه لایه را از هم جدا نگه می‌دارد: دادهٔ حراج و ابزارهای بورس انرژی از TSETMC، تابلوی مالی و گزارش‌های بازار فیزیکی از وب‌سایت رسمی بورس کالای ایران، و محاسبات مشتقه که فقط از ورودی صریح کاربر ساخته می‌شوند. بنابراین یک ستون با واحد «هزار ریال» هیچ‌گاه بدون اعلام به ریال تبدیل نمی‌شود و سررسید قرارداد از روی نام فارسی حدس زده نمی‌شود.

حراج‌های بورس انرژی

```
auctions = att.get_energy_auctions(
    status="upcoming",
    board="physical",
    top=20,
)

one = att.get_energy_auction(
    auctions.iloc[0]["AuctionID"],
    include_trades=True,
    include_instrument_history=False,
)
details = one["auction"]
trades = one["auction_trades"]
```

`get_energy_auctions(status='upcoming', board='physical', top=100)` :

- `status` یکی از : `upcoming` / `ready` ، `active` ، `surplus` / `mazad` ، `ended` یا معادل‌های فارسی `all` ؛ `آماده` نیز پذیرفته می‌شوند `پایان` و `مازاد` ، `فعال`.
- `board` یکی از : `physical` ، `power` ، `special` ، `green` ، `free` یا `all` . این انتخاب مستقیماً به دستهٔ رسمی منبع متصل است.
- `top` : سقف محلی ردیف برای هر ترکیب وضعیت/تابلو، عدد صحیح ۱ تا ۱۰۰۰. چون منبع گاهی پارامتر تعداد را نادیده می‌گیرد، پکیج سقف را بعد از دریافت نیز اعمال می‌کند.

خروجی شامل `AuctionID` , `Description` , `Volume` , `BasePrice` , `AuthorizedPriceMin` , `AuthorizedPriceMax` , `AuctionDateTime` , `Producer` , `PaymentTerms` , `DeliveryTerms` , `DeliveryPlace` , `VolumeUnit` , `PriceUnit` , `MinimumPurchase` , `MinimumPriceDiscoveryVolume` , `MaximumPurchase` , `TickSize` , `LotSize` , `Status` , `DiscoveredPrice` , `TradedVolume` , `SurplusTradedVolume` و `VWAP` است.

`get_energy_auction(auction_id, include_trades=True, include_instrument_history=False)` یک `dict[str, DataFrame]` برمی‌گرداند:

کلید	معنا
`auction`	مشخصات کامل همان حراج
`auction_trades`	فقط معاملات همان `AuctionID` ؛ در صورت `include_trades=True`
`instrument_history`	معاملات حراج‌های قبلی همان ابزار؛ فقط با `include_instrument_history=True`

`include_instrument_history` و `include_trades` فقط `bool` باشد. باید شناسهٔ عددی مثبت حراج `auction_id` تاریخچهٔ ابزار به معنای معاملهٔ همان عرضه نیست؛ endpoint می‌پذیرند. جداسازی دو جدول معامله ضروری است.

بازار برق و اوراق انرژی

```
# برق استاندارد، سبز و آزاد
power = att.list_power_instruments(market="all", top=100)

# سلف، گواهی ظرفیت، گواهی صرفه‌جویی، سپرده و آتی
securities = att.list_energy_securities(
    kind="all",
    active=True,
    top=100,
    enrich_futures=True,
)

# آمار روزانهٔ جریان‌های رسمی
overview = att.get_energy_market_overview(market="all")
```

`list_power_instruments(market='all', top=100)` در `market` مقادیر `standard`، `green`، `free` یا `all` را می‌پذیرد. `top` برای هر دسته است و سقف ۱ تا ۱۰۰۰ دارد.

`list_energy_securities(kind='all', active=True, top=100, enrich_futures=True)`:

- `kind` یکی از: `standard_salaf`، `capacity_certificate`، `energy_saving_certificate`، `deposit_certificate`، `future` یا `all`.
- `active`: در حالت `True` فقط دستهٔ جاری منبع خوانده می‌شود؛ در حالت `False` دسته‌های جاری و منقضی‌شدهٔ منتشرشده نیز خوانده می‌شوند. این پارامتر به معنی فیلتر تاریخ حدسی از نام نماد نیست.
- `top`: سقف ردیف هر دسته، ۱ تا ۱۰۰۰.
- `enrich_futures`: اگر `True` باشد برای هر آتی یک درخواست جزئیات زده می‌شود تا سررسید، اندازهٔ قرارداد و قیمت رسمی پر شود. تعداد واقعی درخواست‌ها را `DataFrame.attrs['request_count']` نشان می‌دهد.

ستون‌های مشترک این دو تابع شامل `Segment`، `InsCode`، `Symbol`، `Name`، `GregorianDate`، `JalaliDate`، `Close`، `Last`، `PreviousClose`، `Open`، `Low`، `High`، `TradeCount`، `Volume`، `Value`، `ValueUnit`، `ExpiryDate`، `ContractSize` و `SettlementPrice` است. در این feed، `ValueUnit='rial'` است.

`get_energy_market_overview(market='all')` مقادیر زیر را می‌پذیرد:

`market`	جریان رسمی
`free_power`	برق آزاد، flow 60
`power`	جمع بازار برق، flow 62
`standard_salaf`	سلف استاندارد، flow 63
`standard_power`	برق استاندارد، flow 64
`securities`	سایر اوراق قابل معامله، flow 65
`future`	آتی انرژی، flow 67
`deposit_certificate`	گواهی سپردهٔ انرژی، flow 68

market	جریان رسمی
green_power	برق سبز، flow 69

خروجی State ، Value ، Volume ، TradeCount ، تاریخ شمسی/میلادی و ValueUnit='rial' دارد. تاریخ کوتاه یا sentinel داخلی منبع به‌عنوان تاریخ معتبر گزارش نمی‌شود و مقدار آن خالی می‌ماند.

مشخصات آتی انرژی

```
contract = att.get_energy_future_contract("13758785995633713")
```

ده‌دهی دقیق را می‌پذیرد و یک ردیف برمی‌گرداند. مشخصات InsCode فقط get_energy_future_contract(ins_code) اندازه و واحد قرارداد، شروع، آخرین روز معامله، تحویل و آماده‌سازی، قیمت‌های بازار، نسبت‌های وجه، ISIN، شامل دارایی پایه تضمین اولیه/نگهداری/اضافی، کارمزد خریدوفروش و تسویهٔ نقدی/فیزیکی، جریمه، سقف موقعیت باز، ساعات معامله و قواعد تسویه است. مقدار صفر رسمی در فیلد تسویه با قیمت پایانی جایگزین نمی‌شود.

تابلوی مالی بورس کالا

```
certificates = att.get_commodity_market(kind="certificate")
salaf = att.get_commodity_market(kind="standard_salaf")
commodity_funds = att.get_commodity_market(kind="fund")
futures = att.get_commodity_market(kind="future")
all_rows = att.get_commodity_market(kind="all")
```

get_commodity_market(kind='all') یکی از certificate / deposit_certificate ، standard_salaf / salaf ، fund ، future یا all را می‌پذیرد. معادل‌های فارسی گواهی ، سلف ، صندوق و آتی نیز معتبرند.

برای گواهی، سلف و صندوق، خروجی این موارد را دارد:

- هویت: InsCode , ISIN , Symbol , Name , EnglishName , Segment
- بازار: TradeCount , Volume , Value , Close , Last , PreviousClose , Open , Low , High , Change , ChangePct
- حقیقی/حقوقی: تعداد و حجم خریدوفروش در ستون‌های Individual* و Legal*
- پنج سطح سفارش: BidVolume1..5 , BidCount1..5 , BidPrice1..5 و ستون‌های متناظر Ask*
- مشتقه: ContractUnit , ContractSize , OpenInterest , SettlementPrice , ExpiryJalali در صورت انتشار منبع

بورس کالا مقدار Value این تابلو را با واحد **هزار ریال** منتشر می‌کند؛ بنابراین ValueUnit='thousand_rial' و DataFrame.attrs['value_unit'] ثبت می‌شود. خالی بودن تابلوی آتی در یک روز، خروجی معتبر با schema کامل است و به خطای شبکه تبدیل نمی‌شود.

تابلوی جزئی قراردادهای اختیار کالایی در وب‌سایت رسمی IME از کانال realtime SignalR منتشر می‌شود و قرارداد HTTP پایدار و قابل بازپخش ندارد. نسخهٔ ۱.۶.۰ برای جلوگیری از دادهٔ ناقص یا حدسی، option chain کالایی نمی‌سازد؛ اما آمار تجمیعی رسمی بازار اختیار را از `get_commodity_market_activity()` ارائه می‌دهد.

بازار فیزیکی و نمای کل بورس کالا

```
daily = att.get_commodity_physical_history(
    start="1405-01-01",
    end="1405-06-01",
)

halls = att.get_commodity_physical_summary(
    start="1405-01-01",
    end="1405-06-01",
    hall="all",
)

markets = att.get_commodity_market_activity(
    start="1405-01-01",
    end="1405-06-01",
    market="all",
)
```

- `start` : تاریخ شروع الزامی؛ شمسی یا میلادی با قرارداد تاریخ عمومی پکیج.
- `end` : تاریخ پایان؛ اگر `None` شود می‌باشد تاریخ تقویم تهران در زمان فراخوانی استفاده.
- `hall` در : `get_commodity_physical_summary()` مقدار `all` ، شناسهٔ تالار یا نام دقیق تالار رسمی.
- `market` در : `get_commodity_market_activity()` مقدار `all` ، شناسهٔ بازار یا نام دقیق رسمی؛ شناسهٔ ۱ فیزیکی، ۲ آتی، ۳ اختیار معامله، ۴ مالی و ۵ گواهی سپرده است.

`get_commodity_physical_history(start, end=None)` سری روزانهٔ `Value` و `Volume` را می‌دهد.

`get_commodity_physical_summary(start, end=None, hall='all')` برای هر تالار `Volume` ، `OfferVolume` ، `Value` و `TradeToOfferRatio` را می‌دهد. نسبت معامله به عرضه فقط وقتی مخرج رسمی غیرصفر باشد محاسبه می‌شود.

`get_commodity_market_activity(start, end=None, market='all')` حجم و ارزش تجمیعی بازارهای رسمی را برمی‌گرداند؛ attrs است و بازهٔ ورودی در `ValueUnit='million_rial'` این تابع جزئیات قرارداد اختیار یا آتی نیست. در هر سه تابع تاریخی نیز ثبت می‌شود.

منحنی آتی، calendar spread و cash-and-carry

```
# contracts می‌تواند خروجی آتی انرژی/کالا یا DataFrame خود کاربر باشد.
curve = att.get_futures_curve(
    contracts,
    underlying="میعانات گازی",
    valuation_date="1405-07-01",
    price="settlement",
    spot_price=100_000,
)

spreads = att.get_calendar_spreads(curve)

carry = att.analyze_cash_and_carry(
    curve,
    annual_rate=0.30,
    storage_rate=0.02,
    convenience_yield=0.01,
)
```

`get_futures_curve(contracts, underlying=None, valuation_date=None, price='settlement', spot_price=None)`:

- `contracts`: `DataFrame` یا ستون سررسید records با `MaturityDate` یا `Expiry`، `ExpiryDate` و حداقل یکی از `SettlementPrice`، `Close`، `Last`.
- `underlying`: فیلتر اختیاری روی `Underlying`؛ اگر وجود نداشته باشد از نام ردیف استفاده می‌شود/`BaseAsset`.
- `valuation_date`: تاریخ ارزش‌گذاری؛ `None` در تهران و یعنی تاریخ تقویم attrs ثبت می‌شود.
- `price`: اولویت قیمت `settlement`، `close` یا `last`. `PriceSource` ستونی را که انتخاب شده نشان می‌دهد.
- `spot_price`: قیمت نقدی مثبت و صریح. بدون آن basis را از نام یا قیمت دیگری حدس spot محاسبه نمی‌شود؛ پکیج نمی‌زند.
- حفظ می‌شود؛ ترکیب قراردادهایی با واحدهای calendar spread اگر در ورودی موجود باشد، در منحنی و `ValueUnit` متفاوت با `InvalidParameterError` متوقف می‌شود.

خروجی بر اساس سررسید مرتب است و `BasisPct`، `AbsoluteBasis`، `DaysToExpiry` و `AnnualizedBasis` دارد. قرارداد بدون سررسید معتبر حذف می‌شود.

`get_calendar_spreads(curve)` سررسیدهای **مجاور** را جفت می‌کند و deterministic به‌صورت `Spread = FarPrice - NearPrice`، درصد spread، فاصلهٔ روز، spread سالانه و `Structure` با مقادیر `contango`، `backwardation` یا `flat` می‌سازد.

`analyze_cash_and_carry(curve, annual_rate, storage_rate=0.0, convenience_yield=0.0)` نرخ‌ها را به‌صورت effective اعشاری و سالانهٔ attrs می‌گیرد. فرمول ثبت‌شده در برابر است با:

```
FairValue = SpotPrice × (1 + annual_rate + storage_rate - convenience_yield) ** (DaysToExpiry / 365)
```

خروجی `MispricingPct` ، `Mispricing` ، `FairValue` و signal محاسباتی دارد. signal هزینهٔ معامله، وجه تضمین، مالیات، محدودیت تحویل و امکان اجرای واقعی آربیتراژ را مدل نمی‌کند و توصیهٔ معاملاتی نیست.

قرارداد منبع، تاریخ و کیفیت دادهٔ ۱.۶.۰

- `DataFrame.attrs` در `source` و در هر ردیف و `Source` منشأ را مشخص می‌کند.
- انتخاب‌شده کامل پردازش شده است، نه اینکه منبع همهٔ تاریخ بازار را نگهداری endpoint یعنی پاسخ `is_partial=False` می‌کند.
- `request_count` تعداد فراخوانی‌های enrichment همان تابع را نشان می‌دهد؛ آتی هزینهٔ درخواست جدا دارد.
- `ValueUnit` و گزارش‌های تاریخی `thousand_rial` IME تابلوی مالی ، `rial` را پیش از جمع‌زدن بررسی کنید: بورس انرژی `million_rial` IME هستند.
- تاریخ‌های حراج timestamp میلادی منبع‌اند و تاریخ شمسی متناظر بدون تغییر روز تقویمی ساخته می‌شود. تاریخ‌های گزارش فیزیکی از تاریخ شمسی رسمی منبع به میلادی تبدیل می‌شوند.
- دادهٔ حراج، snapshot تابلو و گزارش تجمیعی سه سطح پوشش متفاوت‌اند و نباید بدون کلید هویت/تاریخ و کنترل واحد با هم join شوند.

سایر APIهای بازار

این بخش قابلیت‌های قدیمی را در همان مرجع واحد نگه می‌دارد. APIهای legacy برای backward compatibility در دسترس‌اند و در بسیاری از خطاهای قدیمی `None` /پیام کنسول می‌دهند؛ APIهای جدید بیشتر از exceptionهای typed استفاده می‌کنند.

intraday

```
ticks_or_candles = att.get_intraday(
    "فملی",
    interval="1min",       # tick, 1min, 5min, 15min, 30min, 1h, 4h, 12h
    start="1403-05-01",    # حذف start/end برای امروز
    end="1403-05-03",
    progress=False,
)
```

خروجی candle معمولاً `Open, High, Low, Close, Volume` با index زمانی است. snapshot `tick` خام معاملات/قیمت را می‌دهد. interval بزرگ‌تر از دادهٔ پایه resample می‌شود؛ تعطیلی و وقفهٔ بازار را در محاسبه لحاظ کنید. نام‌های canonical بازه `1m` ، `60min` ، `4hour` ، `240m` ، alias‌های عددی/کوتاه مانند `12h` و `tick` ، `1min` ، `5min` ، `15min` ، `30min` ، `1h` ، `4h` ، `720` ، `12hour` و نیز `ticks` / `raw` پشتیبانی می‌شوند. این API رفتار legacy دارد: interval نامعتبر یا تاریخی که validator قدیمی نامعتبر تشخیص دهد را روی کنسول اعلام می‌کند و `None` برمی‌گرداند، نه اینکه عمداً `ValueError` قراردادشده‌ای بدهد.

اطلاعات نماد و سهامدار

```
detail = att.get_detail("فملی")
info = att.get_info("فملی")
stats = att.get_stats("فملی")
shareholders = att.get_shareholders("فملی", include_id=True)

# date تاریخ معامله است؛ خروجی تاریخ مؤثر روز معاملاتی بعد را هم نشان می‌دهد
dated = att.get_shareholders("فولاد", date="1405-06-18", include_id=True)

# start/end هستند TSETMC در این تابع تاریخ‌های مؤثر منتشرشدهٔ
history = att.get_shareholder_history(
    "فولاد",
    start="1404-01-01",
    end="1404-12-29",
    frequency="daily",     # daily | weekly | monthly
    max_requests=150,
    progress=False,
)
capital = att.get_capital_increase("فملی")
```

- `get_detail()` یک `DataFrame|None` با کلید–مقدار برابر index ستون `key`، `value` و ردیف `id` می‌دهد.
- `get_info()` و `get_stats()` یک DataFrame با کلید–مقدار برابر index `key` می‌دهند.
- `get_shareholders(date=None)` آخرین فهرست را می‌دهد. در حالت تاریخی `date`، **تاریخ معامله** است و دارایی پس از پایان آن معامله را با آن TSETMC در خروجی تاریخ روز معاملاتی بعدی است که `date/effective_date` هر دو مفهوم را صریح می‌کند؛ `trade_date` منتشر می‌کند.
- `get_shareholder_history(start, end)` ترکیب سهامداران عمدهٔ یک نماد را روی **تاریخ مؤثر** می‌سازد و `frequency='daily'` متوالی را استخراج می‌کند؛ برای نمونه ۲۳۹ جلسهٔ سال ۱۴۰۴ به حدود ۱۲۰ snapshot از هر پاسخ دو `monthly` و `weekly` به‌ترتیب آخرین جلسهٔ هفتهٔ معاملاتی ایران و آخرین جلسهٔ ماه درخواست نیاز دارد. فرکانس‌های شمسی را نگه می‌دارند.
- `max_requests` تاریخ گمشده از بودجه بیشتر شود، retry یک سقف سخت پیش از دریافت است؛ اگر پوشش جفتی یا شناسهٔ `include_id=True` تاریخ شمسی یا میلادی می‌پذیرند و `start/end` می‌دهد. `InvalidParameterError` تابع را اضافه می‌کند؛ این شناسه هویت پایدار و قابل اتصال بین روزها نیست **snapshot رکورد**.
- `get_capital_increase()` تاریخچهٔ تغییر تعداد سهام/سرمایه را می‌دهد.
- این توابع `ins_code` و keyword-only `asset_type` را نیز می‌پذیرند.

نمونهٔ ستون‌های تاریخ و تغییر در خروجی تاریخی:

```
share_holder_name  change_amount  trade_date  effective_date  change_quality
Holder_A              225000000    20260907        20260908  exact_normalized_name
```

`shareHolderID` و فقط با تطبیق دقیق نام نرمال‌شده محاسبه می‌شود، زیرا `change_amount` از اختلاف دو snapshot برمی‌گردد؛ کل دارایی سهامدار `NA` پاسخ تاریخی در دادهٔ واقعی صفر هستند. برای نام تازه‌وارد یا مبهم مقدار `changeAmount` نیز انجام نمی‌شود fuzzy matching تازه به‌عنوان خرید همان روز فرض نمی‌شود و.

قرارداد رسمی منبع: تاریخ دارایی یک روز معاملاتی بعد از معاملات است؛ تغییر دارایی می‌تواند ناشی از معامله یا سایر نقل‌وانتقالات باشد؛ اطلاعات ساعات بازار مربوط به پایان جلسهٔ قبل است؛ و فهرست عمومی ممکن است جزئیات یا دقت لازم برای استعلام رسمی را نداشته باشد. این موارد در `DataFrame.attrs` با کلیدهای `change_includes_non_trade_transfers`، `during_session_represents_previous_completed_session`، `is_authoritative_register` و `official_verification_required` ثبت می‌شوند.

فعالیت اخیر سهامداران عمده و timeline نماد

دو سطح دادهٔ سهامداران را از هم جدا نگه دارید. `get_shareholders()` برای یک نماد و یک تاریخ دلخواه در سابقهٔ معاملاتی است؛ APIهای زیر تابلوی سراسری «تغییرات سهامداران عمده» را می‌خوانند که فقط یک پنجرهٔ گردان با حداکثر پنج تاریخ منتشرشده دارد:

```
# Holdings in the five published recent-change snapshots
snapshots = att.get_major_shareholder_snapshots(
    days=5,
    enrich_identity=True,   # exact InsCode identity join
    progress=False,
)

# Latest published change versus its previous snapshot
changes = att.get_major_shareholder_changes(
    days=1,
    direction="both",       # both, increase, decrease, unchanged
    progress=False,
)

# Exact symbol or holder filter
foolad = att.get_major_shareholder_changes(
    "1405-06-18", ins_code="46348559193224090"
)
holder = att.get_active_shareholders(holder="Exact Holder Name", days=5)

# Rank each holder/instrument pair
ranking = att.rank_shareholder_accumulation(
    days=5,
    direction="accumulation",  # both | accumulation | distribution
    metric="percent",          # percent | shares
    top=20,
    progress=False,
)

# Latest-snapshot bipartite edge list
network = att.get_shareholder_network(
    min_holdings=1_000_000,
    progress=False,
)

# Per-instrument major-holder concentration
concentration = att.get_ownership_concentration(
    ins_code="46348559193224090", top_n=5, progress=False
)

# TSETMC-native symbol timeline
events = att.get_symbol_events(
    ins_code="778253364357513",
    start="1400-01-01",
    kinds=("messages", "capital_changes", "price_adjustments"),
    limit=100,
    progress=False,
)
```

خروجی نمایندهٔ `get_major_shareholder_changes()` :

```
HolderName  InsCode  Symbol  PreviousGregorianDate  GregorianDate  PreviousHoldings  Holdings
ChangeShares  Direction
115000000         100000000      09-09-2026             08-09-2026  وبملت      ... سهامدار الف
15000000      increase
```

خروجی `get_major_shareholder_snapshots()` تمام مقدارهای منتشرشده را با `HolderRecord, HolderName, InsCode,` `Symbol, InstrumentName, GregorianDate, JalaliDate, Holdings, Source, FetchedAt` نگه می‌دارد. خروجی changes مقدارهای `ChangeShares` , `Holdings` , `PreviousHoldings` و `Direction` را اضافه می‌کند.

`get_active_shareholders()` برای هر جفت ردیف سهامدار/ابزار، مقدار اولیه و آخرین دارایی، تغییر خالص، افزایش و کاهش ناخالص و تعداد روزهای فعال را می‌دهد.

بر اساس قدرمطلق درصد `metric='percent'` همین جفت‌های سهامدار–نماد را با `rank_shareholder_accumulation()` بر اساس قدرمطلق تعداد سهم رتبه‌بندی می‌کند. تعداد سهم چند `metric='shares'` تغییر نسبت به دارایی ابتدای پنجره، و با .شرکت با هم جمع نمی‌شود؛ جمع‌زدن سهم نمادهایی با سرمایه و قیمت متفاوت یک معیار مالی قابل‌اتکا نمی‌سازد

`direction` می‌تواند `both` ، `accumulation` یا `distribution` .باشد

دو سر یال‌اند و `InstrumentNode` و `HolderNode` :دوبخشی می‌دهد edge list یک `get_shareholder_network()` درجهٔ هر گره را در همان `InstrumentHolderCount` و `HolderInstrumentCount` وزن یال است. ستون‌های `Holdings` فقط آخرین تاریخ یا یک تاریخ موجود در پنجرهٔ پنج‌تایی را می‌پذیرد؛ API نشان می‌دهند. این snapshot `min_holdings` .یال‌های کوچک‌تر را حذف می‌کند

برخلاف دو تحلیل قبلی از فهرست سهامداران عمدهٔ یک **نماد** استفاده می‌کند. خروجی `get_ownership_concentration()` بر پایهٔ درصد کل شرکت و `MajorHolderHHI` ،Top-N و Top-1/3/5/10 شامل درصد افشاشده، سهم پس از نرمال‌کردن فقط بخش افشاشده است. باقیماندهٔ گزارش‌نشده به‌صورت یک سهامدار فرض `NormalizedDisclosedHHI` کامل است و این شاخص‌ها تمرکز فهرست عمومی سهامداران عمده‌اند، نه HHI کران پایین `MajorHolderHHI` نمی‌شود؛ بنابراین .دفتر رسمی کامل مالکیت

> **محدودیت ضروری منبع:** این feed فهرست کامل همهٔ سهامداران عمده نیست؛ فقط مواردی را نشان می‌دهد که در تابلوی تغییرات اخیر TSETMC حاضرند. حداکثر پنج تاریخ منتشرشده در دسترس است و backfill سمت سرور وجود ندارد. مقدار `available_dates` را از `DataFrame.attrs` بخوانید و «امروز» را فرض نکنید. تغییر دارایی می‌تواند از معامله یا سایر نقل‌وانتقال‌ها ناشی شود. در ساعات معامله، دارایی منتشرشده مربوط به جلسهٔ تکمیل‌شدهٔ قبلی است و تغییرات همان روز پس از پردازش پایان جلسه منتشر می‌شود. این اطلاعات برای اتکای حقوقی کافی نیست و باید از مرجع ذی‌صلاح استعلام شود.

چون endpoint شناسهٔ پایدار سهامدار نمی‌دهد، `HolderRecord` فقط شمارهٔ ردیف همان پاسخ است و هویت پایدار بین دو فراخوانی محسوب نمی‌شود. نام‌های مشابه، به‌ویژه «شخص حقیقی»، با هم ادغام نمی‌شوند. `enrich_identity=True` فقط `Symbol` و `InstrumentName` را با اتصال دقیق `InsCode` از snapshot جاری تکمیل می‌کند؛ تعداد ابزارهای بدون تطبیق در `unmatched_identity_count` ثبت می‌شود.

`get_symbol_events()` را یکجا می‌آورد: پیام ناظر، تغییر تعداد سهام شرکت و تعدیل قیمت. TSETMC فقط داده‌های بومی `attrs['codal_included'] ==` نیستند؛ timeline هیئت‌مدیره و صورت‌های مالی دادهٔ افشای کدال‌اند و در این DPS، مجمع تأییدشده تلقی DPS این مرز را ماشین‌خوان نگه می‌دارند. رخداد تعدیل قیمت نیز `excluded_disclosure_kinds` و `False`

.نمی‌شود

معرفی شرکت؛ API قدیمیِ unsupported

قدیمی مانده‌اند. معرفی ناشر دادهٔ import/signature فقط برای حفظ `stock_introduction()` و `get_introduction()` این پکیج قرار دارد؛ تابع **همیشه پیش از** source boundary کدال است و خارج از **resolver** یا **شبکه** خطای زیر را می‌دهد:

```
try:
    att.get_introduction("فملی")
except att.UnsupportedDataSourceError as exc:
    print(exc)
```

```
UnsupportedDataSourceError: get_introduction/stock_introduction requires Codal data, which is outside
algotik-tse's TSETMC market-data source boundary
```

برای اطلاعات TSETMC از `get_info()` و `get_detail()` استفاده کنید. helper افشای ناشر یا history آن در این پکیج وجود ندارد.

فهرست نمادها و ابزارها

```
symbols = att.get_symbols(
    bourse=True, farabourse=True, payeh=True,
    haghe_taqadom=False, sandogh=False, bonds=False, options=False,
    mortgage=False, commodity=False, energy=False,
    payeh_color=None, output="dataframe", progress=False,
)

etfs = att.list_etfs(progress=False)
bonds = att.list_bonds(progress=False)
debt = att.list_debt_instruments(["erad", "gam"], active_only=True, progress=False)
funds = att.list_funds(fund_type="fixed_income", progress=False)
print(funds.loc[
    funds["fund_name"].eq("خاتم ایساتیس پویا"),
    ["record_date", "pct_bond", "pct_deposit", "pct_cash"],
])
print(funds.attrs["pct_cash_source"], funds.attrs["pct_cash_missing"])
listed_funds = att.list_listed_funds(progress=False)
options = att.list_options(underlying="خودرو", progress=False)
indices = att.list_indices(progress=False)
members = att.get_index_companies("شاخص صنعت بانکها", progress=False)
industry_indices = att.list_industry_indices(progress=False)
industry_snapshot = att.get_industry_snapshot("بانک", progress=False)
```

.بازار/نوع ابزار را نگه می‌دارد metadata `dataframe` فقط نام نمادها را می‌دهد؛ `get_symbols(output="list")`
ها را صریح تنظیم asset-type flag، اشتباه universe است. برای جلوگیری از `قرمز` ، `نارنجی` ، `زرد` یکی از `payeh_color`
.کنید

`list_bonds()` را می‌دهد NAV/discount برای سازگاری قبلی فقط صندوق‌های بورسی عمومی نوع ۳۰۵ و `list_etfs()` و فیلتر `DebtType` ستون `list_debt_instruments()` اوراق و سررسید را با امضای قدیمی فهرست می‌کند؛ metadata مرجع سایر اوراق در فصل درآمد ثابت آمده است YTM محاسباتی اخزا و analytics. مستقیم اخزا/اراد/گام را اضافه می‌کند. به‌دلیل تکرار نادرست فیلد خام `pct_cash` می‌آید اما TSETMC صندوق‌هاست؛ ترکیب دارایی اصلی از registry `list_funds()` یا جمع باقیماندهٔ `pct_deposit` جایگزین می‌شود. مقدار وجه نقد هرگز از `reg_no` و با اتصال دقیق FIPIRAN از TSETMC، دقیق و InsCode/ISIN ابزارهای واقعاً قابل معاملهٔ نوع ۳۰۵ و ۳۸۰ را با `list_listed_funds()` اجزا ساخته نمی‌شود. طبقه‌بندی قابل فیلتر می‌دهد.

شاخص‌ها

```
index_history = att.get_history("شاخص کل", limit=100, progress=False)
industry_history = att.get_industry_history("بانک", limit=100, progress=False)
members = att.get_industry_members("بانک", progress=False)
```

برای شاخص‌های عمومی همچنان `get_history()` را به‌کار ببرید. برای شاخص صنعت، APIهای اختصاصی فصل شاخص‌ها و تحلیل صنایع عضویت رسمی، تاریخچهٔ اعضا، snapshot و intraday را یکدست ارائه می‌کنند. schema شاخص با سهام فرق دارد و volume جعلی ساخته نمی‌شود. `list_indices()` و `get_index_companies()` برای سازگاری با کد قدیمی حفظ شده‌اند.

طلا، ارز، نقره و سکه از TGJU

با همان رفتار قبلی حفظ `currency_coin()` و `get_currency()` است. provider نام عمومی این `get_tgju_history()` شده‌اند و اکنون تمام دارایی‌های کاتالوگ را هم می‌پذیرند:

```
fx = att.get_currency(
    "dollar", start="1403-01-01",
    output_type="standard", date_format="jalali", progress=False,
)

coins = att.get_currency(["seke", "nim-seke"], limit=10, progress=False)
gold = att.get_tgju_history(
    ["gold-18k", "طلای ۲۴ عیار", "gold-mesghal"],
    limit=30,
    date_format="both",
    progress=False,
)
```

پیش از انتخاب نام، کاتالوگ را ببینید:

```
catalog = att.list_tgju_assets()
gold_assets = att.list_tgju_assets(category=["gold", "silver"])
print(gold_assets[["Name", "PersianName", "Slug", "Unit"]])
```

گروه‌های کاتالوگ:

دسته	تعداد	نمونه‌ها	واحد
`currency`	۳۶	دلار، یورو، فرانک، ۱۰۰ ین، ارزهای منطقه	ریال برای هر واحد؛ ین ژاپن برای ۱۰۰ واحد
`official_rate`	۲۴	سری‌های منتشرشدهٔ سنا، نیما و بانکی	ریال برای هر واحد
`gold`	۱۱	۱۸ و ۲۴ عیار، مثقال، دست‌دوم، آب‌شده، عیار ۷۴۰	ریال بر گرم یا مثقال
`silver`	۲	نقرهٔ ۹۲۵ و ۹۹۹	ریال بر گرم
`coin` / `coin_bubble`	۵ + ۵	امامی، بهار آزادی، نیم، ربع، گرمی و حباب هرکدام	ریال بر سکه
`global_metal`	۴	انس طلا، نقره، پلاتین و پالادیوم	دلار بر انس تروا

نام‌های پرکاربرد:

```
gold-18k, gold-24k, gold-mesghal, used-gold,
melted-gold-cash, melted-gold-wholesale, silver-999, silver-925,
gold-ounce, silver-ounce, platinum-ounce, palladium-ounce,
dollar, euro, yuan, dirham, pound, swiss-franc,
seke, seke-bahar-azadi, nim-seke, rob-seke, seke-gerami,
seke-bubble, nim-seke-bubble, rob-seke-bubble
```

نام canonical، نام فارسی و slug رسمی هر سه پذیرفته می‌شوند؛ بنابراین `طلای ۱۸ عیار` ، `gold-18k` و `geram18` یک سری هستند. رقم فارسی/عربی و ی/ک عربی نرمال می‌شود. برای نام نامعتبر `InvalidParameterError` ، برای HTTP ناموفق `ConnectionError` و برای payload ناسازگار `DataParsingError` صادر می‌شود.

خروجی standard ستون‌های `Open, High, Low, Close` دارد و در حالت چندداراییِ یک DataFrame با MultiIndex ستونی برمی‌گردد. در حالت تک‌دارایی، `attrs` شامل `unit` , `category` , `slug` , `persian_name` , `asset` , `source` است.
`save_path` پوشهٔ مقصد CSV است. «قیمت» و واحد را از `attrs['unit']` / `Unit` بخوانید؛ مخصوصاً انس جهانی ریالی نیست و ین ژاپن به ازای ۱۰۰ ین منتشر می‌شود.

تنظیمات و خطاها

تنظیمات singleton:

```
from algotik_tse import settings

settings.ssl_verify = True           # پیش‌فرض و توصیه‌شده
settings.timeout = 10                # ثانیه
settings.max_retries = 3
settings.retry_backoff_factor = 0.3
settings.rate_limit_delay = 0.3      # فاصلهٔ حداقل شروع درخواست‌ها

settings.market_snapshot_freshness_seconds = 120.0
settings.market_clock_skew_tolerance_seconds = 5.0
settings.client_volume_consistency_tolerance = 0.05
settings.industry_membership_cache_ttl = 3600.0
settings.order_book_max_requests = 250
settings.trade_max_requests = 250
```

:خراب CA فقط برای محیط کنترل‌شده با opt-out .است `True` پیش‌فرض TLS verification

```
from algotik_tse.http_client import safe_get

response = safe_get("https://cdn.tsetmc.com/...", verify=False)
```

این opt-out را سراسری نکنید. URLهای TSETMC همگی HTTPS هستند.

قرارداد دقیق `safe_get(url, **kwargs)` :

- نامجاز با URL/Codal path های مجاز باشد؛provider و داخل HTTP(S) باید `url: str` .رد می‌شود I/O و session، rate-limit پیش از `UnsupportedDataSourceError`
- فقط `verify=settings.ssl_verify` و `timeout=settings.timeout` , `headers=settings.headers` هایdefault .نداده باشد اعمال می‌شوند caller override وقتی
- هستند. نوع نادرست اولی/دومی wrapper پارامترهای خود `max_redirects: int=5` و `allow_redirects: bool=True` .است `ValueError`، `max_redirects<0` و `TypeError`
- درخواست زیرین همیشه `allow_redirects=False` دارد. redirect فقط `scheme,host,effective-`) same-origin `port`) و hop-by-hop است؛ `UnsupportedDataSourceError` cross-origin و loop/عبور از سقف `requests.exceptions.TooManyRedirects` می‌دهد.
- redirect در caller هایheader دوباره فرستاده نمی‌شود؛ redirect فقط روی درخواست اول اعمال می‌شود و روی `params` .حفظ می‌شوند same-origin
- خطاهای transport خود `requests` بعد از retry propagate می‌شوند. `safe_get` به‌تنهایی روی status 4xx/5xx `raise_for_status()` نمی‌کند؛ API مصرف‌کننده باید پیش از parse آن را به خطای typed خود تبدیل کند.

سلسله‌مراتب خطا:

```
AlgotikTSEError
├── AmbiguousSymbolError
├── ConnectionError
├── DataParsingError
├── InvalidParameterError
├── StockNotFoundError
└── UnsupportedDataSourceError
```

legacy سطح بالای پکیج نیست. توابع export برای سازگاری داخلی وجود دارد ولی exceptions در ماژول `RateLimitError` و پیام کنسول بدهند؛ قرارداد هر تابع را بررسی کنید `None`، exception ممکن است به‌جای.

الگوی امن:

```
try:
    df = att.get_price_adjustments(ins_code="35425587644337450")
except att.InvalidParameterError as exc:
    print("bad input", exc)
except att.AmbiguousSymbolError as exc:
    print("pass ins_code", exc)
except att.ConnectionError as exc:
    print("provider unavailable", exc)
except att.DataParsingError as exc:
    print("provider schema changed", exc)
except att.UnsupportedDataSourceError as exc:
    print("outside supported sources", exc)
```

مرجع تفصیلی همهٔ توابع

در این بخش همهٔ exportهای عمومی پوشش داده شده‌اند. هر ردیف API در کنار همان تابع، ورودی‌ها و گزینه‌های اختصاصی، خروجی، خطاهای اصلی و مثال را توضیح می‌دهد. جدول‌های پارامتر مشترک فقط تعریف اصطلاحات را یکسان نگه می‌دارند؛ signature دقیق هر تابع نیز بلافاصله پیش از جدول همان گروه آمده است.

نمای کلی exportها و نام‌های قدیمی

جدول زیر inventory کامل exportهای `algotik_tse.__all__` است. جزئیات خروجی در بخش موضوعی مربوط آمده است.

هویت، تنظیمات و خطا

Export	کاربرد
`settings`	تنظیمات singleton شبکه/بازار
`InstrumentRef`	هویت immutable ابزار

Export	کاربرد
`normalize_instrument_text`, `validate_ins_code`, `resolve_instrument`	نرمال‌سازی و حل دقیق هویت
`AlgotikTSEError`, `AmbiguousSymbolError`, `ConnectionError`, `DataParsingError`, `InvalidParameterError`, `StockNotFoundError`, `UnsupportedDataSourceError`	خطاهای عمومی

قیمت، trades، client type و اطلاعات نماد

Export canonical	Alias/legacy عمومی
`get_history`	`stock`
`get_client_type`	`stock_RI`, `stock_RL`
`get_capital_increase`	`stock_capital_increase`
`get_intraday`	`stock_intraday`
`get_trades`, `get_live_trades`	—
`get_detail`	`stockdetail`
`get_info`	`stock_information`
`get_stats`	`stock_statistics`
`get_introduction` (همیشه unsupported)	`stock_introduction` (همیشه unsupported)
`get_shareholders`	`shareholders`
`get_shareholder_history`, `get_major_shareholder_snapshots`, `get_major_shareholder_changes`, `get_active_shareholders`	—
`rank_shareholder_accumulation`, `get_shareholder_network`, `get_ownership_concentration`	—
`get_symbols`	`stocklist`
`get_currency`	`currency_coin`

پارامترهای قدیمی نیز پذیرفته می‌شوند: `stock` → `symbol`، `values` → `limit`، `tse_format` → `raw`، `dropna` ← `multi_stock_drop` / `multi_currencies_drop` و `output_type="complete"` → `"full"` در مسیرهای مربوط. برای کد جدید نام canonical را به‌کار ببرید.

live، سفارش، watcher و history محلی

Exportها
`get_market_snapshot`, `get_market_client_type`, `get_order_book`, `get_live_market`, `get_live_symbol`
`get_order_book_history`, `get_orderbook_history`, `get_queue`, `get_queue_history`

Exportها
`MarketEvent`، `MarketWatcher`، `watch_market`
`get_market_messages`، `get_instrument_state_changes`، `get_market_overview`، `get_market_breadth`، `get_sector_flow`
`MARKET_HISTORY_SCHEMA_VERSION`، `MARKET_HISTORY_APPLICATION_ID`، `check_market_history`
`save_market_snapshot`، `load_market_snapshots`، `get_live_market_history`
`get_market_overview_history`، `get_market_snapshot_summary_history`، `get_market_breadth_history`، `get_sector_flow_history`
`record_market_event`، `get_market_event_history`، `archive_market_records`
`get_market_messages_history`، `get_instrument_state_changes_history`
`get_trading_calendar`، `get_market_activity`، `get_market_value_history`، `get_index_impact`، `get_market_trades`
`get_theoretical_opening_price`، `get_preopen_imbalance`
`compare_symbols`، `get_liquidity_metrics`، `get_market_regime`، `get_market_map`، `plot_market_map`

سه wrapper صفرآرگومان legacy نیز عمومی‌اند: `market_data()`، `market_client_type()`، `market_watch()`. wrapper deprecated `market_data()` است؛ برای کد جدید `get_market_snapshot()` یا `get_live_market()` را انتخاب کنید.

fundamentals، تعدیل قیمت و ابزارها

Exportها
`get_market_fundamentals`، `get_market_fundamentals_history`
`get_price_adjustments`، `get_latest_price_adjustment`
`get_instrument_master`، `get_instrument_changes`
`list_options`، `get_options_chain`، `list_etfs`، `list_bonds`، `list_debt_instruments`، `list_funds`، `list_listed_funds`
`list_indices`، `get_index_companies`
`list_industry_indices`، `get_industry_members`، `get_industry_snapshot`، `get_industry_membership_events`
`get_industry_history`، `get_industry_members_history`، `get_industry_membership_overlap`، `get_industry_membership_churn`، `get_industry_intraday`، `rank_industries`

TGJU: ارز و فلزات گران‌بها

Exportها
`list_tgju_assets`, `get_tgju_history`, `get_currency`, `currency_coin`

درآمد ثابت

Exportها
`IRAN_TREASURY_FACE_VALUE`, `YieldCurve`
`parse_treasury_maturity`, `day_count_fraction`, `treasury_yield`
`bond_price`, `yield_to_maturity`, `bond_analytics`, `build_yield_curve`
`get_ifb_yield_table`, `get_debt_yields`, `get_treasury_yields`
`get_treasury_yield_history`, `get_treasury_yields_history`
`get_yield_curve`, `get_yield_curve_history`

اختیار معامله

Exportها
`OPTION_SNAPSHOT_SCHEMA_VERSION`
`black_scholes_price`, `black_scholes_greeks`, `option_price_bounds`, `implied_volatility`
`get_option_market`, `analyze_option_chain`, `option_put_call_ratios`
`get_option_history`, `save_option_snapshot`, `load_option_snapshots`

بورس انرژی و بورس کالا

Exportها
`get_energy_auctions`, `get_energy_auction`, `get_energy_market_overview`
`list_power_instruments`, `list_energy_securities`, `get_energy_future_contract`
`get_commodity_market`, `get_commodity_physical_history`, `get_commodity_physical_summary`, `get_commodity_market_activity`
`get_futures_curve`, `get_calendar_spreads`, `analyze_cash_and_carry`

signatureهای پرکاربرد

```
get_live_market(symbol=None, *, strict=False)
get_live_symbol(symbol=None, *, ins_code=None, fallback="none")
get_order_book(symbol=None, *, selector_strict=False)
get_queue(symbol=None, side="both", strict=True, *, selector_strict=False)
get_trades(symbol=None, *, ins_code=None, start=None, end=None,
           include_canceled=False, max_requests=None, raw=False, progress=True)
get_market_messages(flow=0, top=20, since_id=None, *, archive_to=None)
get_instrument_state_changes(top=20, since_id=None, *, archive_to=None)
get_market_overview(flow=0, *, archive_to=None)
compare_symbols(symbols, start=None, end=None, benchmark=None, align='inner', annualization=240,
min_observations=20, include_client_type=True, progress=True, history_data=None, client_data=None,
strict=False)
get_liquidity_metrics(symbols=None, start=None, end=None, window=20, annualization=240,
include_live=True, progress=True, history_data=None, live_data=None, strict=False)
get_market_regime(benchmark='شاخص کل', start=None, end=None, lookback=60, liquidity_window=20,
annualization=240, weights=None, progress=True, max_requests=30, live_data=None,
benchmark_history=None, activity_history=None)
get_market_map(group_by='symbol', size='value', color='return', top=100, flow=None,
instrument_types=None, data=None)
plot_market_map(map_data=None, output_path=None, show=False, title=None)
get_energy_auctions(status='upcoming', board='physical', top=100)
get_energy_auction(auction_id, include_trades=True, include_instrument_history=False)
get_energy_market_overview(market='all')
list_power_instruments(market='all', top=100)
list_energy_securities(kind='all', active=True, top=100, enrich_futures=True)
get_energy_future_contract(ins_code)
get_commodity_market(kind='all')
get_commodity_physical_history(start, end=None)
get_commodity_physical_summary(start, end=None, hall='all')
get_commodity_market_activity(start, end=None, market='all')
get_futures_curve(contracts, underlying=None, valuation_date=None,
                  price='settlement', spot_price=None)
get_calendar_spreads(curve)
analyze_cash_and_carry(curve, annual_rate, storage_rate=0.0,
                       convenience_yield=0.0)
```

پارامترهایی که با `_` شروع می‌شوند seam داخلی تست‌اند و API کاربر محسوب نمی‌شوند، حتی اگر در `inspect.signature` دیده شوند.

روش خواندن مرجع تفصیلی

هر signature زیر عین خروجی پایدارشدهٔ `inspect.signature` است؛ فقط آدرس حافظهٔ `safe_get` به خود نام `safe_get` نرمال شده است. نوع‌هایی که در signature قدیمی annotation ندارند در جدول پارامترها مشخص شده‌اند. مقدار `None` معمولاً یعنی «فیلتر/override اعمال نشود»، نه رشتهٔ `"None"`. پارامترهای مشترک فقط یک‌بار در واژه‌نامهٔ زیر توضیح داده می‌شوند و هر مدخل API علاوه بر آن، override و constraint خاص خود را می‌گوید. «خطاها» خطاهای اصلی قرارداد است، نه فهرست همهٔ خطاهای ممکن Python/pandas.

پارامترهای مشترک هویت، تاریخچه و خروجی

نام	معمول default و type	معنا و constraint
`symbol` / `symbols`	`str | Iterable[str] | None` = `''` API؛ بسته به `None` یا	نماد فارسی، `InsCode` یا مجموعهٔ آن‌ها؛ برای هویت مبهم `ins_code` بدهید. `symbols=None` یعنی کل universe.
`ins_code` / `inscode`	`str | int | None = None`	شناسهٔ دقیق ۱ تا ۲۰ رقم ASCII؛ با selector متناقض مجاز نیست. `inscode` فقط spelling تاریخی reader وضعیت است.
`asset_type`	`str = 'auto'`	حل هویت؛ hint `auto` نوع را از provider تعیین می‌کند.
`start` , `end` , `date`	`str | date | datetime | None`	مرز شامل ابتدا/انتها؛ ISO شمسی یا میلادی در APIهای بازار. `date` در `get_shareholders` یعنی تاریخ دلخواه نماد؛ در API فعالیت سهامداران باید یکی از حداکثر پنج تاریخ منتشرشده باشد.
`limit`	readerها یا `int = 0` `1000`	`0` در historyهای provider یعنی بدون محدودیت بعد از فیلتر؛ در SQLite حداکثر صفحه. منفی نامعتبر است.
`offset`	`int = 0`	offset SQL بعد از فیلترها؛ نامنفی.
`include_today`	`bool = False`	ردیف زندهٔ امروز؛ ممکن است در ساعات بازار آخرین opt-in زنده دارد provenance مشاهدهٔ همین لحظه باشد و.
`raw`	`bool = False`	ممکن است order-book ؛ درprovider نزدیک schema delta/raw-mixed باشد.
`output_type`	`str = 'standard'`	خروجی؛ مقادیر دقیق تابعی‌اند schema / `standard`). `complete` alias قدیمی `full` یا `long` / `wide` است. `full`.
`date_format`	`str = 'jalali'`	شکل index/ستون تاریخ؛ , `gregorian` , `jalali` در مسیرهای پشتیبانی‌شده. `both`
`ascending`	`bool = True`	ترتیب زمانی خروجی بعد از فیلتر.
`progress`	`bool = True`	فقط پیام پیشرفت؛ در داده و schema اثر ندارد.
در فعالیت سهامداران `days`	`int = 5`	تعداد تاریخ‌های انتهایی از پنجرهٔ گردان provider؛ فقط ۱ تا ۵. تاریخ قدیمی‌تر backfill نمی‌شود.
`holder`	`str | None = None`	فیلتر exact پس از نرمال‌سازی فارسی/عربی؛ شناسهٔ پایدار سهامدار در feed سراسری موجود نیست.
`enrich_identity`	`bool = True`	تکمیل `Symbol/InstrumentName` فقط با اتصال دقیق `InsCode` به MarketWatch؛ شکست enrichment دادهٔ مالکیت را حذف نمی‌کند و در attrs ثبت می‌شود.
`save_to_file`	`bool = False`	ذخیرهٔ CSV opt-in.
`save_path`	`str | Path | None`	**پوشهٔ** مقصد، نه نام فایل؛ بدون `save_to_file=True` نوشته نمی‌شود.

نام	معمول default و type	معنا و constraint
`dropna`	`bool = True`	حذف ردیف/ستون کاملاً تهی طبق قرارداد همان API؛ ردیف partial معنادار order-book حفظ می‌شود.
`return_type`	`str | None`	قیمت/ history سازگاری برای انتخاب نوع خروجی در alias را ترجیح دهید `output_type` ارز؛ برای کد جدید.
`max_requests`	`int | None`	سقف سخت fan-out قبل/حین I/O؛ معنای دقیق شمارش در مدخل تابع آمده است.
`strict` / `selector_strict`	`bool`	را فعال می‌کند؛ match خطای داده/فیلتر بدون strict مبهم/گم‌شده را scalar انتخاب `selector_strict` خطا می‌کند.
`allow_stale` / `include_stale`	`bool`	اجازهٔ نگه‌داشتن snapshot/اوراق stale؛ stale بودن همچنان در ستون/attrs گزارش می‌شود.
`archive_to` , `record_to` , `snapshot_path` , `path`	`str | Path | None`	فایل SQLite/JSON صریح؛ نوشتن فقط با opt-in. `path` در reader/writer اجباری است.
`kwargs`	های سازگاریkeyword	فقط aliasهای مستند مانند `stock` , `values` , `stocklist` ; و نام‌های انگلیسی `tse_format` keyword ناشناخته قرارداد عمومی نیست.
`_request` , `_snapshot` , `_client_type` , `_clock` , `_wait` , `_random` , `_recorded_at` , `_live_fetch` , `_request_budget_state`	داخلی seam	فقط تزریق deterministic در تست؛ برای مصرف عادی استفاده نشود و BC عمومی برای آن تضمین نمی‌شود.

پارامترهای مشترک watcher، live و SQLite

نام	type/default	معنا و constraint
`flow`	`int | None = 0`	بازار/جریان provider؛ `None` در analytics یعنی بدون فیلتر.
`sector`	`str | Iterable | None`	فیلتر `SectorCode` .
`instrument_types`	`Iterable[int] | None`	`300,303,309` . ابزار؛ پیش‌فرض تحلیلی سهام universe
`traded_only`	`bool = False`	فقط ابزار دارای معامله را در denominator نگه می‌دارد.
`include_base_market`	`bool = True`	نوع 309 بازار پایه را در universe پیش‌فرض نگه می‌دارد.
`side`	`str = 'both'`	برای صف `both` یا `sell` , `buy` .
`complete_only`	`bool = False`	فقط snapshotهای پنج‌سطح کامل.
`top`	`int = 20`	تعداد پیام/وضعیت در درخواست؛ مثبت و bounded.
در تغییرات سهامدار `direction`	`str='both'`	`both` فقط تغییرهای غیرصفر؛ یا `increase` , `decrease` , `unchanged` .
نماد timeline در `kinds`	`str | Iterable[str]`	یک یا چند مورد از `messages` , `capital_changes` , `price_adjustments` ؛ هیچ kind کدال پذیرفته نمی‌شود.

نام	type/default	معنا و constraint
`since_id`	`int | str | None`	فیلتر client-side رکوردهای پس از شناسه؛ cursor provider نیست.
`source`	`str = 'tsetmc'`	می‌تواند fixed-income ؛ درarchive هویت/provenance باشد hybrid یا حالت مستند `tsetmc`.
`session_id`	`str | None`	شناسهٔ session watcher برای نوشتن/فیلتر replay.
`kind`	`str | None`	kind یا `initial/delta/heartbeat/resync` هایevent پشتیبانی‌شده archive.
`recorded_at` , `as_of`	`None` یا timestamp-like	زمان مشاهده؛ `None` یعنی ساعت تهران/UTC داخلی معتبر تابع.
`max_records` , `record_max_records`	`int = 10000`	بر اساس تعداد؛ مثبت retention.
`retention_seconds` , `record_retention_seconds`	`float | None`	یعنی غیرفعال `None` زمانی؛ retention.
`interval`	`str` در intraday؛ `float=1.0` در watcher	یا (`tick/1min/5min/15min/30min/1h`) candle interval بر حسب ثانیه فاصلهٔ polling.
`max_updates`	`int | None`	سقف eventهای iterator؛ `None` یعنی تا stop.
`notifications`	پیش‌فرض، iterable `('messages','state')`	فقط این دو notification پشتیبانی می‌شوند؛ Codal وجود ندارد.
`error_policy` , `callback_error_policy` , `storage_error_policy`	`str`	`error_policy∈{retry,raise,stop}` ، enum های دقیق: و `callback_error_policy∈{raise,ignore,stop}` ؛ مقدار نامعتبر `storage_error_policy∈{raise,ignore}` خطا است I/O پیش از.
`max_backoff` , `jitter` , `request_timeout`	`float | None`	هر درخواست؛ نامنفی/مثبت طبق timeout و jitter ،سقف backoff کلاس.
`max_consecutive_retries` , `max_retries`	`int | None`	سقف retry؛ `max_retries` نام قدیمی سازگار است.
`include_initial` , `emit_heartbeats` , `copy_snapshot` , `record_heartbeats`	`bool`	کنترل emission/copy/persistence eventها.
`notification_top` , `max_seen_notifications` , `checkpoint_interval`	`int`	`notification_top` بین ۱ و ۱۰۰۰؛ دو مقدار دیگر مثبت `record_max_records` بین ۱ و ۱۰۰۰۰ است.

پارامترهای تحلیل مدیریتی

نام	type/default	معنا و constraint
`benchmark`	`str | None`	شاخص یا نماد مبنا؛ در مقایسه اختیاری و در رژیم بازار پیش‌فرض شاخص کل است.

نام	type/default	معنا و constraint
`align`	`str='inner'`	برای اتحاد تاریخ‌ها `outer` برای تقویم مشترک همهٔ سری‌ها یا `inner`.
`annualization`	`int=240`	تعداد جلسات سال برای نوسان، آلفا و tracking error؛ عدد صحیح ۱ تا ۳۶۶.
`min_observations`	`int=20`	حداقل بازده معتبر برای سنجه‌های ریسک؛ کمبود با `NaN` و flag گزارش می‌شود.
`include_client_type`	`bool=True`	دریافت حقیقی/حقوقی در مقایسه؛ خاموش‌کردن آن fan-out را کم می‌کند.
`history_data`	`Mapping[str, DataFrame] | None`	تاریخچهٔ قیمت صریح به تفکیک نماد؛ جایگزین دریافت شبکه و بدون mutation.
`client_data`	`Mapping[str, DataFrame] | None`	تاریخچهٔ حقیقی/حقوقی صریح به تفکیک نماد.
`window`	`int | None=20`	آخرین جلسات سنجهٔ نقدشوندگی؛ `None` یعنی تمام بازهٔ موجود.
`include_live`	`bool=True`	افزودن گردش، depth، imbalance و spread از market cap و snapshot.
`live_data`	`DataFrame | None`	برای تکرارپذیری و جلوگیری از `get_live_market()` صریح snapshot دریافت تازه.
`lookback`	`int=60`	تعداد مشاهدات انتهایی benchmark در مؤلفهٔ trend.
`liquidity_window`	`int=20`	تعداد روزهای انتهایی فعالیت بازار برای میانهٔ ارزش تاریخی.
`weights`	`Mapping[str, number] | None`	وزن `trend/breadth/flow/liquidity/queue`؛ نامنفی، finite و با مجموع مثبت.
`benchmark_history`	`DataFrame | None`	تاریخچهٔ صریح benchmark برای رژیم بازار.
`activity_history`	`DataFrame | None`	تاریخچهٔ صریح فعالیت کل بازار با ستون `Value`.
`group_by`	`str='symbol'`	گروه‌بندی نقشه: `symbol`، `sector`، `flow`، `instrument_type`.
`size`	`str='value'`	اندازهٔ treemap: `value`، `volume`، `market_cap`، `queue_value`.
`color`	`str='return'`	رنگ: `return`، `net_individual_flow`، `individual_power`، `order_imbalance`.
`instrument_types`	scalar/iterable/ `None` / `'all'`	پیش‌فرض انواع سهامی ۳۰۹/۳۰۳/۳۰۰؛ `'all'` فیلتر نوع را حذف می‌کند.
`data`	`DataFrame | None`	ورودی صریح `get_market_map()`؛ در نبود آن snapshot زنده گرفته می‌شود.
`map_data`	`DataFrame | None`	جدول آمادهٔ نقشه برای renderer؛ `None` نقشهٔ پیش‌فرض را می‌سازد.
`output_path`	`str | Path | None`	مسیر فایل HTML تعاملی؛ `None` یعنی فقط ساخت `Figure`.
`show`	`bool=False`	نمایش figure با renderer تنظیم‌شدهٔ Plotly.
`title`	`str | None`	عنوان دلخواه نمودار؛ `None` عنوان خودکار فارسی.

پارامترهای درآمد ثابت و اختیار

نام	type/default	معنا و constraint
`settlement_date`, `maturity_date`, `issue_date`, `valuation_date`	`None` یا date-like	تاریخ تسویه/سررسید/انتشار/ارزش‌گذاری؛ `None` در live یعنی امروز تهران.
`face_value`	`float` ؛ اخزا `1_000_000.0`	ارزش اسمی مثبت؛ `IRAN_TREASURY_FACE_VALUE` همین default است.
`day_count` / `convention`	`str='ACT/365F'`	پشتیبانی‌شده؛ تاریخ پایان باید بعد از شروع convention باشد.
`price`, `annual_yield`, `coupon_rate`, `accrued_interest`	`float`	قیمت/بازده/کوپن/بهرهٔ تحقق‌یافته؛ bounds در تابع math اعتبارسنجی می‌شود.
`cashflows`	iterable `(date, amount)` یا `None`	جریان‌های نقدی؛ در حالت `None` از maturity/face/coupon ساخته می‌شود.
`compounding`, `frequency`, `price_type`	`str`, `int`, `str`	نوع مرکب، دفعات سالانه و `dirty/clean` .
`nodes`	DataFrame/iterable mapping	rate/discount؛ maturity و شامل curve های node. تعیین‌کنندهٔ تکرار است duplicate policy.
`interpolation`, `extrapolate`, `duplicate_policy`	`log_discount`, تابعی, `False`	interpolation discount؛ extrapolation opt-in ؛ سیاست. مستند policy یا `error` duplicate.
`price_source`	`str='auto'`	انتخاب `Last/Close/Final/bid/ask` با provenance.
`maturity_map`	`None` یا mapping	جلوگیری fuzzy صریح نماد→سررسید؛ از حدس override می‌کند.
`spot`, `strike`, `option_price`, `volatility`	`float`	نامنفی volatility نامنفی و premium، مثبت spot/strike.
`time_to_expiry`, `rate`, `dividend_yield`	`float`	سال تا سررسید، نرخ بدون ریسک و yield پیوسته.
`option_type`, `exercise_style`	`str='call'`, `str='european'`	را می‌پذیرد European فعلی فقط math ؛ `call/put` .
`lower_volatility`, `upper_volatility`, `tolerance`, `max_iterations`	`5.0`, `0.0`, تابعی، تابعی	و شمار lower < upper؛ solver و همگرایی bracket مثبت iteration.
`risk_free_rate`, `yield_curve`	`None` یا scalar/curve	نرخ ثابت یا `YieldCurve` ; اگر هر دو داده شوند قرارداد تحلیل آن‌ها را اعتبارسنجی می‌کند.
`parity_tolerance`, `liquidity_weights`	`float | mapping | None`	default یعنی `None` ؛scoring و وزن‌های band parity داخلی مستند.

پارامترهای بورس انرژی، کالا و term structure

نام	type/default	معنا و constraint
`status`	`str='upcoming'`	وضعیت حراج: `upcoming/ready` ، `active` ، `surplus/mazad` ، `ended` یا `all` .
`board`	`str='physical'`	تابلوی حراج: `physical` ، `power` ، `special` ، `green` ، `free` یا `all` .
`auction_id`	`int`	شناسهٔ مثبت حراج رسمی بورس انرژی؛ سقف قراردادی `10**12` .
`include_trades`	`bool=True`	افزودن معاملات خود همان حراج به dict خروجی.
`include_instrument_history`	`bool=False`	افزودن سابقهٔ عرضه‌های قبلی همان ابزار؛ با معاملات حراج جاری یکی نیست.
در انرژی `market`	`str='all'`	در برق: `standard/green/free/all` ؛ در overview یکی از flow labelهای مستند؛ در activity کالا شناسه/نام بازار.
در انرژی `kind`	`str='all'`	`standard_salaf` ، `capacity_certificate` ، `energy_saving_certificate` ، `deposit_certificate` ، `future` یا `all` .
در کالا `kind`	`str='all'`	`certificate` ، `standard_salaf` ، `fund` ، `future` یا `all` .
`active`	`bool=True`	دستهٔ جاری؛ `False` دسته‌های جاری و expired منتشرشده را نیز می‌خواند.
`enrich_futures`	`bool=True`	مشخصات رسمی آتی برای سررسید/اندازه/تسویه؛ هزینه در fan-out `request_count` .
`hall`	`str|int='all'`	همه، ID تالار یا نام exact تالار رسمی بازار فیزیکی.
`contracts`	`DataFrame | iterable[Mapping]`	ردیف‌های قرارداد با ستون سررسید و یکی از قیمت‌های settlement/close/last.
`underlying`	`str|None`	فیلتر substring اختیاری روی دارایی پایه؛ fuzzy identity resolver نیست.
curve در `price`	`str='settlement'`	اولویت ستون قیمت: `close` ، `settlement` یا `last` ؛ ستون انتخاب‌شده در `PriceSource` .
`spot_price`	`float|None`	basis؛ `None` یعنی basis محاسبه نشود. مثبت و صریح برای spot
`curve`	`DataFrame | iterable[Mapping]`	خروجی `get_futures_curve()` یا جدول هم‌قرارداد با ستون‌های لازم.
`annual_rate`	`float`	نرخ مالی سالانهٔ effective به‌صورت اعشاری؛ مثلاً `0.30` .
`storage_rate`	`float=0.0`	هزینهٔ انبارداری سالانهٔ effective به‌صورت اعشاری.
`convenience_yield`	`float=0.0`	منفعت نگهداری کالای فیزیکی به‌صورت نرخ سالانهٔ اعشاری.

پارامترهای کم‌تکرار نیز بخشی از قراردادند:

دامنه	پارامترها و معنا
قیمت history	`auto_adjust` (`bool=True`) تعدیل OHLC؛ `adjust_volume` (`bool=False`) تعدیل volume متناظر.
فهرست بازار	`bourse` , `farabourse` , `payeh` (`bool=True`) و `haghe_taqadom` , `sandogh` , `bonds` , `options` , `mortgage` , `commodity` , `energy` (`bool=False`) سوییچ inclusion؛ `output` (`str='dataframe'`) format؛ `index_name` (`str`) نام InsCode/نماد؛ `fund_type` (`str|list|None`) category؛ `name` (`str|list=''`) نام ارز؛ `include_id` (`bool=False`) شناسهٔ سهامدار شاخص؛
fundamentals	`pe_min` , `pe_max` (`float|None`) مرزهای شامل؛ `positive_pe` (`bool=True`) فقط P/E مثبت معتبر.
live/option	`category` کد بازار اختیار؛ (`int=0`) point opt-in؛ `exchange` (`str='none'`) مسیر `fallback` (`str='treasury'`) دسته IFB؛ `lock_timeout` (`float=10.0`) و `stale_lock_seconds` (`float=300.0`) قفل فایل مثبت.
fixed math	`bump_size` (`float=0.0001`) شوک DV01؛ `rate_compounding` (`str='effective'`), `rate_frequency` (`int=1`); `enforce_monotonic_discount` (`bool=True`) guard curve.
storage	`event` (`MarketEvent`) رخداد ورودی و (`DataFrame`) batch archive.
resolver	`selector` (`Any`) انتخاب ورودی؛ `require_active` (`bool=True`) الزام فعالیت.

فیلدهای باقی‌ماندهٔ `MarketEvent` همگی constructor contract هستند: `sequence` (`int`) شمارهٔ افزایشی، `changed_inscodes` (`tuple[str,...]`) و delta، `changed_order_levels` (`tuple[tuple[str,int],...]`) cursor (`tuple[str,str,str]`)، `notification_tokens` (`bool`)، `market_state_changed` های notification، `retry_error` (`str\|None`), `retry_count` (`int`)، `cursor_after` / `cursor_before` (`int`)، `state_changes` (`DataFrame\|None`)، `notification_errors` (`tuple[str,...]`) و (`str\|None`)) (`persistence_error` / `persistence_status`) هستند. فیلد `is_active` (`bool\|None`) در `InstrumentRef` سه حالت فعال/غیرفعال/نامعلوم دارد.

نوع‌ها، ثابت‌ها، تنظیمات و exceptionهای عمومی

```
InstrumentRef(ins_code: 'str', symbol: 'str | None' = None, name: 'str | None' = None, asset_type: 'str' = 'unknown', is_active: 'bool | None' = None, provenance: 'str' = '', selector: 'str | None' = None) -> None
```

نماد/نام، نوع دارایی، وضعیت فعال، منبع اثبات و canonical، هویت است؛ فیلدها به‌ترتیب شناسهٔ dataclass immutable ورودی هستند. ساخت مستقیم فقط برای دادهٔ از قبل اعتبارسنجی‌شده مناسب است؛ مسیر عادی selector `resolve_instrument()` است. خطای `TypeError` برای field گم‌شده/اضافی است. استاندارد constructor.

```
MarketEvent(kind: 'str', sequence: 'int', fetched_at: 'pd.Timestamp', trade_date: 'Optional[_dt.date]', snapshot: 'dict[str, Any]', changed_inscodes: 'tuple[str, ...]' = (), changed_order_levels: 'tuple[tuple[str, int], ...]' = (), market_state_changed: 'bool' = False, notification_tokens: 'tuple[str, str, str]' = ('', '', ''), messages: 'Optional[pd.DataFrame]' = None, state_changes: 'Optional[pd.DataFrame]' = None, cursor_before: 'int' = 0, cursor_after: 'int' = 0, retry_count: 'int' = 0, retry_error: 'Optional[str]' = None, notification_errors: 'tuple[str, ...]' = (), persistence_status: 'Optional[str]' = None, persistence_error: 'Optional[str]' = None) -> None
```

هستند؛ mutable کپی دفاعی ولی `copy_snapshot=True` با `snapshot/messages/state_changes` است. event watcher را نگه می‌دارند. مثال ساخت دستی لازم نیست؛ نمونهٔ persistence provenance/و رشته‌های خطا delta/cursor هاtuple بالاتر است `watch_market()` موضوعی.

```
YieldCurve(settlement_date: datetime.date, maturities: tuple, times: tuple, discount_factors: tuple,
continuous_zero_rates: tuple, day_count: str = 'ACT/365F', interpolation: str = 'log_discount',
extrapolate: bool = False, node_metadata: tuple = <factory>, diagnostics: dict = <factory>) -> None
```

بسازید. `build_yield_curve()` است؛ آن را با metadata/diagnostics محاسباتی با آرایه‌های هم‌طول و curve immutable می‌دهند. `ValueError` ، `extrapolate=False` خارج از محدوده با interpolation متدهای

ثابت export	type/value	معنا
`MARKET_HISTORY_SCHEMA_VERSION`	`int = 2`	نسخهٔ schema SQLite market history.
`MARKET_HISTORY_APPLICATION_ID`	`int = 1096045381`	برای رد فایل نامرتبط application id SQLite.
`IRAN_TREASURY_FACE_VALUE`	`float = 1_000_000.0`	ارزش اسمی پیش‌فرض اخزا، نه override اجباری همهٔ اوراق.
`OPTION_SNAPSHOT_SCHEMA_VERSION`	`int = 1`	نسخهٔ سند JSON snapshot اختیار؛ reader mismatch را رد می‌کند.

`settings` یک `Settings` از singleton عمومی مهم mutable است. تنظیمات: `ssl_verify: bool=True` , `timeout: int|float=10` , `max_retries: int=3` , `retry_backoff_factor: float=0.3` , `rate_limit_delay: float=0.3` , `market_snapshot_freshness_seconds: float=120.0` , `market_clock_skew_tolerance_seconds: float=5.0` , `client_volume_consistency_tolerance: float=0.05` , `industry_membership_cache_ttl: float=3600.0` , `order_book_max_requests: int=250` , `trade_max_requests: int=250` و `order_book_discovery_lookback_days: int=10` هستند؛ provider قرارداد تنظیم `url_*: str` های روز/ارز/صندوق/بازار پایه وmapping ، `headers: dict` .هستند را نقض کند و برای مصرف عادی توصیه نمی‌شود source-boundary می‌تواند URL تغییر.

دارند: `AlgotikTSEError` مشترک base و `(*args)` ارث‌بردهٔ constructor های عمومی همگیexception برای `DataParsingError` ،HTTP/provider برای `ConnectionError` ،برای چند هویت هم‌رتبه `AmbiguousSymbolError` برای نبود هویت و `StockNotFoundError` ،برای ورودی نامعتبر `InvalidParameterError` ،نامن schema/identity .در فصل تنظیمات است catch برای خروج از مرز منبع. نمونهٔ `UnsupportedDataSourceError`

constructor	قرارداد
`AlgotikTSEError`	نگه می‌دارد؛ `Exception` را مانند context/پیام `args: tuple[Any,...]` ؛base exception ندارد return خودش.
`AmbiguousSymbolError`	چند هویت exact هم‌رتبه؛ راه‌حل ارائهٔ `ins_code` است.
`ConnectionError`	HTTP/status/redirect/provider failure با cause اصلی (`raise ... from exc`).
`DataParsingError`	کور معمولاً مناسب نیست retry هویت/فایل ناسازگار؛/payload/schema.
`InvalidParameterError`	I/O. های جدید پیش ازAPI ورودی؛ در constraint

constructor	قرارداد
`StockNotFoundError`	انجام نمی‌شود fuzzy-first-hit پیدا نشده؛ selector exact.
`UnsupportedDataSourceError`	است fail-before-I/O نمونهٔ `get_introduction`؛ boundary نیازمند منبع خارج API.
`YieldCurve`	نامعتبر field/shape و خطای object curve بالا؛ خروجی constructor dataclass `build_yield_curve` . `TypeError/ValueError` ؛ مثال

قرارداد API: هویت، قیمت، معاملات و APIهای قدیمی

```
normalize_instrument_text(value: 'Any') -> 'str'
resolve_instrument(selector=None, *, ins_code=None, asset_type='auto', snapshot=None,
require_active=True) -> 'InstrumentRef'
validate_ins_code(value: 'Any') -> 'str'
get_history(symbol='', start=None, end=None, limit=0, raw=False, auto_adjust=True,
output_type='standard', date_format='jalali', progress=True, save_to_file=False, dropna=True,
adjust_volume=False, return_type=None, ascending=True, save_path=None, include_today=False, *,
ins_code=None, asset_type='auto', **kwargs)
get_client_type(symbol='', start=None, end=None, limit=0, raw=False, output_type='standard',
date_format='jalali', progress=True, save_to_file=False, dropna=True, ascending=True, save_path=None,
include_today=False, *, ins_code=None, asset_type='auto', **kwargs)
get_capital_increase(symbol='', *, ins_code=None, asset_type='auto', **kwargs)
get_intraday(symbol='شتران', interval='1min', start=None, end=None, progress=True, **kwargs)
get_trades(symbol=None, *, ins_code=None, start=None, end=None, include_canceled=False,
max_requests=None, raw=False, progress=True)
get_live_trades(symbol=None, *, ins_code=None, include_canceled=False, max_requests=None, raw=False,
progress=True)
get_detail(symbol='', *, ins_code=None, asset_type='auto', **kwargs)
get_info(symbol='', *, ins_code=None, asset_type='auto', **kwargs)
get_stats(symbol='', *, ins_code=None, asset_type='auto', **kwargs)
get_introduction(symbol='', *, ins_code=None, asset_type='auto', **kwargs)
get_symbols(bourse=True, farabourse=True, payeh=True, haghe_taqadom=False, sandogh=False, bonds=False,
options=False, mortgage=False, commodity=False, energy=False, payeh_color=None, output='dataframe',
progress=True, **kwargs)
get_shareholders(symbol='', date=None, include_id=False, *, ins_code=None, asset_type='auto', **kwargs)
get_shareholder_history(symbol='', start=None, end=None, frequency='daily', max_requests=60,
include_id=False, progress=True, *, ins_code=None, asset_type='auto')
get_currency(name='', start=None, end=None, limit=0, output_type='standard', date_format='jalali',
progress=True, save_to_file=False, dropna=True, return_type=None, ascending=True, save_path=None,
**kwargs)
get_tgju_history(name='', start=None, end=None, limit=0, output_type='standard', date_format='jalali',
progress=True, save_to_file=False, dropna=True, return_type=None, ascending=True, save_path=None,
**kwargs)
list_tgju_assets(category=None, include_aliases=False)
get_market_snapshot(*args, **kwargs)
get_market_client_type(*args, **kwargs)
```

تابع	ورودی‌ها و گزینه‌ها	خروجی، schema و attrs	خطاهای اصلی و مثال
`normalize_instrument_text`	به رشتهٔ تهی و `None` `value: Any` ؛ متن با یکسان‌سازی ی/ک، فاصله و ZWNJ به کلید مقایسه تبدیل می‌شود.	`str` ؛ ورودی رcanonicall mutate نمی‌کند.	خطای ویژه ندارد؛ `normalize_instrument_text("کگل") == "کگل"`.

تابع	ورودی‌ها و گزینه‌ها	خروجی، schema و attrs	خطاهای اصلی و مثال
`validate_ins_code`	`value: str|int` ؛ integer مثبت یا رشتهٔ ۱..۲۰ رقم ASCII (فاصلهٔ ابتدا/انتها رقم فارسی/ ،bool ،float. (می‌شود strip داخلی، صفر و طول space ،sign ،عربی بیش از ۲۰ رد می‌شوند.	`str` ؛ integer ASCII معتبر دقیقاً به رشته تبدیل می‌شود.	`InvalidParameterError` ؛ resolver. مثال بخش
`resolve_instrument`	`snapshot: dict|DataFrame|None` منبع حتی اگر تهی؛ authoritative `require_active: bool=True` ابزار غیرفعال را حذف می‌کند. اولویت resolver در فصل exact آمده است.	`InstrumentRef` ; مسیر اثبات `provenance` (`explicit_ins_code` , و snapshot/search/index registry مشابه) را ثبت می‌کند.	`InvalidParameterError` , `StockNotFoundError` , `AmbiguousSymbolError` , `DataParsingError` , `ConnectionError` ; مثال exact/ambiguity بالاتر.
`get_history`	`symbol/ins_code/asset_type` هویت؛ `start/end/limit/ascending` schema TSE؛ `raw` بازه و ترتیب؛ `auto_adjust` تعدیل OHLC؛ `adjust_volume` تعدیل حجم؛ `output_type∈{standard,full}` ؛ `date_format∈{jalali,gregorian,both}` ؛ `return_type∈{simple,log,both,list}` `include_today` ؛ `progress/dropna` ردیف زنده؛ نمایش/ترکیب؛ `save_to_file/save_path` ذخیرهٔ CSV؛ `kwargs` فقط aliasهای مستند.	برای سهم و `auto_adjust=True` ، `output_type='standard'` دقیقاً `Open,High,Low,Close,Volume` است؛ ستون‌های `full` و تقویم/ `Final,No.,Value` را اضافه می‌کند. با `Ticker` ، `auto_adjust=False` standard ستون `Adj Close` هم دارد. index/industry schema محدودتر خود را دارند؛ چند نماد MultiIndex. attrs پیش‌فرض تهی و با `include_today=True` شامل `include_today_appended/include_today_warning` است.	ورودی‌های ناسازگار legacy معمولاً `None` / `ValueError` و `ConnectionError` provider; مثال فصل قیمت.
`get_client_type`	`symbol/ins_code/asset_type` هویت؛ `start/end/limit/ascending` schema provider؛ `raw` بازه؛ `output_type∈{standard,full}` ؛ `date_format∈{jalali,gregorian,both}` `include_today` ؛ `progress/dropna` ردیف امروز؛ نمایش/چندنمادی؛ `save_to_file/save_path` CSV؛ `kwargs` aliasهای قدیمی.	روزانهٔ `DataFrame` , `Buy_I/N_Count` , `Buy_I/N_Volume` , `Sell_I/N_Count` , `Sell_I/N_Volume` قدرت/ , سرانه‌های مشتق؛ چند نماد opt-in است. ردیف امروز MultiIndex.	خطای selector/provider یا legacy `None` ; مثال فصل حقیقی/حقوقی.
`get_capital_increase`	`stock=` . قدیمی alias و selector	`DataFrame|None` با index `date` و `old_shares_amount,new_shares_amount` .	در provider/نماد گم‌شده/index `None` ؛ قرارداد قدیمی پیام و `att.get_capital_increase("فملی")` .

تابع	ورودی‌ها و گزینه‌ها	خروجی، schema و attrs	خطاهای اصلی و مثال
`get_intraday`	`interval` یکی از canonical `tick,1min,5min,15min,30min,1h,4h,12h` و aliasهای مانند `_INTERVAL_MAP` `1m,60min,4hour,240m,12hour,720,ticks,raw`؛ بدون تاریخ snapshot `start` معاملات امروز، با تاریخی.	`DataFrame|None` ; شامل tick شامل candle زمان/قیمت/حجم و `Open,High,Low,Close,Volume,TradeCount` با DatetimeIndex.	نامعتبر یا تاریخی که interval قدیمی نامعتبر تشخیص validator `None` دهد پیام چاپ می‌کند و نماد نیز در مسیر/provider می‌دهد؛ `ValueError` ؛ `None` legacy نیست؛ API خطای عمدی قرارداد این مثال `att.get_intraday("فملی", interval="5min")` .
`get_trades`	بدون تاریخ امروز تهران؛ Thu/Fri قبل از budget حذف؛ `include_canceled` ; `max_requests` فقط Trade endpoint؛ `raw` ستون‌های provider را اضافه می‌کند.	standard: `DataFrame[InsCode,Symbol,GregorianDate,JalaliDate,TradeNo,Time,Timestamp,Price,Volume,Value,Canceled,Source]` ; شامل attrs request/provenance/failures. raw نیز دارد audit ستون‌های.	`InvalidParameterError` , resolver errors, `ConnectionError` , مثال ; `DataParsingError` فصل معاملات.
`get_live_trades`	هویت؛ `symbol/ins_code` رکورد ابطالی؛ `include_canceled` سقف درخواست؛ `max_requests` audit schema؛ `raw` `progress` نمایش. تاریخ به‌صورت خودکار امروز تهران است.	دقیق attrs و schema معاملات امروز با : `get_trades` زمان، قیمت، حجم، ارزش، وضعیت provenance. ابطال و	`InvalidParameterError` , resolver errors, `ConnectionError` , `DataParsingError` ; `att.get_live_trades(ins_code="...")` .
`get_detail`	selector دقیق؛ API HTML قدیمی.	`DataFrame|None` با index به‌همراه ، `value` و ستون `key` row `id` .	در رفتار index/no-match/HTTP مثال ; `None` legacy `att.get_detail("فملی")` .
`get_info`	selector دقیق.	`DataFrame|None` از key/value `instrumentInfo` . کامل flatten	`None` یا legacy ؛ مثال فصلconnection/parsing اطلاعات نماد.
`get_stats`	selector دقیق.	`DataFrame|None` key/value عددی value آمار با کلید فارسی و.	`None` یا legacy connection/parsing؛ `att.get_stats("فملی")` .
`get_introduction`	؛ هیچ پارامترBC فقط برای signature نمی‌شود I/O باعث.	هرگز خروجی موفق ندارد.	همیشه `UnsupportedDataSourceError` **پیش از I/O**؛ جایگزین market-data: `get_info/get_detail` .
`get_symbols`	boolean های market/asset، `payeh_color: str|list|None` ; `output: str='dataframe'` ; aliasهای `kwargs` انگلیسی در .	`DataFrame` فهرست ابزارها یا قدیمی انتخابی؛ ستون‌های format ؛ خروجی تهیtype هویت/نام/بازار و پایدار دارد schema.	فیلتر/output نامعتبر `ValueError` یا `None` ;legacy مثال فصل فهرست ابزارها.

تابع	ورودی‌ها و گزینه‌ها	خروجی، schema و attrs	خطاهای اصلی و مثال
`get_shareholders`	`symbol/ins_code/asset_type` آخرین فهرست `date=None` هویت؛ و تاریخ مشخص، روز معامله است؛ `include_id=False` شناسهٔ رکورد هایalias را حذف می‌کند؛ snapshot `stock/shh_id` حفظ شده‌اند.	`DataFrame|None[share_holder_name,number_of_shares,percentage_of_shares,change_state,change_amount,date,trade_date,effective_date,trade_date_jalali,effective_date_jalali,identity_quality,change_quality,source]` opt-in و با `share_holder_id` . `date` و `effective_date` همان معمولاً روز معاملاتی بعد از است؛ تغییر با نام `trade_date` محاسبه و برای ورودی مبهم exact هشدار انتقال attrs .می‌شود `NA` غیرمعاملاتی، تأخیر جلسه، نبود شناسهٔ پایدار و غیررسمی‌بودن ثبت را دارد.	تاریخ در قرارداد/provider/selector `None` ؛ مثال فصل پیام و legacy .اطلاعات نماد
`get_shareholder_history`	اجباری و بر مبنای **تاریخ** `start/end` **مؤثر**؛ `frequency∈{daily,weekly,monthly}` `max_requests=60`؛ `include_id` ; سقف سخت ۱..۱۰۰۰؛ با exact هویت ; `progress` `symbol/ins_code/asset_type` . daily هر دو snapshot را پاسخ آخرین جلسهٔ weekly مصرف می‌کند؛ آخرین جلسهٔ monthly هفتهٔ شنبه‌محور و .ماه شمسی است	همان schema توسعه‌یافتهٔ `get_shareholders` در -long form برای چند تاریخ. attrs شامل `date_basis='effective_date'` , `coverage_start/end` , `requested_snapshot_count` , `returned_snapshot_count` , `missing_effective_dates` , `shareholder_requests_used` , `pair_request_optimization` , `change_period` و تمام هشدارهای رسمی منبع است.	بازه/enum/bool/bودجه `InvalidParameterError` ؛ هویت، شبکه یا schema خطاهای typed؛ مثال `att.get_shareholder_history('فولاد', start='1404-01-01', end='1404-12-29', max_requests=150)`.
`get_currency`	canonical، نام `name: str|list` برای هر دارایی کاتالوگ؛ slug فارسی یا `start/end/limit/ascending` schema؛ `output_type` بازه؛ `date_format∈{jalali,gregorian,both}` `return_type` بازده؛ نمایش/ `progress/dropna` چنددارایی؛ `save_to_file/save_path` .های قدیمیalias `kwargs` ؛CSV	تک دارایی `DataFrame[Open,High,Low,Close]` با attrs منبع/ نام/slug/category/unit؛ چند دارایی ستون MultiIndex.	نام `InvalidParameterError` ، HTTP `ConnectionError` ، schema `DataParsingError`؛ `att.get_currency("gold-18k", limit=30)`.
`get_tgju_history`	قرارداد و defaultهای دقیق `get_currency` ؛ نام پیشنهادی چون دامنه فقط ارز نیست.	همان frame/MultiIndex و attrs؛ wrapper معنایی است.	همان؛ `att.get_tgju_history("طلای ۲۴ عیار", limit=30)`.

تابع	ورودی‌ها و گزینه‌ها	خروجی، schema و attrs	خطاهای اصلی و مثال
`list_tgju_assets`	`category=None` یا str/sequence از هفت دسته؛ `include_aliases=False` tuple ها را اضافه می‌کندalias.	`DataFrame[Name,PersianName,Slug,Category,Unit]` و attrs `Aliases` ؛ opt-in در حالت endpoint و عرض ردیف تأییدشده.	نامعتبر category/bool ؛ `InvalidParameterError` بدون شبکه؛ `att.list_tgju_assets("gold")` .
`get_market_snapshot`	به تابع BC برای `*args/**kwargs` صفرآرگومان `market_watch` می‌شود؛ در عمل آرگومان غیرتهی forward می‌دهد `TypeError`.	با کلیدهای دقیق `dict` `stocks,order_book,market_time,index_value,migration,trade_date,market_state,exchange_time,fetched_at,snapshot_age_seconds,is_today_trade_date,is_history_eligible,is_realtime_fresh,is_previous_trade_date,is_stale,is_partial` .	`ConnectionError` , مثال ; `DataParsingError` live.
`get_market_client_type`	`*args/**kwargs` wrapper `market_client_type` .	`DataFrame[InsCode,Buy_I_Count,...,Net_I_Volume,Net_N_Volume]` .	`ConnectionError` , مثال ; `DataParsingError` live.

قرارداد API: live، سفارش، watcher و تحلیل بازار

```
get_order_book(symbol=None, *, selector_strict=False)
get_live_market(symbol=None, *, strict=False)
get_live_symbol(symbol=None, *, ins_code=None, fallback='none')
get_order_book_history(symbol='', start=None, end=None, limit=0, raw=False, output_type='standard',
date_format='jalali', progress=True, save_to_file=False, dropna=True, ascending=True, save_path=None,
include_today=False, complete_only=False, *, max_requests=None, _request_budget_state=None, **kwargs)
get_orderbook_history(symbol='', start=None, end=None, limit=0, raw=False, output_type='standard',
date_format='jalali', progress=True, save_to_file=False, dropna=True, ascending=True, save_path=None,
include_today=False, complete_only=False, *, max_requests=None, _request_budget_state=None, **kwargs)
get_queue(symbol=None, side='both', strict=True, *, selector_strict=False)
get_queue_history(symbol='', start=None, end=None, limit=0, date_format='jalali', progress=True,
save_to_file=False, dropna=True, ascending=True, save_path=None, include_today=False,
complete_only=False, side='both', strict=True, *, max_requests=None, **kwargs)
MarketWatcher(symbol=None, interval=1.0, max_updates=None, include_initial=True, emit_heartbeats=True,
notifications=('messages', 'state'), notification_top=50, stop_event=None, error_policy='retry',
max_backoff=30.0, jitter=0.1, callback_error_policy='raise', max_consecutive_retries=5,
max_retries=None, max_seen_notifications=10000, copy_snapshot=True, request_timeout=None, *,
record_to=None, record_heartbeats=False, checkpoint_interval=100, record_max_records=10000,
record_retention_seconds=None, storage_error_policy='raise', _request=safe_get, _clock=None,
_wait=None, _random=None)
watch_market(*args, **kwargs)
get_market_messages(flow=0, top=20, since_id=None, *, archive_to=None, _recorded_at=None,
_request=safe_get)
get_instrument_state_changes(top=20, since_id=None, *, archive_to=None, _recorded_at=None,
_request=safe_get)
get_market_overview(flow=0, *, archive_to=None, _recorded_at=None, _request=safe_get)
get_market_breadth(symbol=None, flow=None, sector=None, traded_only=False, include_base_market=True,
instrument_types=None, *, _snapshot=None)
get_sector_flow(symbol=None, flow=None, sector=None, traded_only=False, include_base_market=True,
instrument_types=None, *, _snapshot=None, _client_type=None)
```

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
`get_order_book`	scalar/list/ `None`؛ فقط `selector_strict` انتخاب را سخت می‌کند resolution.	با long `DataFrame` `InsCode,Symbol,Name,Level,BidOrderCount,BidVolume,BidPrice,AskPrice,AskVolume,AskOrderCount` و metadata `trade_date,market_state,exchange_time,fetched_at,is_*` پنج سطح از همان response snapshot.	`InvalidParameterError`، در `StockNotFoundError`؛ مثال connection/parsing، strict. فصل سفارش.

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
`get_live_market`	`strict=False` selectorهای گم‌شده را در `attrs['missing_selectors']` ثبت می‌کند؛ strict آن‌ها را خطا می‌کند.	`DataFrame` با client، `STOCK_COLUMNS` wide، سطح‌های columns، `BidPrice1..5/AskPrice1..5` و metrics جمله از `EstimatedNetIndividualFlow` ; freshness ستون row-wise attrs. است دقیق: `field_validity,source_schema_presence,migration,missing_selectors` . `Last` آخرین معامله و `Close` پایانی است.	`InvalidParameterError` ، در `StockNotFoundError` strict، `ConnectionError/DataParsingError` و مثال quickstart ; attrs بالا.
`get_live_symbol`	`fallback: 'none'|'point'` ; MarketWatch. فقط در غیاب point برای `fallback='none'` no-match frame تهی نمی‌دهد.	exception؛ دقیقاً یک‌ردیفی یا frame provenance `SnapshotSource='market_watch'|'closing_price_info_fallback'` و ستون‌های `PresentInMarketWatch,MarketStateTitle,has_trade_today,PriceActionable,FallbackReason,IdentityVerified` .	no-match با fallback none: `StockNotFoundError` ; همچنین `InvalidParameterError` ، `AmbiguousSymbolError` ، `DataParsingError` ; فصل live.
`get_order_book_history`	`raw` ; `output_type='standard'` long و `'wide'` ; `complete_only` ; workflow. این calls همهٔ budget	long/wide/raw `DataFrame` ; سطح و ۵ identity/time ستون‌های `is_partial,is_complete,market_partial_status,is_reconstructed,is_stale,record_type,source` ; attrs `schema,failed_requests,request_count,max_requests` .	`InvalidParameterError` ، `DataParsingError` ، resolver/connection؛ failure ؛ مثال فصل attrs + warning جزئی تاریخچه سفارش.
`get_orderbook_history`	کاملاً یکسان با signature هویتی و alias `get_order_book_history` .	دقیقاً همان object/return/schema/attrs.	همان خطاها و همان مثال؛ `att.get_orderbook_history is att.get_order_book_history` .
`get_queue`	فقط `strict=True` ; `side` های قطعی را نگه می‌داردqueue.	`DataFrame[InsCode,Symbol,Name,...,Side,QueuePrice,QueueVolume,QueueOrders,QueueValue,PriceLimit,is_queue,book_state,...]` .	نامعتبر side `InvalidParameterError` ; مثال selector/provider errors؛ صف.

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
`get_queue_history`	`symbol/start/end/limit/ascending` تاریخ؛ `date_format` بازه؛ `include_today` live؛ `complete_only` snapshot کامل؛ `side∈{buy,sell,both}` ؛ `strict` فقط صف قطعی؛ `max_requests` بودجه؛ `progress/dropna` نمایش/تهی؛ `save_to_file/save_path` CSV؛ `kwargs` alias. threshold از as-of snapshot است. همان تاریخ و	`is_queue` ؛ `QUEUE_COLUMNS` nullable، crossed book false؛ attrs budget/failures.	`InvalidParameterError` , resolver/connection/parsing؛ مثال فصل صف.
`MarketWatcher`	فقط `notifications` ; policy enumها `messages/state` glossary؛ دقیقاً در `interval,max_backoff>=0` , `0<=jitter<=1` , retry limit نامنفی، `request_timeout/record_retention_seconds>0` ; دارای `stop_event` `is_set/wait` ; `record_to` opt-in.	`MarketEvent` ; iterator سنکرون kindهای `initial/delta/heartbeat/resync` ; constructor I/O خود نمی‌کند.	نامعتبر policy/range `InvalidParameterError` iteration؛ runtime پیش از `ConnectionError/DataParsingError` ؛ مثال policy طبق watcher.
`watch_market`	تمام `*args/**kwargs` بدون تغییر به `MarketWatcher` می‌روند.	`MarketWatcher` , نه DataFrame.	همان constructor/runtime؛ مثال `for event in att.watch_market(max_updates=3): ...` .
`get_market_messages`	`since_id` فیلتر local؛ `archive_to` نوشتن اتمیک opt-in.	`DataFrame[message_id,date,time,timestamp,title,description,flow]` ; attrs provenance/archive.	`InvalidParameterError` , `ConnectionError` , `DataParsingError` , storage ؛. مثال فصل پیام error
`get_instrument_state_changes`	`since_id` bounded؛ `top` تعداد فقط رکوردهای بعد از شناسه؛ `archive_to` مسیر SQLite opt-in؛ `_recorded_at/_request` فقط seam تست.	`DataFrame[event_id,date,time,timestamp,InsCode,Symbol,Name,state_code,state,real_time,under_supervision,state_title]` .	`InvalidParameterError` , `ConnectionError` , یا خطای `DataParsingError` ؛. مثال فصل وضعیت storage
`get_market_overview`	`flow` ; payload است تهی صفر ردیف.	provider overview `DataFrame` با payload/fast-view و فیلدهای `flow` ؛ attrs source/time/archive.	parameter/connection/parsing/storage مثال overview.
`get_market_breadth`	فیلترهای universe؛ denominator ابزار انتخاب‌شده.	`DataFrame` یک‌ردیف با counts/percentages، A/D، volume/value، limit counts و freshness. attrs analytics/source.	`InvalidParameterError` , `DataParsingError` ; مثال breadth.

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
`get_sector_flow`	فیلتر؛ `symbol/flow/sector` universe `traded_only` معامله‌شده؛ بازار پایه؛ `include_base_market` کد نوع ابزار؛ `instrument_types` فقط `_snapshot/_client_type` فقط برای ردیف client feed .تست .شده به‌کار می‌رودreconcile	یک ردیف در هر `SectorCode` با breadth + `client_coverage,net_individual_volume,estimated_net_individual_value,value_available,value_method` و .freshness	parameter/connection/parsing؛ مثال sector.

قرارداد API: فعالیت سهامداران عمده و timeline نماد

```
get_major_shareholder_snapshots(date=None, days=5, symbol=None, *, ins_code=None, holder=None, enrich_identity=True, progress=True)
get_major_shareholder_changes(date=None, days=5, symbol=None, *, ins_code=None, holder=None, direction='both', enrich_identity=True, progress=True)
get_active_shareholders(symbol=None, days=5, *, ins_code=None, holder=None, enrich_identity=True, progress=True)
rank_shareholder_accumulation(days=5, symbol=None, *, ins_code=None, holder=None, direction='both', metric='percent', top=20, enrich_identity=True, progress=True)
get_shareholder_network(date=None, symbol=None, *, ins_code=None, holder=None, min_holdings=0, enrich_identity=True, progress=True)
get_ownership_concentration(symbol='', date=None, *, ins_code=None, asset_type='auto', top_n=5, progress=True)
get_symbol_events(symbol=None, start=None, end=None, kinds=('messages', 'capital_changes', 'price_adjustments'), limit=100, ascending=False, progress=True, *, ins_code=None, max_requests=3)
```

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
`get_major_shareholder_snapshots`	یا یک تاریخ منتشرشده؛ `date=None` نادیده date و در حضور `days=1..5` گرفته می‌شود؛ `symbol/ins_code` گرفته؛ exact فیلتر نام `holder` فیلتر ابزار؛ اتصال `enrich_identity=True` دقیق هویت؛ `progress` .	`DataFrame[HolderRecord,HolderName,InsCode,Symbol,InstrumentName,GregorianDate,JalaliDate,Holdings,Source,FetchedAt]` . attrs شامل `available_dates` , `coverage_start/end` , `rolling_window=True` , `no_backfill=True` , `is_complete_major_shareholder_list=False` , `stable_shareholder_id_available=False` , `unmatched_identity_count` .	خارج پنجره days/bool/date ؛ `InvalidParameterError` قرارداد/عدد/شناسهٔ خراب ؛ شبکه `DataParsingError` `ConnectionError` . مثال فصل .فعالیت سهامداران

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
`get_major_shareholder_changes`	همان selectorها؛ `direction='both'` برای تغییرهای غیرصفر یا `increase/decrease/unchanged` . `days=1` آخرین تاریخ را همچنان با snapshot قبلی موجود مقایسه می‌کند.	به‌علاوهٔ schema snapshot `PreviousGregorianDate,PreviousJalaliDate,PreviousHoldings,ChangeShares,Direction` . `ChangeShares = Holdings - PreviousHoldings`؛ قدیمی‌ترین snapshot پنجره prior ندارد.	enum/date/provider typed؛ مثال `att.get_major_shareholder_changes(days=1, direction='increase')` .
`get_active_shareholders`	`symbol/ins_code` , `holder` , `days=1..5` , `enrich_identity` , `progress` . فقط تغییرهای غیرصفر را تجمیع می‌کند.	با `DataFrame` `InitialHoldings,LatestHoldings,NetChangeShares,GrossIncreaseShares,GrossDecreaseShares,ActiveDays,ActivityDirection` ; direction یکی از `accumulation/distribution/mixed` . attrs تعریف activity و rolling feed را حفظ محدودیت می‌کند.	همان خطاهای snapshot/change؛ مثال `att.get_active_shareholders(symbol='وبملت')` .
`rank_shareholder_accumulation`	exact فیلتر ; `days=1..5` ؛ `symbol/ins_code/holder` ؛ `direction∈{both,accumulation,distribution}` ؛ `metric∈{percent,shares}` ؛ `top>0` enrichment/progress.	هر ردیف یک جفت سهامدار–نماد با `Rank,NetChangePercent,RankingMetric,Score` و `Score` ستون‌های activity است. قدرمطلق metric و `ranking_unit='holder_instrument_pair'` است؛ تعداد سهم نمادها با هم جمع نمی‌شود.	enum/top/provider typed؛ مثال بالا.
`get_shareholder_network`	یا یکی snapshot آخرین `date=None` نماد/ exact از تاریخ‌های موجود؛ فیلتر ; `min_holdings>=0` سهامدار؛ enrichment/progress.	با edge list `HolderNode,InstrumentNode,Holdings,HolderInstrumentCount,InstrumentHolderCount` فقط در همان holder شناسهٔ . نوع شبکه و attrs پاسخ اعتبار دارد؛ شناسه را نگه می‌دارد scope.	تاریخ خارج پنجره، حد منفی و provider typed؛ مثال بالا.
`get_ownership_concentration`	هویت exact `symbol/ins_code/asset_type` ؛ `date` تاریخ معامله یا `None` برای آخرین فهرست؛ progress ; `top_n>0` .	یک ردیف با `DisclosedOwnershipPercent,Top1/3/5/10Percent,TopNPercent,MajorHolderHHI,NormalizedDisclosedHHI` . هر دو HHI روی مقیاس ۰..۱۰۰۰۰اند؛ HHI سهامداران گزارش‌شده کران پایین HHI کامل است و attrs صریح می‌کند که فهرست دفتر کامل مالکیت نیست.	selector/top/provider typed؛ درصد منفی، بیش از ۱۰۰ برای یک دارنده یا مجموع ناسازگار DataParsingError ؛ مثال `att.get_ownership_concentration('فولاد', top_n=5)` .

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
`get_symbol_events`	هویت exact؛ inclusive `start/end`؛ یک یا چند مورد `kinds` از `messages/capital_changes/price_adjustments`؛ `limit=100` و صفر یعنی همه؛ `ascending=False`؛ `max_requests=3` باید پیش از I/O تعداد sourceهای انتخابی را پوشش دهد.	`DataFrame[EventID,EventType,InsCode,Symbol,Name,GregorianDate,JalaliDate,Timestamp,Title,Description,OldValue,NewValue,ChangeValue,Source,FetchedAt]`. attrs شامل `sources`, `request_count`, `codal_included=False`, `excluded_disclosure_kinds`, `price_adjustment_is_not_confirmed_dps=True`, `no_fuzzy_join=True`.	هویت/enum/تاریخ/budget یا `InvalidParameterError` resolver error؛ شبکه/schema typed. مثال فصل timeline نماد.

قرارداد API: تقویم، نمای کلان، مستر ابزار و پیش‌گشایش

```
get_trading_calendar(start=None, end=None, market='all', include_closed=False, limit=90)
get_market_activity(start=None, end=None, market='all', max_requests=30, progress=True)
get_market_value_history(start=None, end=None, market='all', limit=90)
get_index_impact(date=None, market='all', top=10, direction='both')
get_market_trades(date=None, market='all', progress=True)
get_instrument_master(active=None, market=None, asset_type=None, progress=True)
get_instrument_changes(previous, current=None, progress=True)
get_theoretical_opening_price(*, username=None, password=None, market='all', timeout=None,
progress=True)
get_preopen_imbalance(top_data=None, *, username=None, password=None, market='all', timeout=None,
progress=True)
```

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
`get_trading_calendar`	تاریخ شمسی/میلادی؛ `start/end` بورس/فرابورس؛ `market='all'` فقط `include_closed=False` در نبود بازه، `limit=90` session؛ مثبت و حداکثر ۵۰۰۰.	`DataFrame[Market,Flow,GregorianDate,JalaliDate,IsTradingDay,ReferenceIndex,ReferenceInsCode,Source,]` شامل attrs؛ `calendar_semantics='published_reference_index_sessions'`.	ترتیب/نوع تاریخ، بازار، bool یا limit نامعتبر؛ `InvalidParameterError` typed؛ مثال فصل payload/شبکه؛ تقویم.
`get_market_activity`	بازهٔ session؛ `start/end` `market` ; `max_requests=30` بودجهٔ سخت ۱..۳۶۶؛ `progress`. بدون بازه آخرین جلسه را می‌گیرد.	`DataFrame[Market,GregorianDate,JalaliDate,InstrumentCount,TradedInstrumentCount,TradeCount,Volume,Value,AverageTradeValue,Source]` مشتق از bulk daily و attrs؛ request count.	بازه/بودجه/market نامعتبر؛ `InvalidParameterError` typed؛ مثال فصل نمای کلان؛ provider.

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
`get_market_value_history`	`start/end` ؛ `market` ؛ start مثبت تا ۵۰۰۰؛ با `limit=90` داخلی ۵۰۰۰ است fetch سقف.	`DataFrame[Market,Flow,GregorianDate,JalaliDate,MarketValue,Change,ChangePct,Source]` ؛ واحد `attrs['unit']='rial'` .	پارامتر/تاریخ `InvalidParameterError` و connection/schema typed؛ مثال فصل نمای کلان.
`get_index_impact`	آخرین جلسه؛ `date=None` مثبت تا ۱۰۰۰؛ `top=10` ؛ `market` ؛ `direction∈{both,positive,negative}` .	`DataFrame[Market,Flow,GregorianDate,JalaliDate,Rank,InsCode,Symbol,Name,Close,Impact,Direction,AbsShare,Pct,Source]` ؛ attrs. درصد در denominator	direction/top/date/market نامعتبر `InvalidParameterError` ؛ مثال provider typed؛ `att.get_index_impact(top=10)` .
`get_market_trades`	آخرین جلسه؛ `date=None` فیلتر بازار با . `market` ؛ `progress` پیشوند ISIN است.	یک خلاصهٔ روزانه برای هر ابزار با هویت، OHLC، معامله/حجم/ارزش و تغییر؛ `attrs['transaction_level']=False` و شمار طبقه‌بندی نامعلوم.	parameter/connection/schema ؛ برای معاملات ریز ازtyped استفاده شود؛ مثال `get_trades` فصل bulk.
`get_instrument_master`	`active: bool|None` ؛ `market: str|None` ؛ `asset_type: str|iterable|None` از ۱۰ نوع None غیر active . `progress` مستند؛ اضافه می‌گیرد MarketWatch یک.	`DataFrame[InsCode,ISIN,Symbol,Name,EnglishName,CompanyCode,CompanyISIN,Market,MarketGroup,Industry,AssetType,InstrumentStatus,Active,Source]` identity؛ key در attrs.	نامعتبر filter/type ؛ `InvalidParameterError` HTML/schema/network typed؛ مثال ذخیرهٔ Parquet.
`get_instrument_changes`	اجباری و `previous` `current=None` ؛ هرکدام DataFrame یا CSV/Parquet/JSON؛ `progress` . کلید اجباری . `InsCode`	long-form `DataFrame[ChangeType,InsCode,Field,OldValue,NewValue,Symbol,Source]` ؛ و `added/removed/changed` attrs. در snapshot دو count	مسیر/پسوند/ستون نامعتبر `InvalidParameterError` ؛ current None ممکن است provider error بدهد؛ بدون backfill تاریخی.
`get_theoretical_opening_price`	keyword-only اشتراک `username/password` ؛ `timeout` ؛ `market` رسمی؛ `progress` . credentials در package ذخیره نمی‌شوند.	،رسمی با قیمت/حجم تئوریک TOP attrs و تغییر؛ remainder، buy/sell `official=True,subscriber_service=True,credentials_stored=False` .	ناموجود credentials ؛ `InvalidParameterError` SOAP/network یا `ConnectionError` ؛ مثال `DataParsingError` فصل TOP.
`get_preopen_imbalance`	`top_data: DataFrame|None` ؛ از None در `username/password/market/timeout/progress` برای TOP استفاده می‌کند.	همهٔ ستون‌های + TOP `ImbalanceVolume,ImbalanceRatio,ImbalanceSide` ؛ فرمول دقیق در attrs.	نامعتبر type/ستون/bool ؛ `InvalidParameterError` ؛ مثال fetch در حالت TOP خطاهای فصل TOP.

قرارداد API: تاریخچهٔ محلی و fundamentals

```
get_market_fundamentals(symbols=None, *, pe_min=None, pe_max=None, positive_pe=True, instrument_types=(300, 303, 309), strict=False, allow_stale=False, archive_to=None, max_requests=1, progress=True)
get_market_fundamentals_history(path, start=None, end=None, symbols=None, *, pe_min=None, pe_max=None, positive_pe=True, instrument_types=(300, 303, 309), strict=False, allow_stale=True, limit=1000, offset=0)
get_price_adjustments(symbol=None, *, ins_code=None, start=None, end=None, progress=True)
get_latest_price_adjustment(symbol=None, *, ins_code=None, start=None, end=None, progress=True)
check_market_history(path)
save_market_snapshot(path, snapshot=None, *, as_of=None, _live_fetch=None)
load_market_snapshots(path, start=None, end=None, symbol=None, *, limit=1000, offset=0)
get_live_market_history(path, start=None, end=None, symbol=None, *, limit=1000, offset=0)
get_market_overview_history(path, start=None, end=None, flow=0, *, source='tsetmc', limit=1000, offset=0)
get_market_snapshot_summary_history(path, start=None, end=None, flow=None, *, limit=1000, offset=0)
get_market_breadth_history(path, start=None, end=None, symbol=None, flow=None, sector=None, traded_only=False, include_base_market=True, instrument_types=None, *, limit=1000, offset=0)
get_sector_flow_history(path, start=None, end=None, symbol=None, flow=None, sector=None, traded_only=False, include_base_market=True, instrument_types=None, *, limit=1000, offset=0)
record_market_event(path, event, *, session_id=None, include_snapshot=True, max_records=10000, retention_seconds=None)
get_market_event_history(path, start=None, end=None, kind=None, *, session_id=None, limit=1000, offset=0)
archive_market_records(path, kind, frame, *, source='tsetmc', recorded_at=None)
get_market_messages_history(path, start=None, end=None, flow=0, since_id=None, *, source='tsetmc', limit=1000, offset=0)
get_instrument_state_changes_history(path, start=None, end=None, symbol=None, since_id=None, *, inscode=None, source='tsetmc', limit=1000, offset=0)
```

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
`get_market_fundamentals`	`pe_min/pe_max: float|None` شامل مرز؛ `positive_pe=True` مثبت؛ P/E فقط؛ `archive_to` bulk. `max_requests=1`	schema ثابت بالا؛ attrs دقیق `request_count,max_requests,missing_selectors,strict,stale_rejected,source,price_source,eps_source,no_lookahead,no_backfill,archive_path` .	در bounds/filter/no-match strict: `InvalidParameterError` با stale/resolver error؛ `allow_stale=False` frame exception؛ است، نه attr تهی و ؛ مثال provider typed errors fundamentals.
`get_market_fundamentals_history`	`path` SQLite؛ `start/end/limit/offset` فیلتر؛ `symbols` صفحه و بازه؛ `pe_min/pe_max/positive_pe` غربال P/E؛ `instrument_types` universe؛ `strict` no-match؛ `allow_stale` snapshot نگه‌داشتن قدیمی.	schema زنده با attrs `missing_selectors,strict,source,price_source,eps_source,current_eps_used,no_lookahead,no_backfill,coverage_start,coverage_end` ; `current_eps_used=False` .	file/schema/filter یا `InvalidParameterError` مثال ; `DataParsingError` backtest.

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
`get_price_adjustments`	شامل date bounds دقیق و selector.	`DataFrame[InsCode,Symbol,GregorianDate,JalaliDate,AdjustedClosingPrice,UnadjustedClosingPrice,AdjustmentAmount,CorporateTypeCode,CorporateActionType,IsConfirmedDPS,IdentityVerified,Source,FetchedAt]` ; `IsConfirmedDPS=False` , attrs `dps_available=False` .	resolver/parameter/connection/parsing؛ مثال فصل تعدیل.
`get_latest_price_adjustment`	هویت؛ `symbol/ins_code` فیلتر تاریخ؛ `start/end` نمایش. پس از اعمال فیلتر `progress` فقط جدیدترین رخداد انتخاب می‌شود.	schema `get_price_adjustments` ، attrs و typed صفر یا یک ردیف provenance.	`InvalidParameterError` , resolver errors, `ConnectionError` , مثال ; `DataParsingError` فصل تعدیل.
`check_market_history`	موجود باشد SQLite باید `path`.	دقیقاً `dict[str, bool|int]` با `ok=True,SchemaVersion,ApplicationID` ؛DataFrame نیست.	فایل گم‌شده `InvalidParameterError` ؛ غیرSQLite/نسخه ناسازگار ؛ مثال `DataParsingError` `att.check_market_history(db)` .
`save_market_snapshot`	`snapshot: DataFrame|dict|None` ; اگر `as_of` فقط یک fetch؛ `None` override مشاهده.	`snapshot_id` ; SHA-256 `str` ، همزمان و live/client/order با DB داخل attrs اتمیک؛ transaction بازیابی می‌شود reader.	path/type/schema/storage `InvalidParameterError/DataParsingError` ; network ؛ مثالsnapshot=None فقط هنگام SQLite.
`load_market_snapshots`	فیلتر inclusive زمان و symbol؛ pagination SQL.	ردیف‌های `DataFrame` با observation `STOCK_COLUMNS` و meta `SnapshotID,AsOf,Source,SchemaVersion,NoBackfill,CoverageStart,SnapshotAtomic,PersistenceAtomic,SourceAtomic,PriceSourceAsOf,ClientSourceAsOf,NoLookahead` .	؛ مثالfile/schema/filter typed SQLite.
`get_live_market_history`	`path` SQLite؛ `start/end/symbol/limit/offset/ascending/include_stale` معنایی alias. برای فیلتر و صفحه‌بندی alias identity. است، نه reader rich live	سازگار با frame/meta `load_market_snapshots` .	`InvalidParameterError` و خطاهای file/schema؛ `att.get_live_market_history(db, symbol="فملی")` .

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
`get_market_overview_history`	`flow/source`، مستقل archive.	payload exact provider + قبلی meta archive `ArchiveIdentity,Version,FirstObservedAt,ObservedAt,ProviderTimestamp,AsOf,Source,SchemaVersion,NoBackfill,...`.	kind/source/schema نامعتبر ؛ مثال typed archive_to.
`get_market_snapshot_summary_history`	snapshot atomic؛ مشتق از summary همه `flow=None`.	`DataFrame[flow,instrument_count,trade_count,total_volume,total_value,market_cap,trade_date,exchange_time,fetched_at,is_realtime_fresh]` + meta.	file/filter/schema errors؛ مثال SQLite.
`get_market_breadth_history`	همان فیلترهای live روی هر snapshot persisted.	`BREADTH_COLUMNS` + meta؛ بدون snapshot-by-snapshot، look-ahead.	file/filter/schema errors؛ مثال breadth history.
`get_sector_flow_history`	همان فیلترهای client، live و prices همان observation.	`SECTOR_FLOW_COLUMNS` + meta/no-lookahead.	file/filter/schema errors؛ مثال sector history.
`record_market_event`	`event: MarketEvent`؛ `include_snapshot`؛ retention/session.	`event_id`؛ SHA-256، `str` در notification/snapshot و event transaction.	type/kind/storage `InvalidParameterError/DataParsingError`؛ مثال watcher. `record_to`
`get_market_event_history`	`kind=None|initial|delta|heartbeat|resync`؛ session/pagination.	`DataFrame[event_id,SessionID,kind,SnapshotMode,sequence,fetched_at,trade_date,changed_inscodes,changed_order_levels,market_state_changed,notification_tokens,cursor_before,cursor_after,retry_count,retry_error,notification_errors,snapshot,messages,state_changes]` + meta.	ناشناخته kind `InvalidParameterError` schema حتی برای فایل گم‌شده؛ replay؛ مثال errors
`archive_market_records`	یکی از `kind` `messages,state_changes,overview`؛ `frame: DataFrame`؛ هویت `recorded_at/source` archive.	`int` تعداد versionهای تازهٔ، صفر true duplicate نوشته‌شده؛ transaction atomic.	kind/type/storage errors؛ مثال helperهای `archive_to`.
`get_market_messages_history`	`flow/since_id/source` و time/page filters.	schema + archive meta؛ پیام dedup provider ID.	file/schema/filter errors؛ مثال archive.
`get_instrument_state_changes_history`	قدیمی spelling و/یا `symbol` `since_id/source`؛ `inscode`.	schema state + archive meta.	selector/متناقض/فایل schema؛ مثال errors archive.

قرارداد API: درآمد ثابت

```
parse_treasury_maturity(symbol)
day_count_fraction(start, end, convention='ACT/365F')
treasury_yield(price, maturity_date, settlement_date=None, face_value=1000000.0, day_count='ACT/365F')
bond_price(annual_yield, cashflows=None, settlement_date=None, day_count='ACT/365F',
compounding='nominal', frequency=1, price_type='dirty', accrued_interest=None, maturity_date=None,
face_value=None, coupon_rate=None, issue_date=None)
yield_to_maturity(price, cashflows=None, settlement_date=None, day_count='ACT/365F',
compounding='nominal', frequency=1, price_type='dirty', accrued_interest=None, maturity_date=None,
face_value=None, coupon_rate=None, issue_date=None, tolerance=1e-12, max_iterations=300)
bond_analytics(price, cashflows=None, settlement_date=None, day_count='ACT/365F',
compounding='nominal', frequency=1, price_type='dirty', accrued_interest=None, maturity_date=None,
face_value=None, coupon_rate=None, issue_date=None, annual_yield=None, bump_size=0.0001)
build_yield_curve(nodes, settlement_date, interpolation='log_discount', extrapolate=False,
duplicate_policy='error', day_count='ACT/365F', rate_compounding='effective', rate_frequency=1,
enforce_monotonic_discount=True)
get_ifb_yield_table(category='treasury')
get_debt_yields(debt_type=None)
get_treasury_yields(symbol=None, settlement_date=None, face_value=1000000.0, include_stale=False,
min_volume=0, price_source='auto', day_count='ACT/365F', strict=False, face_value_source=None,
source='tsetmc', maturity_date=None, maturity_map=None, allow_no_trade=False)
get_treasury_yield_history(symbol=None, start=None, end=None, limit=0, settlement_date=None,
face_value=1000000.0, include_today=False, date_format='jalali', price_source='auto',
day_count='ACT/365F', ascending=True, progress=True, strict=False, face_value_source=None,
max_requests=250, maturity_date=None, maturity_map=None, use_ifb_reference=False)
get_treasury_yields_history(symbol=None, start=None, end=None, limit=0, settlement_date=None,
face_value=1000000.0, include_today=False, date_format='jalali', price_source='auto',
day_count='ACT/365F', ascending=True, progress=True, strict=False, face_value_source=None,
max_requests=250, maturity_date=None, maturity_map=None, use_ifb_reference=False)
get_yield_curve(symbol=None, settlement_date=None, face_value=1000000.0, include_stale=False,
min_volume=0, min_nodes=3, price_source='auto', day_count='ACT/365F', interpolation='log_discount',
extrapolate=False, duplicate_policy='volume_weighted', enforce_monotonic_discount=True,
source='tsetmc')
get_yield_curve_history(symbol=None, start=None, end=None, limit=0, face_value=1000000.0,
include_today=False, date_format='jalali', price_source='auto', day_count='ACT/365F', min_nodes=3,
interpolation='log_discount', duplicate_policy='volume_weighted', enforce_monotonic_discount=True,
ascending=True, progress=True, max_requests=250, maturity_map=None)
```

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
`parse_treasury_maturity`	`symbol: Any` ; فقط full-match پس از تبدیل رقم `YYMMDD|اخزا` ؛ فارسی/عربی و حذف ZWNJ و `00..79→1400..1479` و `80..99→1380..1399`.	`dict|None` ; موفق: `maturity_jalali: str` , `maturity_gregorian: datetime.date` , `maturity_source='user_confirmed_symbol_jalali_yymmdd'` .	**None** ، تاریخ نامعتبر/non-match deterministic ؛ مثال exception نه بالاتر.

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
`day_count_fraction`	`start/end: date-like`؛ `convention`.	`float` year fraction.	نامعتبر date order/convention؛ `ValueError`؛ `day_count_fraction(date(2025,1,1), date(2026,1,1)) == 1.0`.
`treasury_yield`	zero-coupon price/face/maturity/settlement.	شامل `dict` `DiscountFactor,EffectiveAnnualYield,ContinuousYield,SimpleAnnualYield,BankDiscountYield,MacaulayDuration,ModifiedDuration,Convexity,DV01,DaysToMaturity/Tenor`.	قیمت face/date/day-count نامعتبر؛ مثال `ValueError`؛ deterministic.
`bond_price`	یا `cashflows` صریح، یا پارامترهای ساخت schedule؛ `annual_yield`؛ `price_type`.	`float` clean یا dirty price.	cashflow/rate/frequency/date نامعتبر؛ مثال `ValueError`؛ `bond_price(0.2, [(date(...), amount)], ...)`.
`yield_to_maturity`	قیمت مشاهده‌شده؛ `price` صریح یا `cashflows` برای `settlement_date/maturity_date/face_value/coupon_rate/frequency/issue_date` schedule؛ `accrued_interest/price_type` clean/dirty؛ `compounding`؛ `tolerance/max_iterations` solver.	با `float` annual yield compounding انتخابی.	bracket عدم/bounds خارج price؛ `ValueError` یا عدم همگرایی مثال round-trip ثابت درآمد فصل.
`bond_analytics`	ورودی schedule مانند `yield_to_maturity`؛ `annual_yield=None` یعنی حل YTM از `price`؛ `day_count/compounding/price_type` convention؛ `bump_size=0.0001` شوک DV01.	`dict` با price/yield، `MacaulayDuration,ModifiedDuration,Convexity,DV01` و metadata schedule.	نامعتبر cashflow/rate/date/price؛ `ValueError` یا ناموفق solver مثال deterministic.
`build_yield_curve`	node‌های rate/discount، compounding، duplicate و monotonic guard.	`YieldCurve`؛ و `node_metadata` با کلیدهای دقیق `diagnostics` `input_node_count,node_count,duplicate_count,duplicate_policy,monotonic_discount_enforced`.	node کم/duplicate/discount غیرمثبت/non-monotonic curve. مثال `ValueError`؛

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
`get_ifb_yield_table`	`category: str='treasury'` دستهٔ جدول صفحهٔ IFB.	`DataFrame[Symbol,Price,LastTradeJalali,LastTradeDate,PublishJalali,PublishDate,MaturityJalali,Maturity,Volume,ReferenceYTM,ReferenceSimpleYield,ReferenceSource]` ; attrs provenance URL/time.	category/HTML/schema/connection typed ؛ مثال comparison IFB.
`get_debt_yields`	str/sequence یا `debt_type=None` ؛ از `treasury,erad,gam,other` و طبقه‌بندی IFB رسمی نمای `all` بر پیشوند نماد exact.	schema IFB + ؛ `DebtType` attrs `cashflows_inferred=False` , `classification='exact_symbol_prefix'` .	فیلتر `InvalidParameterError` و HTTP/HTML `ConnectionError/DataParsingError` ؛ `att.get_debt_yields(["erad", "gam"])` .
`get_treasury_yields`	live universe؛ `min_volume` ; maturity `face_value_source` ; override/map؛ `allow_no_trade` ; source.	هویت/ : `TREASURY_COLUMNS` ، price provenance/سررسید/تسویه ، چهار yield، duration/convexity/DV01، stale/status؛ attrs source/universe/fetch time.	`InvalidParameterError` , `StockNotFoundError/AmbiguousSymbolError` , `ConnectionError/DataParsingError` ؛ اخزا live مثال .
`get_treasury_yield_history`	history OHLC؛ `max_requests` hard؛ `use_ifb_reference` فقط مگر reference؛ settlement per row override.	با `TREASURY_HISTORY_COLUMNS` `TradeDate,JalaliDate,Open,High,Low,Close,Final,Volume...` و analytics؛ attrs failures/request/provenance.	parameter/resolver/provider/parsing؛ مثال attrs/warning در partial ؛ history.
`get_treasury_yields_history`	alias identity با signature کامل یکسان.	دقیقاً همان object/schema/attrs `get_treasury_yield_history` .	همان؛ `att.get_treasury_yields_history is att.get_treasury_yield_history` .
`get_yield_curve`	`min_nodes` ; duplicate حداقل ؛default volume-weighted ؛extrapolation opt-in.	از calibrated `YieldCurve` و snapshot live؛ diagnostics و metadata nodeها را ثبت provenance/no-lookahead می‌کند.	node ناکافی/invalid curve و provider typed؛ `ValueError` مثال curve live.
`get_yield_curve_history`	trade date؛ جدا برای هر curve ؛ بدون hard budget , `maturity_map` عمومی extrapolate.	با panel `DataFrame` , `CURVE_HISTORY_COLUMNS` `CurveID,CurveStatus,CurveError,CurveNodeCount` و flags no-lookahead؛ attrs failures/request.	parameter/budget/provider/curve errors در ناموفق ؛ روز status/attrs؛ مثال curve history.

قرارداد API: شاخص‌های صنعت

```
list_industry_indices(progress=True, include_member_count=False, refresh=False, max_workers=6)
get_industry_members(industry, include_live=True, include_client_type=False, include_orderbook=False,
progress=True, refresh=False)
get_industry_snapshot(industries=None, include_client_type=True, include_orderbook=False,
include_empty=False, progress=True, refresh=False, max_workers=6)
get_industry_history(industry, start=None, end=None, limit=0, ascending=True, progress=True)
get_industry_members_history(industry, days=30, ascending=True, progress=True, refresh=False)
compare_industries(industries, start=None, end=None, limit=0, metric="close", ascending=True,
progress=True, max_workers=6)
get_industry_relative_strength(industries, benchmark, start=None, end=None, limit=0, metric="close",
ascending=True, progress=True, max_workers=6)
get_industry_correlation(industries, start=None, end=None, limit=0, ascending=True, progress=True,
max_workers=6)
get_industry_membership_overlap(industries, progress=True, refresh=False, max_workers=6)
get_industry_membership_churn(industries, days=0, progress=True, refresh=False, max_workers=6)
get_industry_membership_events(industries, days=30, progress=True, refresh=False, max_workers=6)
get_industry_concentration(industries, top_n=10, include_weights_by_market_value=True, progress=True,
refresh=False, max_workers=6)
get_industry_momentum_profile(industries, windows=(5, 20, 60), start=None, end=None, progress=True,
max_workers=6)
get_industry_correlation_neighborhood(industry, industries=None, top=5, min_correlation=0.0,
by_absolute=False, start=None, end=None, limit=0, ascending=False, progress=True, max_workers=6)
get_industry_health_score(industries=None, start=None, end=None, top_concentration=10,
momentum_windows=(20, 60), progress=True, refresh=False, max_workers=6)
get_industry_intraday(industry, interval='1min', progress=True)
rank_industries(metric='IndexChangePct', top=None, ascending=False, include_client_type=True,
include_orderbook=False, progress=True, refresh=False, max_workers=6)
```

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
`list_industry_indices`	پیام؛ `progress: bool=True` عضویت ۴۵ صنعت را `include_member_count: bool=False` واکشی می‌کند؛ `refresh: bool=False` cache را دور می‌زند؛ `max_workers: int=6` در بازهٔ `1..16`.	`DataFrame[IndustryName,IndustryNameEn,IndustryGroupCode,IndexInsCode,IndexValue,IndexPreviousValue,DayHigh,DayLow,IndexChange,IndexChangePct,MemberCount,HasMembers,ExchangeTime]` attrs source/cache/count.	نامعتبر bool/worker ؛ `InvalidParameterError` provider `ConnectionError/DataParsingError`؛ مثال فصل صنعت.
`get_industry_members`	`industry: str|int` نام/alias/ کد؛ `include_live=True` MarketWatch؛ `include_client_type=False` client metrics؛ `include_orderbook=False` پنج دو . `refresh=False` سطح/صف؛ نیاز دارند live به enrichment گزینهٔ.	`INDUSTRY_MEMBER_COLUMNS` ثابت با هویت، قیمت، معامله، market cap تخمینی، client، order book و freshness؛ attrs exact membership/current/cache/coverage request.	صنعت گم‌شده `StockNotFoundError` ؛ ترکیب/نوع نامعتبر `InvalidParameterError` ؛ provider typed؛ مثال `get_industry_members("نامک")`.

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
`get_industry_snapshot`	`industries: None|str|int|iterable=None`؛ `include_client_type=True` ؛ `include_orderbook=False` ؛ `include_empty=False` ؛ `refresh=False` ؛ `max_workers=6` .	یک ردیف در هر شاخص با `INDUSTRY_SNAPSHOT_COLUMNS`: وضعیت شاخص، breadth، equal-weight/median/dispersion، معامله، flow/coverage، queue/coverage و freshness؛ attrs overlap/current membership/cache.	selector/bool/worker یا `InvalidParameterError` ؛ `StockNotFoundError` ؛ snapshot/schema/provider typed؛ مثال فصل صنعت.
`get_industry_history`	`industry` اجباری؛ `start/end: str|None` شمسی/میلادی؛ `limit: int=0` پس از فیلتر؛ `ascending: bool=True` ; `progress` .	`DataFrame[IndustryName,IndustryIndexCode,TradeDate,JalaliDate,High,Low,Close,Change,ChangePct]` ; attrs `volume_available=False,completed_sessions_only=True` .	تاریخ/order/limit نامعتبر `InvalidParameterError` ؛ صنعت/provider/schema typed؛ مثال تاریخچه.
`get_industry_members_history`	`industry` ، `days: int=30` تاریخ آخر؛ `1..30` دقیقاً `ascending=True` ; `refresh=False` .	long-form `INDUSTRY_MEMBER_HISTORY_COLUMNS` attrs برای اعضای فعلی؛ `point_in_time_membership=False,survivorship_bias_possible=True` .	days/bool/selector typed؛ مثال provider/schema typed ؛ فصل صنعت.
`compare_industries`	`industries` (str iterable)، `start/end: str|None` ، `limit: int=0` ، `metric: close|price|change_pct|log_return` ، `ascending=True` ، `progress=True` ، `max_workers=6` .	int	
`get_industry_relative_strength`	`industries` ، `benchmark` ، `start/end: str|None` ، `limit: int=0` ، `metric: close|price|change_pct|log_return` ، `ascending=True` ، `progress=True` ، `max_workers=6` .	long-form `TradeDate,JalaliDate,BenchmarkIndexCode,BenchmarkName,IndustryIndexCode,IndustryName,IndustryReturn,BenchmarkReturn,RelativeStrength` ; `attrs` شامل `analysis='industry_relative_strength'` ، `benchmark_index_code` ، `industry_count` .	benchmark/generic نامعتبر یا دادهٔ metrics/worker ناکافی `InvalidParameterError` ; مثال `relative_strength = ...` .
`get_industry_correlation`	`industries` (str iterable)، `start/end: str|None` ، `limit: int=0` ، `ascending=True` ، `progress=True` ، `max_workers=6` .	int	

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
`get_industry_membership_overlap`	`industries` (str	int	iterable), `progress=True`, `refresh=False`, `max_workers=6`.
`get_industry_membership_events`	`industries` (str	int	iterable), `days: int=1..30`, `progress=True`, `refresh=False`, `max_workers=6`.
`get_industry_membership_churn`	`industries` (str	int	iterable), `days: int=1..30`, `progress=True`, `refresh=False`, `max_workers=6`.
`get_industry_concentration`	`industries` (str	int	iterable), `top_n: int=10`, `include_weights_by_market_value`=True, `progress=True`, `refresh=False`, `max_workers=6`.
`get_industry_momentum_profile`	`industries` (str	int	iterable), `windows=(5,20,60)`, `start/end: str|None`, `progress=True`, `max_workers=6`.
`get_industry_correlation_neighborhood`	`industry` هدف (str	int), `industries`، `top=5`, `min_correlation=0.0`, `by_absolute=False`, `start/end: str|None`, `limit=0`, `ascending=False`, `progress=True`, `max_workers=6`.	`DataFrame[IndustryName, IndustryIndexCode,NeighborName,NeighborIndexCode,Correlation,CorrelationAbs,CorrelationRank]`؛ شامل `attrs` `analysis='industry_correlation_neighborhood'`, `target_industry_code`, `target_industry_name`, `top`, `min_correlation`, `by_absolute`, `start_date`, `end_date`.
`get_industry_health_score`	`industries=None` (str	int	iterable)، `start/end: str|None`, `top_concentration=10`, `momentum_windows=(20,60)`, `progress=True`, `refresh=False`, `max_workers=6`.

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
`get_industry_intraday`	`industry` ؛ `interval: str='1min'` یکی از `raw,1min,5min,15min,30min,60min,1h` ; `progress` .	`DataFrame[IndustryName,IndustryIndexCode,Timestamp,JalaliDate,Interval,Open,High,Low,Close,Change,ChangePct]` ; timezone؛ تهران مصنوعی و volume بدون attrs latest-day.	interval/selector `InvalidParameterError/StockNotFoundError`؛ مثال provider/schema typed؛ `get_industry_intraday("خودرو", "5min")` .
`rank_industries`	`metric` مستند؛ canonical/alias ; `top: int|None` ; `ascending=False` ; client/order switches؛ `refresh` ; `max_workers` .	تمام ستون‌های `Rank` + snapshot؛ ردیف metric تهی حذف؛ attrs `ranking_metric,ranking_ascending` و provenance snapshot.	نامعتبر metric/top/bool/worker `InvalidParameterError` ؛ مثال provider typed؛ `rank_industries(metric="breadth", top=5)` .

قرارداد API: اختیار معامله و فهرست ابزارها

```
list_options(underlying=None, progress=True)
get_options_chain(underlying, fetch_oi=False, progress=True)
black_scholes_price(spot, strike, time_to_expiry, rate, volatility, option_type='call',
dividend_yield=0.0, exercise_style='european')
black_scholes_greeks(spot, strike, time_to_expiry, rate, volatility, option_type='call',
dividend_yield=0.0, exercise_style='european')
option_price_bounds(spot, strike, time_to_expiry, rate, option_type='call', dividend_yield=0.0,
exercise_style='european')
implied_volatility(option_price, spot, strike, time_to_expiry, rate, option_type='call',
dividend_yield=0.0, exercise_style='european', lower_volatility=0.0, upper_volatility=5.0,
tolerance=1e-08, max_iterations=200)
get_option_market(exchange=0, underlying=None, progress=True, max_requests=1)
analyze_option_chain(options=None, underlying=None, spot=None, risk_free_rate=None, yield_curve=None,
dividend_yield=0.0, valuation_date=None, exercise_style='european', parity_tolerance=None,
liquidity_weights=None, progress=True, *, allow_unverified_freshness=True)
option_put_call_ratios(options, group_by='market')
get_option_history(symbol, start=None, end=None, limit=0, include_today=False, snapshot_path=None,
progress=True, max_requests=3)
save_option_snapshot(path, options=None, exchange=0, progress=True, *, lock_timeout=10.0,
stale_lock_seconds=300.0)
load_option_snapshots(path)
list_etfs(progress=True)
list_bonds(progress=True)
list_debt_instruments(debt_type=None, active_only=False, progress=True)
list_funds(fund_type=None, progress=True, *, listed_only=False, strategy=None,
commodity_underlying=None, classification_status=None)
list_listed_funds(progress=True, *, fund_category=None, strategy=None, commodity_underlying=None,
classification_status=None, include_unknown=True)
list_indices(progress=True)
get_index_companies(index_name, progress=True)
```

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
`list_options`	`underlying: str|None` filter نام underlying.	`DataFrame` قراردادها با هویت، `OptionType,Underlying*,Strike,BeginDate,EndDate,DaysToExpiry,ContractSize` و قیمت/حجم.	provider parse/connection یا تهی؛ مثال فصل اختیار frame.
`get_options_chain`	`underlying` اجباری؛ `fetch_oi=False` از fan-out OI جلوگیری می‌کند.	`calls: DataFrame` دقیقاً شامل `dict` , `puts: DataFrame` , `underlying_name` , `underlying_price` , `expiry_dates` , `fetch_oi` با `market_time` ; ستون‌های `OpenInterest,ContractSize,BeginDate,EndDate` .	underlying/provider errors؛ مثال فصل اختیار chain.
`black_scholes_price`	فقط European؛ scalar math params.	`float` premium.	نامعتبر bounds/type/style مثال ; `ValueError` deterministic.
`black_scholes_greeks`	ورودی‌های `spot/strike/time_to_expiry/rate/volatility` عددی؛ `option_type∈{call,put}` ؛ نرخ پیوسته؛ `dividend_yield` `exercise_style='european'` تنها سبک پشتیبانی‌شده.	`dict[Delta,Gamma,Vega,Vega1Pct,ThetaPerYear,ThetaPerDay,Rho,Rho100bp,Status]` ; `Status='ok'` یا `undefined_at_expiry_or_zero_volatility` .	نامعتبر constraint مثال ; `ValueError` Greeks.
`option_price_bounds`	بدون volatility؛ no-arbitrage bound.	tuple `(lower: float, upper: float)` .	مثال ; `ValueError` با خروجی deterministic `(4.87705755, 100.0)` .
`implied_volatility`	premium + bracket/tolerance/iterations؛ `tolerance>0` , عدد صحیح مثبت `max_iterations` و `0<=lower<upper` .	`dict` با `ImpliedVolatility,Status,Iterations` ; statusهای `ok/missing/expiry/out_of_bounds/no_bracket/non_converged` و کلیدهای تشخیصی اختیاری.	premium ناموجود/خارج bounds و هستند، نه status عدم همگرایی ؛ فقط exception نامعتبر constraint/style/bracket مثال ; `ValueError` IV.

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
`get_option_market`	`exchange: int=0` ; underlying filter؛ `max_requests=1` bulk.	`DataFrame[InsCode,PairID,PairSequence,ISIN,Symbol,Name,OptionType,UnderlyingInsCode,UnderlyingSymbol,UnderlyingName,ContractSize,Strike,BeginDate,EndDate,DaysToExpiry,Last,Close,Yesterday,Volume,Value,TradeCount,NotionalValue,OpenInterest,YesterdayOpenInterest,BidPrice,AskPrice,BidVolume,AskVolume,UnderlyingLast,UnderlyingClose,Price,PriceSource,AsOf,AsOfSource,SnapshotFreshnessKnown,PriceFreshnessKnown,Stale,NoTrade,AnalyticsEligible,AnalyticsEligibilityReason,MetadataConflict,Source]` ; attrs snapshot provenance.	exchange/budget/filter `InvalidParameterError` ؛ مثال provider typed ؛ snapshot.
`analyze_option_chain`	`options=None` fetch می‌کند؛ spot/rate/curve overrides؛ `allow_unverified_freshness` explicit risk switch.	input columns + `TimeToExpiry,Spot,RiskFreeRate,DividendYield,ImpliedVolatility*` , Greeks per unit/contract، spread/depth/liquidity، `ParityResidual,ParityStatus,ImpliedForward,AnalyticsReliability` ; attrs assumptions/warnings.	input type `TypeError` ; math/freshness/rate/column problems `ValueError` ؛ مثال provider errors if fetch ؛ حرفه‌ای.
`option_put_call_ratios`	از یک `options: DataFrame` دقیقاً یکی `group_by` اتمیک؛ `AsOf` از `market,underlying,expiry,underlying_expiry` .	call/put با `DataFrame` volume/value/OI، `PCRVolume,PCRValue,PCROpenInterest` و status های دقیق `PCRVolumeStatus,PCRValueStatus,PCROpenInterestStatus` ; attrs coverage/source ورودی.	ستون/group/AsOf یا قرارداد تکراری نامعتبر `ValueError` و نوع غیر `TypeError` ;DataFrame مثال PCR.

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
`get_option_history`	قرارداد دقیق؛ + server history ; `include_today` فقط join `snapshot_path` snapshotهای opt-in؛ budget.	`DataFrame[Timestamp,InsCode,Symbol,Open,High,Low,Close,Last,Volume,Value,TradeCount,OpenInterest,BidPrice,AskPrice,BidVolume,AskVolume,UnderlyingLast,UnderlyingClose,ContractSize,Strike,EndDate,Price,PriceSource,Source,AsOf,Stale,NoTrade,AnalyticsEligible]` ; attrs failures/no-lookahead.	selector/date/budget/provider/snapshot؛ مثال تاریخچه schema errors اختیار.
`save_option_snapshot`	`options=None` یک fetch؛ lock نامنفی و timeout مثبت stale lock.	`DataFrame` شده بادedupe ترکیب attrs ; `OPTION_COLUMNS` `schema_version,source,path` و lock با JSON versioned فایل . اتمیک نوشته می‌شود replace.	گم‌شده parent ، `FileNotFoundError` غیرDataFrame options مقدار نامعتبر ، `TypeError` قفل ، `ValueError` اگر provider `TimeoutError` ؛ snapshot. ؛ مثالfetch
`load_option_snapshots`	`path` JSON versioned.	با `DataFrame` ؛ فایلattrs و `OPTION_COLUMNS` تهی با frame گم‌شده **خطا نیست** و `attrs['status']='missing'` می‌دهد.	خراب JSON version و `JSONDecodeError` مثال ; `ValueError` ناسازگار history.
`list_etfs`	فقط `progress` .	`DataFrame[InsCode,ISIN,Symbol,Name,Last,Close,Yesterday,Volume,Value,TradeCount,Low,High,NAV,NAV_Discount,Change,ChangePct,MarketCode]` .	؛ مثالprovider typed/empty `list_etfs().query("NAV_Discount < -1")` .
`list_bonds`	فقط `progress` ؛ signature قدیمی ثابت است.	`DataFrame[InsCode,ISIN,Symbol,Name,BondType,Ticker,MaturityJalali,MaturityGregorian,DaysToMaturity,Last,Close,Yesterday,Volume,Value,TradeCount,Change,ChangePct]` ؛ `BondType='gam'` نیز پشتیبانی می‌شود.	نامعلوم maturity؛ parse/provider؛ nullable؛ `att.list_bonds(progress=False)` .
`list_debt_instruments`	`debt_type=None` یا str/sequence از `treasury,erad,gam,murabaha,ijara,salaf,other` ؛ `active_only=False` ؛ `progress` .	`list_bonds` + schema ، coverage ،منبع attrs ؛ `DebtType` و فیلترها.	فیلتر/bool و `InvalidParameterError` خطاهای provider؛ `att.list_debt_instruments("gam", active_only=True)` .

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
`list_funds`	از `fund_type: str|list|None` ؛ فیلترهای `listed_only=False` ، keyword-only: `strategy` ، `commodity_underlying` ، `classification_status` ؛ نام قدیمی `venture` alias است.	با registry rich funds NAV/returns/composition/manager؛ exact با FIPIRAN از `pct_cash` و بدون inference؛ `reg_no` تاریخ گزارش منبع؛ `record_date` شامل وضعیت/پوشش وجه نقد attrs و `pct_cash_inferred=False` ؛ InsCode بدون تضمین.	type/category/combo/filter نامعتبر یا `InvalidParameterError` ؛ خطای legacy رفتار سازگار را خالی `pct_cash` ،FIPIRAN TSETMC می‌کند و مقدار تکراری استفاده نمی‌شود؛ مثال فصل صندوق.
`list_listed_funds`	یک MarketWatch bulk؛ `fund_category` ، `strategy` ، `commodity_underlying` ، `classification_status` ؛ `include_unknown=True` .	شامل `LISTED_FUND_COLUMNS` و تمام `InstrumentType` ؛ هر دو نوع taxonomy۳۰۵ محورهای attrs و ۳۸۰؛ `no_fuzzy_join=True,registry_joined=False,taxonomy_version` .	فیلتر نامعتبر `InvalidParameterError` ؛ connection/parsing؛ مثال `list_listed_funds(commodity_underlying="gold")` .
`list_indices`	فقط `progress` .	`DataFrame[Name,InsCode,Value,High,Low,Change,ChangePct]` حرکت `Change` از 1.2.0؛ درصد `ChangePct` واحد شاخص و است.	مسیر legacy در خطای provider frame تهی/پیام؛ برای صنعت `list_industry_indices` پیشنهاد می‌شود.
`get_index_companies`	فارسی یا `index_name: str` InsCode.	`DataFrame[Symbol,Name,InsCode,Close,Yesterday,Last]` .	در مسیر provider/گم‌شده index تهی/پیام؛ مثال فصل legacy frame شاخص.

قرارداد API: تحلیل مدیریتی بازار

```
compare_symbols(symbols, start=None, end=None, benchmark=None, align='inner', annualization=240,
min_observations=20, include_client_type=True, progress=True, history_data=None, client_data=None,
strict=False)
get_liquidity_metrics(symbols=None, start=None, end=None, window=20, annualization=240,
include_live=True, progress=True, history_data=None, live_data=None, strict=False)
get_market_regime(benchmark='شاخص کل', start=None, end=None, lookback=60, liquidity_window=20,
annualization=240, weights=None, progress=True, max_requests=30, live_data=None,
benchmark_history=None, activity_history=None)
get_market_map(group_by='symbol', size='value', color='return', top=100, flow=None,
instrument_types=None, data=None)
plot_market_map(map_data=None, output_path=None, show=False, title=None)
```

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
`compare_symbols`	`symbols` ،الزامی؛ تاریخ benchmark، و حداقل annualization ، `align` تقویم و client type مشاهده؛ دریافت ورودی‌های صریح قابل کنترل‌اند.	۳۱ ستون شامل بازده/ریسک/افت، beta/correlation/alpha، نقدشوندگی و حقیقی؛ attrs مبنا، alignment، پوشش، نماد مفقود و واحد ریال.	نامعتبر enum/count/date ؛ `InvalidParameterError` یا malformed صریح history ناموفق `strict` ؛ نمونه `DataParsingError` فصل تحلیل مدیریتی.

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
`get_liquidity_metrics`	کل snapshot فقط `symbols=None` بازار؛ نماد مشخص تاریخچه را هم می‌گیرد؛ و ورودی live کل بازه؛ `window=None` صریح قابل کنترل.	۳۲ ستون؛ zero-trade/return، Amihud به میلیارد ریال، ارزش/حجم/ معامله، volatility، turnover، spread، L1/L5 و depth؛ attrs پوشش و واحد.	پارامتر/date نامعتبر `InvalidParameterError` ؛ history malformed/strict `DataParsingError` ؛ نمونه فصل نقدشوندگی.
`get_market_regime`	و پنجره‌ها؛ پنج وزن اختیاری؛ benchmark صریح DataFrame بودجهٔ درخواست؛ سه برای replay.	یک ردیف ۳۰ ستونی با label/score/confidence، هر پنج component و weight، اجزای خام و quality؛ attrs آستانه، روش و missing.	وزن/نوع/date/budget نامعتبر `InvalidParameterError` ؛ تاریخچهٔ benchmark malformed `DataParsingError` ؛ نمونه فصل رژیم.
`get_market_map`	چهار گروه‌بندی، چهار معیار اندازه، چهار معیار رنگ؛ `flow` ، `top` ، نوع ابزار و snapshot صریح.	۲۳ ستون renderer-independent شامل اندازه/رنگ، دو وزن universe/display، پوشش رنگ، freshness و ریال؛ attrs انتخاب‌ها و partial بودن.	enum/filter نامعتبر `InvalidParameterError` ؛ ستون لازم ناقص `DataParsingError` ؛ نمونه نقشه.
`plot_market_map`	HTML، اختیاری، مسیر `map_data` نمایش و عنوان.	`plotly.graph_objects.Fig` فقط با HTML `ure`؛ نوشتن `output_path` .	`ImportError` ؛ مفقود Plotly تهی/ناقص frame `DataParsingError` ؛ مثال فصل نقشه.

قرارداد API: بورس انرژی و بورس کالا

```
get_energy_auctions(status='upcoming', board='physical', top=100)
get_energy_auction(auction_id, include_trades=True, include_instrument_history=False)
get_energy_market_overview(market='all')
list_power_instruments(market='all', top=100)
list_energy_securities(kind='all', active=True, top=100, enrich_futures=True)
get_energy_future_contract(ins_code)
get_commodity_market(kind='all')
get_commodity_physical_history(start, end=None)
get_commodity_physical_summary(start, end=None, hall='all')
get_commodity_market_activity(start, end=None, market='all')
get_futures_curve(contracts, underlying=None, valuation_date=None, price='settlement', spot_price=None)
get_calendar_spreads(curve)
analyze_cash_and_carry(curve, annual_rate, storage_rate=0.0, convenience_yield=0.0)
```

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
`get_energy_auctions`	`status` ، `board` ، fan-out `all` `top=1..1000` ؛ صریح.	`DataFrame` کامل عرضه با قیمت، lot، حجم، شرایط پرداخت/تحویل منبع/ attrs وضعیت و معامله؛ درخواست.	enum/top نامعتبر `InvalidParameterError` ؛ مثال فصلschema typed/شبکه ؛ انرژی.
`get_energy_auction`	مثبت؛ `auction_id` `include_trades=True` ؛ تاریخچهٔ opt-in ابزار.	`dict` با `auction` و کلیدهای `TradeScope` معاملهٔ درخواستی؛ تفاوت دامنه را ثبت می‌کند.	نامعتبر یا پاسخ ناقص ID/bool typed؛ مثال single auction.

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
`get_energy_market_overview`	`market` یکی از هشت flow label . `all` مستند یا	`DataFrame[Market,Flow,GregorianDate,JalaliDate,Time,TradeCount,Volume,Value,ValueUnit,StateCode,State,Source]` .	market/provider/schema typed؛ مثال overview.
`list_power_instruments`	`market∈{standard,green,free,all}` `top=1..1000` .	schema مشترک ابزار انرژی، `ValueUnit='rial'` .	enum/top/provider typed؛ مثال برق سبز.
`list_energy_securities`	`kind` ؛/پنج دسته `all` `active` ; `top` ; `enrich_futures` .	enrichment مشترک بازار؛ schema attrs سررسید/اندازه/تسویه؛ `request_count` .	enum/bool/top/provider/detail typed؛. مثال اوراق انرژی
`get_energy_future_contract`	`ins_code` .دقیق	یک ردیف مشخصات، تاریخ‌ها، قیمت‌ها، fee، position، margin و delivery rules و limits.	InsCode/provider/schema typed؛. مثال فصل آتی
`get_commodity_market`	`kind∈{certificate,standard_salaf,fund,future,all}` و aliasهای مستند.	هویت/قیمت/ `DataFrame` پنج سطح/client type/معامله سفارش و مشتقه؛ `ValueUnit='thousand_rial'` .	kind/provider/schema typed؛ معتبر است frame بازار خالی.
`get_commodity_physical_history`	یعنی `end=None` ،الزامی `start` .امروز تهران	روزانه `GregorianDate,JalaliDate,Volume,Value,ValueUnit,Source`؛ واحد میلیون ریال.	date order/provider/embedded JSON typed؛. مثال تاریخچه
`get_commodity_physical_summary`	بازه مانند بالا؛ `hall='all'` یا ID/نام exact.	`HallID,Hall,Volume,OfferVolume,TradeToOfferRatio,Value` .و بازه؛ واحد میلیون ریال	تاریخ/provider typed؛ no-match frame خالی معتبر.
`get_commodity_market_activity`	بازه؛ `market='all'` یا ID/نام exact.	فعالیت تجمیعی فیزیکی/آتی/اختیار/مالی/گواهی؛ جزئیات قرارداد نیست.	تاریخ/provider typed؛ مثال market ID 3 برای اختیار.
`get_futures_curve`	underlying؛ صریح؛ فیلتر contracts اختیاری spot؛ price ؛ اولویتvaluation مثبت.	term structure با tenor، basis مرتب provenance مطلق/درصدی/سالانه و قیمت.	ستون/date/price/spot نامعتبر `InvalidParameterError` ؛ مثال curve.
`get_calendar_spreads`	دارای `curve` `ExpiryDate,Price` .	جفت سررسید مجاور، spread، annualized spread و structure.	نامعتبر schema/date `InvalidParameterError` ؛ مثال spread.
`analyze_cash_and_carry`	؛ نرخ‌هایspot/price/tenor با curve مثبت base و finite.	ورودی + `FairValue,Mispricing,MispricingPct,Signal` و فرض‌ها در attrs.	نامعتبر schema/rate `InvalidParameterError` ؛ توصیهٔ معامله نیست signal.

قرارداد aliasها و wrapperهای backward-compatible

قدیمی و تفاوت رفتاری مخفی نماند. تمام پارامترها signature های این جدول مدخل مستقل دارند تاaliasتابع target جدول type/default/constraint فقط تفاوت صریح را دارند؛ override است.

```
stock(symbol='', start=None, end=None, limit=0, raw=False, auto_adjust=True, output_type='standard',
date_format='jalali', progress=True, save_to_file=False, dropna=True, adjust_volume=False,
return_type=None, ascending=True, save_path=None, include_today=False, *, ins_code=None,
asset_type='auto', **kwargs)
stock_RI(symbol='', start=None, end=None, limit=0, raw=False, output_type='standard',
date_format='jalali', progress=True, save_to_file=False, dropna=True, ascending=True, save_path=None,
include_today=False, *, ins_code=None, asset_type='auto', **kwargs)
stock_RL(symbol='', start=None, end=None, limit=0, raw=False, output_type='standard',
date_format='jalali', progress=True, save_to_file=False, dropna=True, ascending=True, save_path=None,
include_today=False, *, ins_code=None, asset_type='auto', **kwargs)
stock_capital_increase(symbol='', *, ins_code=None, asset_type='auto', **kwargs)
stock_intraday(symbol='شتران', interval='1min', start=None, end=None, progress=True, **kwargs)
stockdetail(symbol='', *, ins_code=None, asset_type='auto', **kwargs)
stock_information(symbol='', *, ins_code=None, asset_type='auto', **kwargs)
stock_statistics(symbol='', *, ins_code=None, asset_type='auto', **kwargs)
stock_introduction(symbol='', *, ins_code=None, asset_type='auto', **kwargs)
stocklist(bourse=True, farabourse=True, payeh=True, haghe_taqadom=False, sandogh=False, bonds=False,
options=False, mortgage=False, commodity=False, energy=False, payeh_color=None, output='dataframe',
progress=True, **kwargs)
shareholders(symbol='', date=None, include_id=False, *, ins_code=None, asset_type='auto', **kwargs)
currency_coin(name='', start=None, end=None, limit=0, output_type='standard', date_format='jalali',
progress=True, save_to_file=False, dropna=True, return_type=None, ascending=True, save_path=None,
**kwargs)
market_watch()
market_client_type()
market_data()
```

alias/wrapper	و تفاوت target	خروجی/schema/attrs	خطا و مثال
`stock`	همان ; target `get_history` و implementation history signature.	همان DataFrame/attrs.	همان خطاها؛ `att.stock("فملی", limit=10)`.
`stock_RI`	target `get_client_type` .	همان DataFrame/attrs.	همان؛ `att.stock_RI("فملی", limit=10)`.
`stock_RL`	که به wrapper compatibility `stock_RI` delegate می‌کند؛ identity برابر است signature نیست ولی alias.	همان DataFrame/attrs.	همان؛ برای کد جدید `get_client_type` .
`stock_capital_increase`	target canonical `get_capital_increase` .	همان frame/None قدیمی.	همان.
`stock_intraday`	target `get_intraday` ; signature برابر.	همان tick/candle frame.	همان.
`stockdetail`	target `get_detail` .	همان key/value frame یا `None` .	همان.
`stock_information`	target `get_info` .	همان key/value frame یا `None` .	همان.
`stock_statistics`	target `get_stats` .	همان key/value frame یا `None` .	همان.

alias/wrapper	و تفاوت target	خروجی/schema/attrs	خطا و مثال
`stock_introduction`	target `get_introduction`؛ wrapper legacy unsupported.	هیچ خروجی موفق.	همیشه `UnsupportedDataSourceError` پیش از I/O.
`stocklist`	همان؛ target `get_symbols` و signature و aliasهای kwargs.	همان فهرست.	همان.
`shareholders`	target `get_shareholders`.	همان frame/None.	همان.
`currency_coin`	`get_currency`؛ اکنون سازگار target را می‌پذیرد TGJU همهٔ ۸۷ سری کاتالوگ.	همان frame/MultiIndex و attrs دقیق source/category/unit.	نام typed/HTTP/schema؛ برای کد جدید `get_tgju_history`.
`market_watch`	تابع صفرآرگومان canonical قدیمی snapshot؛ target مفهومی `get_market_snapshot`.	`dict[stocks,order_book,market_time,...]` ستون‌های؛ legacy `Yesterday/BaseVolume/Change` حفظ شده و ستون‌های corrected additive هستند.	`ConnectionError/DataParsingError`؛ `snapshot=att.market_watch()`.
`market_client_type`	تابع صفرآرگومان bulk؛ target مفهومی `get_market_client_type`.	`CLIENT_COLUMNS` DataFrame.	typed provider errors.
`market_data`	صفرآرگومان به wrapper deprecated؛ identity alias `market_watch` نیست.	همان dict.	همان؛ برای کد جدید `get_market_snapshot/get_live_market`.

دو alias identity غیرlegacy نیز قبلاً مدخل دارند: `get_orderbook_history is get_order_book_history` و `get_treasury_yields_history is get_treasury_yield_history`. تفاوت signature یا خروجی ندارند.

signature constructor exceptionها نیز دقیقاً چنین است:

```
AlgotikTSEError(*args)
AmbiguousSymbolError(*args)
ConnectionError(*args)
DataParsingError(*args)
InvalidParameterError(*args)
StockNotFoundError(*args)
UnsupportedDataSourceError(*args)
```

الگوهای کاربردی

فیلتر قدرت خریدار حقیقی با نقدشوندگی

```
live = att.get_live_market()
screen = live.loc[
    (live["IndividualPower"] > 1.5)
    & (live["Value"] > 50_000_000_000)
    & live["is_realtime_fresh"].fillna(False)
].sort_values("EstimatedNetIndividualFlow", ascending=False)

print(screen[[
    "Symbol", "Last", "ChangePct", "IndividualPower",
    "EstimatedNetIndividualFlow", "SpreadBps", "L5Imbalance",
]])
```

backtest بدون look-ahead

```
db = "research.sqlite"
att.save_market_snapshot(db)

fund = att.get_market_fundamentals_history(db, symbols="فملی")
assert fund.attrs["no_lookahead"] is True
assert fund.attrs["current_eps_used"] is False

curves = att.get_yield_curve_history(
    start="1403-01-01", end="1403-03-31", max_requests=100, progress=False
)
safe_curves = curves.loc[curves["CurveNoLookahead"].fillna(False)]
```

مانیتور بازار و archive مستقل

```
db = "monitor.sqlite"

for event in att.watch_market(
    interval=3,
    max_updates=20,
    record_to=db,
    notifications=("messages", "state"),
):
    if event.kind in {"initial", "delta"}:
        print(event.sequence, len(event.changed_inscodes))

# archive_to باید روی helper مستقل فعال شود.
att.get_market_messages(flow=0, top=50, archive_to=db)
messages = att.get_market_messages_history(db, flow=0)
```

منابع داده

منبع	استفاده
TSETMC (`tsetmc.com` و subdomainهای رسمی)	قیمت، client type، market watch، سفارش، trades، پیام، وضعیت، ابزار، صندوق و اطلاعات بازار
FIPIRAN (`fipiran.com/services`)	مقدار تفکیک‌شدهٔ وجه نقد صندوق‌ها در `list_funds()` با اتصال دقیق شمارهٔ ثبت؛ بدون fuzzy join یا inference
بورس انرژی / TSETMC	حراج‌ها، معاملات همان عرضه، تابلوهای برق و انرژی و مشخصات آتی انرژی
بورس کالای ایران (`ime.co.ir` و `cdn.ime.co.ir`)	تابلوی گواهی/سلف/صندوق/آتی، بازار فیزیکی و آمار تجمیعی بازارها
فرابورس ایران (`ifb.ir/ytm.aspx`)	جدول مرجع YTM برای مقایسه/دسته‌بندی اوراق؛ منبع مجاز و با provenance جدا
TGJU (`api.tgju.org`)	تاریخچهٔ ۸۷ سری تأییدشدهٔ ارز آزاد/رسمی، طلا، نقره، سکه/حباب و فلزات گران‌بهای جهانی؛ نام و واحد در کاتالوگ عمومی

کدال منبع این پکیج نیست؛ حتی endpointهای proxyشدهٔ آن زیر host دیگر در boundary شبکه رد می‌شوند.

تست و مشارکت

تست پیش‌فرض کاملاً آفلاین است:

```
python -m pytest -m "not online"
```

یا:

```
make test
```

:است opt-in آنلاین محدود و smoke

```
python -m pytest -m online --timeout=30
```

تست آنلاین به وضعیت بازار/provider وابسته است و جایگزین تست deterministic آفلاین نیست. پیش از PR:

```
python -m pytest tests/test_release_offline.py -q
python -m pytest -m "not online" -q
```

برای مشارکت، issue یا pull request در GitHub باز کنید. انتشار PyPI فقط با مسیر دستی و تأیید صریح انجام می‌شود؛ target عادی `release` صرفاً artifact محلی می‌سازد و upload نمی‌کند.

مجوز و ارتباط

این پروژه تحت GNU General Public License v3 منتشر می‌شود.

- وب‌سایت: algotik.com
- تلگرام: t.me/algotik
- نویسنده: `alipour@algotik.ir` — Mohsen Alipour

تاریخچهٔ تغییرات

این بخش در هر انتشار به‌روزرسانی می‌شود؛ جزئیات کامل‌تر در HISTORY.rst ثبت شده است.

- **1.8.1 — 2026-10-25:** اصلاح `pct_cash` در `list_funds()` با منبع مستقل FIPIRAN و اتصال دقیق `reg_no`؛ حذف fallback نادرست `portfolioCash` تکرارشدهٔ TSETMC؛ ثبت پوشش و وضعیت منبع در attrs؛ اصلاح `record_date` از `recordDate` رسمی و افزودن تست‌های عدم تطبیق نامی، عدم inference و قطع منبع.
- **1.8.0 — 2026-09-13:** افزودن شناسایی و فیلتر مستقیم اراد/گام و سررسیدهای شش/هشت‌رقمی، YTM مرجع رسمی همهٔ اوراق بدون حدس جریان نقدی، و توسعهٔ TGJU به کاتالوگ ۸۷ سری شامل ۳۶ ارز آزاد، ۲۴ نرخ رسمی، طلای ۱۸/۲۴ عیار و سایر طلاها، نقره، سکه/حباب و فلزات جهانی با alias فارسی، slug رسمی، واحد و خطاهای typed.
- **1.7.0 — 2026-09-13:** افزودن مقایسهٔ چندنمادی بازده/ریسک/افت/بتا/نقدشوندگی و جریان حقیقی؛ سنجه‌های نقدشوندگی تاریخی و زنده با Amihud، گردش، spread و عمق پنج‌سطحی؛ رژیم توضیح‌پذیر بازار از پنج مؤلفه با confidence و رفتار صریح دادهٔ مفقود؛ داده و treemap نقشهٔ بازار با گروه‌بندی و معیارهای اندازه/رنگ قابل انتخاب؛ پشتیبانی ورودی DataFrame صریح برای schema/attrs، replay پایدار، واحدهای روشن و Plotly اختیاری.
- **1.6.0 — 2026-09-13:** افزودن حراج‌های بورس انرژی با شرایط کامل عرضه و تفکیک معاملات حراج از سابقهٔ ابزار؛ تابلوهای برق استاندارد/سبز/آزاد، سلف و گواهی‌ها و مشخصات آتی انرژی؛ تابلوی رسمی بورس کالا برای گواهی، سلف، صندوق و آتی با پنج سطح سفارش و حقیقی/حقوقی؛ تاریخچه و خلاصهٔ بازار فیزیکی و آمار تجمیعی بازارهای فیزیکی/آتی/اختیار/مالی؛ منحنی سررسید، calendar spread و cash-and-carry با فرض‌های صریح و ثبت دقیق واحد ریال/هزار ریال/میلیون ریال.
- **1.5.1 — 2026-09-13:** اصلاح هویت و `change_amount` در `get_shareholders(date=...)` بر پایهٔ اختلاف دو snapshot و تطبیق exact؛ تفکیک صریح تاریخ معامله از تاریخ مؤثر روز معاملاتی بعد؛ افزودن `get_shareholder_history()` با فرکانس روزانه/هفتگی/ماه شمسی و بودجهٔ سخت درخواست؛ افزودن رتبه‌بندی انباشت/توزیع، edge list شبکهٔ سهامدار–نماد و سنجه‌های Top-N/HHI تمرکز مالکیت با metadata کامل محدودیت‌های رسمی منبع.
- **1.5.0 — 2026-09-12:** تفکیک صریح تاریخچهٔ بلندمدت هر نماد از پنجرهٔ گردان حداکثر پنج‌تاریخهٔ بازار؛ افزودن snapshot، تغییرات متوالی و جمع‌بندی سهامداران فعال با اتصال دقیق `InsCode` و metadata محدودیت منبع؛ افزودن timeline بومی TSETMC برای پیام ناظر، تغییر سرمایه و تعدیل قیمت بدون برچسب‌گذاری ساختگی DPS یا ادغام دادهٔ کدال.
- **1.4.0 — 2026-09-12:** تجمیع تقویم و نمای کلان بازار با ابزارهای پیش‌گشایش؛ تقویم رسمی بورس/فرابورس، ارزش بازار تاریخی، فعالیت روزانه، اثر نمادها بر شاخص، خلاصهٔ bulk روز، مستر کامل ابزارها و diff مبتنی بر TOP، `InsCode` رسمی

مشترکان و عدم تعادل پیش‌گشایش.

- **1.3.0 — 2026-09-12:** طبقه‌بندی چندمحورهٔ صندوق‌ها، اصلاح شناسه‌های registry، افزودن پنج دستهٔ جدید، پوشش صندوق‌های بورسی نوع ۳۸۰، فیلتر طلا/نقره/زعفران و شواهد قابل ممیزی.
- **1.2.4 — 2026-09-02:** رویدادهای افزوده/حذف‌شدهٔ اعضای شاخص صنعت.
- **1.2.3 — 2026-09-01:** churn، تمرکز، مومنتوم، همسایگی همبستگی و امتیاز سلامت صنایع.
- **1.2.2 — 2026-09-01:** سنجش همپوشانی اعضای رسمی صنایع.
- **1.2.1 — 2026-09-01:** مقایسه، قدرت نسبی و همبستگی صنایع.
- **1.2.0 — 2026-09-01:** API تاریخچه، اعضا، شاخص‌های صنعت، کامل، intraday و رتبه‌بندی.
- **1.1.3 — 2026-08-28:** بازطراحی راهنمای PDF و هماهنگ‌سازی badge نسخه.
- **1.1.2 — 2026-08-28:** بازطراحی README و تکمیل قرارداد ورودی/خروجی توابع.
- **1.1.1 — 2026-08-25:** بهبود نمایش فارسی و تکمیل badgeهای پروژه.
- **1.1.0 — 2026-08-24:** دادهٔ زنده/تاریخی، SQLite، watcher، اخزا، منحنی بازده و اختیار معامله.
- **1.0.3 — 2026-07-12:** جست‌وجوی املای canonical نماد با نرمال‌سازی فارسی/عربی.
- **1.0.2 — 2026-03-24:** گسترش شاخص‌های صنعت و افزودن فهرست شاخص‌ها و اعضا.
- **1.0.1 — 2026-02-19:** تکمیل وابستگی‌ها و اصلاح نوع دادهٔ `NAV_Discount` .
- **1.0.0 — 2025-07-12:** انتشار پایدار با HTTP client مرکزی، تنظیمات و exceptionهای اختصاصی.
- **0.3.12 تا 0.2.8 — 2023 تا 2024:** نسخه‌های آغازین قیمت، سهامداران، افزایش سرمایه، اطلاعات نماد، ارز و فهرست ابزارها.